



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**CRAYCARE: AN IOT- AND MOBILE-BASED AQUACULTURE MONITORING  
SYSTEM FOR SMALL-SCALE CRAYFISH FARMING**

A Capstone Project

Presented to the Faculty of the College of Computer and Information Sciences  
Polytechnic University of the Philippines  
Sta. Mesa, Manila

In Partial Fulfilment of the Requirements for the Degree  
Bachelor of Science in Information Technology

by

**JUSTINE BONG V. MONTANTE  
JENNY ROSE G. MONTIPALCO  
BRYAN ACE A. NUITE  
DAVE Q. PETALLANO  
SOFIA ANGELA T. VILAN**

**2026**

**CAPSTONE PROJECT**



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                                               | <b>Page</b> |
|---------------------------------------------------------------|-------------|
| <b>Title Page</b>                                             | i           |
| <b>Table of Contents</b>                                      | ii          |
| <b>List of Tables</b>                                         | v           |
| <b>List of Figures</b>                                        | vii         |
| <br>                                                          |             |
| <b>1 INTRODUCTION</b>                                         |             |
| Project Context                                               | 1           |
| Technical Background                                          | 4           |
| Equipment/Hardware                                            | 4           |
| Software                                                      | 8           |
| Peopleware/Manpower                                           | 9           |
| Network Infrastructure/Architecture                           | 11          |
| Storage, Backup and Recovery Procedure                        | 13          |
| Security Procedures                                           | 17          |
| Policies and Procedures                                       | 21          |
| Problem Analysis                                              | 28          |
| Fishbone Diagram                                              | 29          |
| Problem and Solution Statement                                | 31          |
| Problem-Requirements Matrix                                   | 32          |
| Purpose and Description                                       | 34          |
| Specific Objectives                                           | 36          |
| Scope and Limitations                                         | 36          |
| Definition of Terms                                           | 40          |
| <br>                                                          |             |
| <b>2 REVIEW OF LITERATURE, STUDIES, AND SYSTEMS</b>           |             |
| IoT-Based Water Quality Monitoring Systems                    | 46          |
| IoT Systems with Mobile Application Integration               | 49          |
| Fuzzy Logic-Integrated Aquaculture Monitoring                 | 50          |
| Machine Learning and AI Integration in Aquaculture Monitoring | 51          |
| Cost-Effective and Locally Adapted IoT Solutions              | 52          |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                                                                  |     |
|----------------------------------------------------------------------------------|-----|
| Wireless Sensor Networks and Remote Monitoring in Aquaculture                    | 53  |
| IoT Monitoring for Specific Aquaculture Species                                  | 54  |
| Recirculating Aquaculture Systems (RAS) and the Potential for Smart Technologies | 56  |
| Growth and Molting Mechanisms of Crayfish                                        | 57  |
| Morphological Development and Reproductive Characteristics                       | 58  |
| Crayfish Farming Practices and Production Performance                            | 60  |
| Synthesis of the Study                                                           | 61  |
| <b>3 METHODOLOGY</b>                                                             |     |
| Requirements Analysis                                                            | 66  |
| Requirements – Features Matrix                                                   | 67  |
| Use Case Diagram                                                                 | 73  |
| Use Case Report                                                                  | 76  |
| Design Specifications                                                            | 86  |
| Activity Diagram                                                                 | 87  |
| GUI Design                                                                       | 111 |
| Hardware Sketch and Setup                                                        | 146 |
| Database Schema                                                                  | 149 |
| Data Dictionary                                                                  | 151 |
| Development Methodology                                                          | 171 |
| Process Model                                                                    | 171 |
| Development Tools                                                                | 174 |
| Test Methodology/Procedures                                                      | 176 |
| System Requirements                                                              | 177 |
| Quality Plan                                                                     | 179 |
| Implementation Plan                                                              | 180 |
| Evaluation Plan                                                                  | 182 |
| Ethical Considerations                                                           | 183 |
| Data Analysis (Procedure and Treatment)                                          | 184 |
| Statistical Treatments                                                           | 186 |

## CAPSTONE PROJECT



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**REFERENCES**

190

**CAPSTONE PROJECT**



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**LIST OF TABLES**

| <b>Table No.</b> | <b>Description</b>                                                                | <b>Page</b> |
|------------------|-----------------------------------------------------------------------------------|-------------|
| 1.1              | List of Hardware Components for the CrayCare System                               | 4           |
| 1.2              | Role-Based Access Control (RBAC) Permissions                                      | 19          |
| 1.3              | Problem Requirements Matrix of CrayCare                                           | 33          |
| 3.1              | Functional Requirements Matrix                                                    | 67          |
| 3.2              | Non-Functional Requirements Matrix                                                | 71          |
| 3.3              | Use Case Report                                                                   | 76          |
| 3.4              | Crayfish Size and Weight Classification Reference                                 | 126         |
| 3.5              | Temperature Ranges                                                                | 137         |
| 3.6              | pH Ranges                                                                         | 138         |
| 3.7              | Dissolved Oxygen Ranges                                                           | 138         |
| 3.8              | Turbidity Ranges                                                                  | 138         |
| 3.9              | Water Level or Water Depth Ranges                                                 | 139         |
| 3.10             | Data Dictionary of Collection: /users/\$uid/profile                               | 152         |
| 3.11             | Data Dictionary of Collection: /users/\$uid/growth_stage                          | 153         |
| 3.12             | Data Dictionary of Collection: /users/\$uid/notifPrefs                            | 154         |
| 3.13             | Data Dictionary of Collection: /users/\$uid/notifications                         | 155         |
| 3.14             | Data Dictionary of Collection: /users/\$uid/fcmToken                              | 155         |
| 3.15             | Data Dictionary of Collection: /sensor_readings/latest                            | 156         |
| 3.16             | Data Dictionary of Collection: /sensor_readings/history                           | 157         |
| 3.17             | Data Dictionary of Collection: /sensor_readings/config                            | 158         |
| 3.18             | Data Dictionary of Collection:<br>/sensor_readings/config/\$stageName/\$sensorKey | 158         |
| 3.19             | Data Dictionary of Collection: /devices/modes                                     | 159         |
| 3.20             | Data Dictionary of Collection: /devices/logs/\$deviceId                           | 160         |
| 3.21             | Data Dictionary of Collection: /feeder/commands                                   | 161         |
| 3.22             | Data Dictionary of Collection: /feeder/status                                     | 162         |
| 3.23             | Data Dictionary of Collection: /feeder/schedules                                  | 163         |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|        |                                                                              |     |
|--------|------------------------------------------------------------------------------|-----|
| 3.24   | Data Dictionary of Collection: /feeder/logs                                  | 164 |
| 3.25   | Data Dictionary of Collection:<br>/feeder/dispatched/\$dateKey/\$scheduleKey | 164 |
| 3.26   | Data Dictionary of Collection: /tank_data/\$ownerUid/inventory               | 165 |
| 3.27   | Data Dictionary of Collection: /tank_data/\$ownerUid/sampling                | 166 |
| 3.28   | Data Dictionary of Collection: /tank_data/\$ownerUid/mortality               | 167 |
| 3.29   | Data Dictionary of Collection: /tank_data/\$ownerUid/logs                    | 167 |
| 3.30   | Data Dictionary of Collection: /ml_predictions/latest                        | 168 |
| 3.30.1 | Per-Sensor Object Structure (within the sensors array)                       | 169 |
| 3.31   | Data Dictionary of Collection: /system/authorizedOperators                   | 170 |
| 3.32   | Development Tools                                                            | 174 |
| 3.33   | System Requirements for the CrayCare System                                  | 177 |



## LIST OF FIGURES

| Figure No. | Description                                                     | Page |
|------------|-----------------------------------------------------------------|------|
| 1.1        | Network Architecture of the CrayCare System                     | 11   |
| 1.2        | Level 0 Data Flow Diagram (DFD) of the Proposed CrayCare System | 23   |
| 1.3        | Level 1 Data Flow Diagram (DFD) of the CrayCare System          | 26   |
| 1.4        | Factors Affecting Crayfish Production and Management            | 29   |
| 3.1        | Use Case Diagram                                                | 73   |
| 3.2        | Activity Diagram for Open Mobile Application                    | 87   |
| 3.3        | Activity Diagram for View Real-Time Dashboard                   | 89   |
| 3.4        | Activity Diagram for Control Hardware Devices                   | 91   |
| 3.5        | Activity Diagram for Manage Feeding System                      | 93   |
| 3.6        | Activity Diagram for Receive Alerts and Notifications           | 95   |
| 3.7        | Activity Diagram for View Analytics and Historical Data         | 97   |
| 3.8        | Activity Diagram for Manage Tank Grow-out — Inventory           | 99   |
| 3.9        | Activity Diagram for Manage Tank Grow-out — Sampling            | 101  |
| 3.10       | Activity Diagram for Manage Tank Grow-out — Growth Trend        | 103  |
| 3.11       | Activity Diagram for Manage Water Thresholds                    | 104  |
| 3.12       | Activity Diagram for Manage Personal Account                    | 106  |
| 3.13       | Activity Diagram for Manage Notification Preferences            | 107  |
| 3.14       | Activity Diagram for Manage User Accounts                       | 109  |
| 3.15       | Splash Screen                                                   | 111  |
| 3.16       | User Authentication Screens                                     | 112  |
| 3.17       | Dashboard Module Overview                                       | 113  |
| 3.18       | Parameter Detail Modal                                          | 114  |
| 3.19       | Analytics Module                                                | 115  |
| 3.20       | Chart Detail Dialog                                             | 116  |
| 3.21       | Analytics Timeframe Filters                                     | 118  |
| 3.22       | Tank Module — Inventory Tab                                     | 119  |
| 3.23       | Tank Module — Sampling Tab                                      | 120  |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|      |                                               |     |
|------|-----------------------------------------------|-----|
| 3.24 | Sampling Tab Modal Overlays                   | 123 |
| 3.25 | Tank Module — Trends Tab                      | 124 |
| 3.26 | Growth Classification Reference Modal         | 125 |
| 3.27 | Controls Module — Feeding Tab                 | 129 |
| 3.28 | Controls Module — Devices Tab                 | 130 |
| 3.29 | Notifications Module                          | 131 |
| 3.30 | Profile & Settings Menu                       | 132 |
| 3.31 | Edit Profile Sub-Screen                       | 133 |
| 3.32 | Change Password Sub-Screen                    | 134 |
| 3.33 | Notifications Preferences Sub-Screen          | 135 |
| 3.34 | Crayfish Stage & Threshold Settings Interface | 136 |
| 3.35 | CrayAI Analytics ML Predictions               | 140 |
| 3.36 | Role-Based Monitor View Interface             | 142 |
| 3.37 | Admin Module and Settings Interface           | 144 |
| 3.38 | System Architecture                           | 146 |
| 3.39 | Hardware Setup Sketch                         | 148 |
| 3.40 | Database Schema of CrayCare                   | 149 |
| 3.41 | Waterfall Model                               | 172 |





## CHAPTER 1

### INTRODUCTION

#### 1.1 PROJECT CONTEXT

Aquaculture plays an important role in food production and livelihood in the Philippines. In recent years, crayfish farming has gained attention as an emerging aquaculture activity due to its economic potential and increasing demand. Still, keeping the right water conditions is crucial for crayfish survival and healthy growth. Parameters like temperature, pH, turbidity, and dissolved oxygen need regular checks to maintain those ideal settings.

Many small-scale farms often rely on manual monitoring with simple tools and visual inspection. Such approaches tend to be slow, unreliable, and error-prone. Any delays or mistakes in readings can lead to poor water quality, harming crayfish health and survival rates.

IoT technology has changed that picture, making real-time monitoring more feasible and affordable. These setups use sensors for nonstop data gathering and allow instant access to information through mobile devices. Overall, such systems have the potential to boost efficiency, precision, and smarter choices in aquaculture management.

According to existing laws in the Philippines, such as **Republic Act No. 8550**, known as the Philippine Fisheries Code of 1998 (as amended by **Republic Act No. 10654**), aquaculture includes the raising and culture of aquatic organisms in fresh, brackish, or marine waters. This law provides a clear definition of aquaculture and



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

highlights government efforts to support the fishery sector through technology and research. According to Section 47, the Department of Agriculture must establish a code of practice for aquaculture farms so that these can be constructed and operated according to environmental safety standards. Moreover, the introduction of pollutants into Philippine waters, as stated in Section 102 of the same law, is prohibited when there is a possibility of harm to aquatic life. On the other hand, Section 48 encourages sustainable practices in aquaculture farms by providing financial and other incentives, as well as imposing penalties on activities that go against these objectives.

Additionally, specific guidelines for farming Australian Redclaw Crayfish (*Cherax quadricarinatus*) in the Philippines are provided under **BFAR Administrative Circular No. 001**, Series of 2025. These regulations include the requirement to use clean water sources located away from pollution and to maintain records of water quality to support responsible farm management.

Related studies have proven the effectiveness of IoT technologies for improving aquaculture water quality monitoring, matching the objectives of this study. For instance, **Cheng et al. (2025)** designed an intelligent aquaculture system that uses IoT sensors and computer vision for the cultivation of Australian Blue Crayfish (*Cherax quadricarinatus*) in fiber tanks. This framework uses IoT sensors to track water quality and uses local edge devices to process the data and connect with the cloud. This system allows real-time processing of data and has an accuracy of 87.4% for density prediction, enabling IoT applications for crayfish monitoring in controlled environments. Similarly, **Ahmed et al. (2024)** introduced an IoT-based system for shrimp farms, noting that traditional manual monitoring systems are cumbersome, time-consuming, and lacking real-time capabilities.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The microcontroller and sensor-based system allows continuous measurement of pH levels, temperature, TDS, EC, and salinity, including machine learning predictive model ( $r^2=0.94$ ), cloud integration, and notifications via a web app, resulting in a 97.84% accuracy in categorizing production using Random Forest method. These findings suggest the feasibility of applying sensors, mobile access, and automation for small-scale farming contexts.

This study introduced CrayCare, a smart aquaculture solution that integrated IoT and mobile-based technologies into a Recirculating Aquaculture System (RAS) for small-scale crayfish farming. The experimental setup consisted of two grow-out tanks, each operating under a separate RAS. Tank A utilized the CrayCare system for monitoring water parameters and implementing automated responses, while Tank B used manual monitoring and management practices. Data collected from Tank A were transmitted to the mobile application for real-time monitoring and system management.

As there was no specific client for the system, the researchers established an experimental crayfish culture setup to simulate real-world aquaculture conditions. Both tanks were maintained under comparable baseline conditions to provide a basis for comparison between IoT-based and manual management approaches.

The study employed a comparative experimental design to evaluate the monitoring efficiency and overall system performance of CrayCare against manual management. The findings were used to assess the applicability of IoT-based technologies in small-scale crayfish farming.



## 1.2 TECHNICAL BACKGROUND

This section provides the technical background of the CrayCare system, describing the environment in which the system was developed and implemented. It includes the hardware and software, peopleware/manpower, network architecture, data storage and recovery procedure, security and operational policies. All these aspects formed the technological and operational architecture that supported the development, implementation, and evaluation of the IoT-based aquaculture monitoring system.

### 1.2.1 Equipment/Hardware

Table 1.1

**List of Hardware Components for the CrayCare System**

| Equipment                                                | Quantity |
|----------------------------------------------------------|----------|
| ESP32 Development Board                                  | 1        |
| DFRobot Gravity Analog Dissolved Oxygen Sensor (SEN0237) | 1        |
| DFRobot Gravity Analog pH Sensor (SEN0161)               | 1        |
| DFRobot Gravity Analog Turbidity Sensor (SEN0189)        | 1        |
| DS18B20 Waterproof Temperature Sensor                    | 1        |
| HC-SR04 Ultrasonic Sensor (Water Level Monitoring)       | 1        |
| 16×2 I2C LCD Display                                     | 1        |

*Continuation of Table 1.1*

| Equipment                      | Quantity |
|--------------------------------|----------|
| Servo Motor (Automatic Feeder) | 1        |
| 4-Channel Relay Module         | 1        |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                                               |          |
|---------------------------------------------------------------|----------|
| Breadboard                                                    | 1        |
| Jumper Wires (Male-to-Male, Male-to-Female, Female-to-Female) | Multiple |
| Water Pump                                                    | 1        |
| Air Pump (Aerator)                                            | 2        |
| Grow-out Tank (Tarpaulin-made Container)                      | 2        |
| PVC Pipes (Hideouts and Water Distribution)                   | Multiple |
| Mesh Materials                                                | Multiple |
| MicroSD Card Module                                           | 1        |
| MicroSD Card (16GB or 32GB)                                   | 1        |
| Wi-Fi Router / Internet Source                                | 1        |
| Power Supply Unit                                             | 1        |
| Android Smartphone                                            | 4        |
| Laptop                                                        | 2–3      |

Table 1.1 presents the hardware components and equipment that will be utilized in the development of the proposed CrayCare system, along with the specified quantities. The listed hardware includes the main processing unit, environmental sensors, actuators, networking equipment, computing devices, mobile devices, and supporting materials needed for system development, data collection, control operations, and overall system functionality.

The ESP32 microcontroller will serve as the central processing unit of the proposed system. It will be responsible for collecting data from connected sensors, processing monitoring information, and controlling automated hardware components. The system will utilize sensors for measuring important water



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

conditions, including temperature, pH, turbidity, dissolved oxygen, and water level, to support continuous monitoring of the crayfish culture environment.

The proposed system will also include various actuator components to support automated responses. These components include relay modules, aeration devices, a water pump, and a servo motor feeder. The relay module will serve as a switching component that allows the ESP32 microcontroller to control connected electrical devices by turning them on or off based on the configured system conditions. The aeration devices will support oxygen management, the water pump will assist in water circulation, and the servo motor feeder will support automated feeding operations.

Networking equipment will be necessary to enable communication between the ESP32 microcontroller, cloud database, and mobile device. A Wi-Fi-enabled network will be used to transmit collected data and allow remote access to system information. Since the ESP32 has built-in Wi-Fi capability, additional wireless communication hardware will not be required.

In addition to the main hardware components, supporting materials will be utilized to assist in the assembly, integration, and operation of the proposed CrayCare system. Electronic materials such as the breadboard and jumper wires will be used for connecting and organizing the hardware components during development. The MicroSD card module and MicroSD card will support local data storage functions by providing additional storage capacity for recorded system information. The display component will be used to present selected system



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

information, such as monitoring readings and controls status, through a dedicated visual interface. Tank-related materials such as the grow-out tanks, PVC pipes, and mesh materials will be used to establish the physical environment required for crayfish cultivation and support the preparation of the experimental setup. The power supply unit will provide the necessary electrical power for the operation of the connected components.

Aside from that, the proposed system will utilize multiple Android-based mobile devices as part of the hardware components for system operation and testing. These mobile devices will serve as portable access devices for viewing system information and interacting with the CrayCare setup.

Also, multiple laptop units will be utilized to support different project activities, including mobile application development, ESP32 microcontroller programming, and documentation tasks. The laptops should have sufficient processing capability, memory, and storage capacity to support the required development tools and applications. Since the proposed system focuses on IoT-based monitoring and control operations, standard laptop specifications are sufficient for the development process.

The hardware components selected for the proposed CrayCare system are intended to be suitable for small-scale crayfish cultivation environments. The system will utilize accessible and practical hardware to support monitoring, automation, and data management without requiring complex infrastructure.



### 1.2.2 Software

This subsection describes the software tools, platforms, and operating systems required for the development and operation of the CrayCare system. Since the study did not involve a specific client or existing software environment, the selected software components were based on the requirements of the system development and implementation.

The project team utilized the Windows operating system as the primary development environment for creating and managing the CrayCare system. The CrayCare mobile application was designed for the Android operating system, allowing users to access monitoring information, system records, and available control functions through an Android-based mobile device.

Visual Studio Code was used as the primary development environment for writing, editing, and managing source codes for both the CrayCare mobile application and ESP32 microcontroller programming. It was selected due to its lightweight performance, extensive extension support, and capability to manage different programming tasks within a single environment.

PlatformIO was integrated with Visual Studio Code to support the development of the ESP32-based components. It was used for firmware development, library management, code compilation, and uploading of program codes to the microcontroller for sensor monitoring and automated control functions.

For data management, the system utilized Cloud Firestore as the cloud-based database for storing sensor data, system records, and other information





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

generated by the CrayCare system. Firebase Authentication was also used to manage authorized access to the mobile application.

These software tools and technologies provided the necessary environment for developing and integrating the Android application, ESP32 microcontroller, cloud database, and connected hardware components of the CrayCare system.

### **1.2.3 Peopleware/Manpower**

This subsection presents the people involved in the development, implementation, and operation of the CrayCare system. The project team was divided into different roles to organize the tasks needed for the project. These roles included the Project Manager, Mobile App Developer, IoT Programmer, Hardware Engineer, and Documentors.

The Project Manager handled the overall project activities, assigned tasks to the team members, monitored the progress of the project, and made sure that the project requirements and schedule were followed. The Mobile App Developer was responsible for developing the CrayCare mobile application, including its interface, monitoring features, control functions, and connection to the cloud database. The IoT Programmer was in charge of programming the ESP32 and handling the collection and transmission of sensor data, as well as the automated system functions. The Hardware Engineer worked on installing and connecting the sensors, actuators, ESP32, and other hardware components and assisted in testing and troubleshooting the system. The Documentors prepared the project



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

and research documents, including system documentation, testing records, evaluation results, and the final research manuscript.

Since the study did not have a specific client, the project team also served as the actual users of the system during the experiment. The project team used the mobile application to monitor sensor readings, view system records, and perform the available system operations. The team also handled the manual monitoring and other activities needed for the experimental setup. Small-scale crayfish farmers were considered the target users of the system, but no farmers directly used the system during the study.

### 1.2.4 Network Infrastructure/Architecture

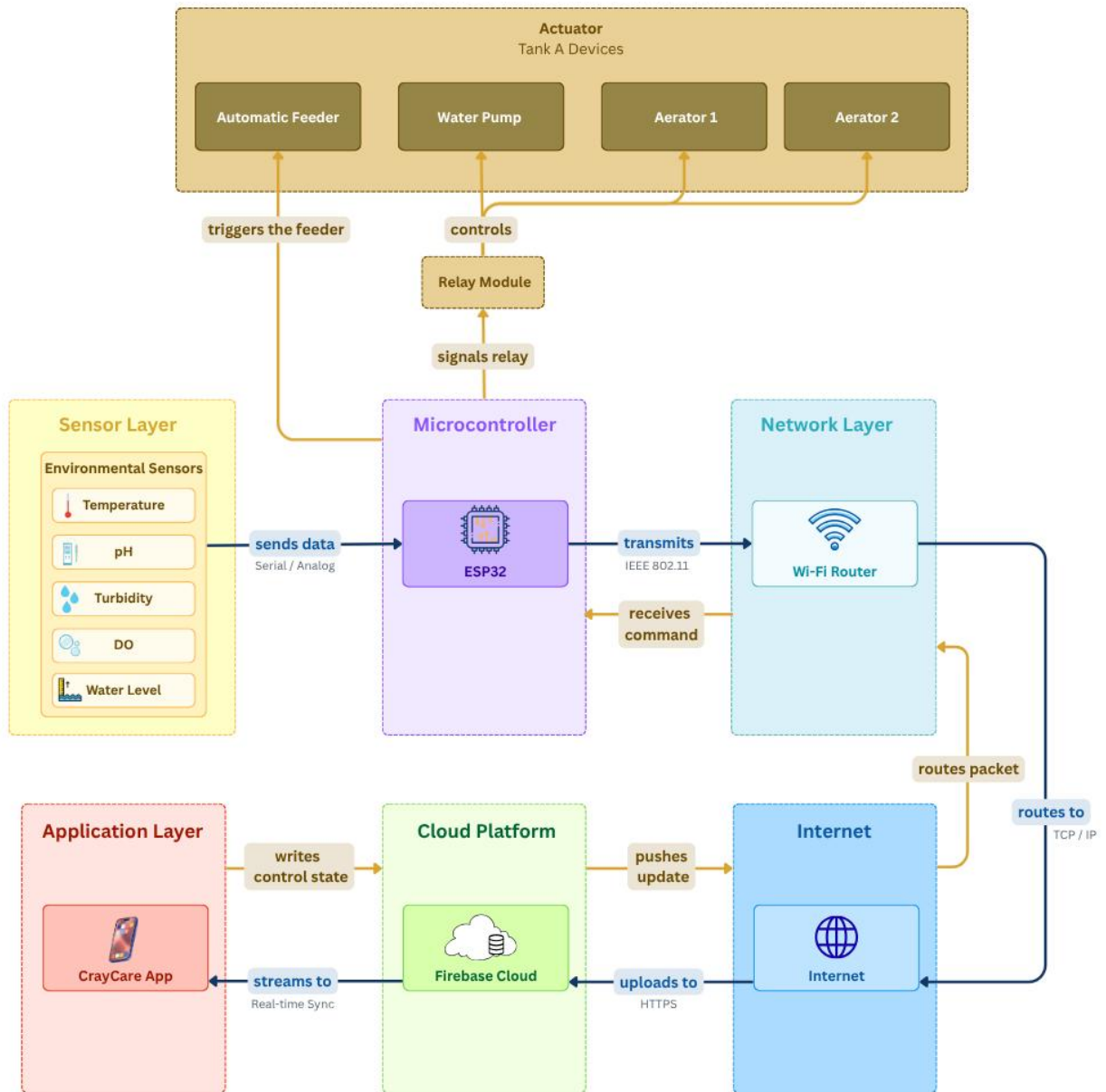


Figure 1.1. Network Architecture of the CrayCare System



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 1.1 presents the network architecture of the CrayCare system and shows how the system components communicate with one another. The sensors collect water parameter data and send the readings to the ESP32 microcontroller. The ESP32 transmits the data through the Wi-Fi router and Internet to the Firebase cloud platform, where the system data are stored in Cloud Firestore. The CrayCare mobile application retrieves the data from the cloud for real-time monitoring and can also send commands back through the same network. The ESP32 then uses the received commands to control the actuators, including the automatic feeder, water pump, and air pumps.

### **1.2.5 Storage, Backup and Recovery Procedure**

This subsection outlines the mechanisms for data storage, protection, and recovery in the CrayCare system. Given the nature of the study as a comparative experimental and research-oriented project with no specific client, all storage and backup strategies are designed to align with the operational requirements of a small-scale IoT-based aquaculture monitoring environment.

#### **Data Storage**

Firebase will serve as the primary repository for data generated by the CrayCare system. For Tank A (IoT Grow-out Tank), the stored data will include sensor logs for temperature, pH, turbidity, dissolved oxygen, and water level; water quality status classifications and alert levels; automated response logs such as aeration activation events, water circulation events, and feeding skip events with corresponding reasons and timestamps; scheduled and executed feeding records; feeding skip records; and manually entered crayfish growth measurements. For



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Tank B (Manual Grow-out Tank), the stored data will include manually entered water quality readings, feeding logs, and crayfish growth measurements. Since Tank B functions as a manual comparison unit, it will not generate automated sensor data.

To improve reliability during network interruptions, the proposed system may incorporate a microSD card module for local data storage. Sensor readings, system events, and operational logs that cannot be transmitted to Firebase due to connectivity issues will be temporarily stored on the microSD card. Once network connectivity is restored, the ESP32 will attempt to synchronize the locally stored records with Firebase, ensuring that historical data is preserved and minimizing data loss during communication outages.

In addition to cloud storage, the Android mobile application will maintain a temporary local cache of the most recent information retrieved from Firebase. This cache is intended to improve usability during short periods of network interruption and is not designed to function as a permanent data storage mechanism.

### **Backup Strategy**

Firebase Realtime Database has built in redundancy and automatic replication of data across multiple servers managed by Google. Data stored on the platform is automatically replicated in geographically separated infrastructure, which notably reduces the risk of all the data being lost in a single point of failure. At the application level, it is not necessary to have any additional manual backup configuration to achieve basic redundancy.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

To facilitate research and documentation purposes, it is proposed to periodically make manual exports of Firebase data at regular intervals - at least once per week during the course of the experiments. These exports will be in the form of JSON files and are to be stored in at least two different locations: a dedicated folder in a cloud storage service like Google Drive, and a local backup drive that will be maintained by the research team. This dual-location backup solution can help to safeguard experimental data in the event of either accidental deletion or account management issues in the Firebase project.

To further support the backup strategy beyond manual exports, it is offered to use Google Apps Script as an automated tool to export Firebase Realtime Database data directly to a specially designated folder in Google Drive on a scheduled basis. Google Apps Script runs fully on Google servers, and does not need any more software to be installed, or any third-party dependencies. As Firebase and Google Apps Script belong to the same Google ecosystem, the integration of the two platforms is both straightforward and reliable. The tool is free of any extra charge and is compatible with a standard 15 GB free storage allocation within Google Drive - which is considered to be enough to store system data anticipated to be produced during the experimental period.

The automated backup script will execute the following operations: fetch the current state of the Firebase Realtime Database using a URL fetch request, convert the retrieved data into a file in the form of a JSON, append a timestamp to the file in order to identify the version, and save the resulting file to a pre-specified Google Drive folder. A format of a file name like `Firestore_Backup_YYYY-MM-`



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

DD\_HHmm.json should be used, and every file of the backups should be uniquely identified by date and time of creation and supporting orderly version management throughout the experimental period.

Google Apps Script's internal Triggers functionality handles scheduling, allowing executing Google Apps Scripts by time, without manual operation. The backup functionality is suggested to operate automatically on a daily basis - preferably during low-activity periods like midnight to 1:00 AM - to reduce the possible impact on active data transmission performed by the ESP32 microcontroller. Admittedly, Google Apps Script is limited to the maximum six-minute run time. Since CrayCare operates within a small-scale experimental setup with a limited number of tanks and a defined set of monitored parameters and records, the total volume of data per backup cycle is not expected to exceed this limit. Regular checking of the size of backup files during the study period is however recommended to maintain script reliability.

### **Recovery Procedure**

To recover data in the event of accidental data loss or corruption in Firebase, data recovery is to be performed by restoring it to the latest available backup. The main source of recovery will be the automatically generated daily JSON backup files in the designated Google Drive folder which are generated through the use of Google Apps Script automation. In the situations, when the automated execution of the backup was not performed successfully, the second source of recovery is the periodically-made manual exports. Backup files versioned with label should be kept so that the appropriate dataset could be



located and restored immediately. Re-importation of recovered JSON file to Firebase is to be performed via the import facility of Firebase console that can support the direct upload of the recovered JSON file. The in-built data persistence capabilities provided by Firebase should also be able to facilitate partial recovery in cases involving short interruption of services.

In case the ESP32 microcontroller undergoes a reset or power interruption, the system is designed to automatically resume data transmission when it is reconnected on the Wi-Fi network, the Firebase will again start recording the incoming data in real time. The historical data stored before the interruption is still stored in the cloud database and is not affected by disruptions on the hardware side.

In the case of the Android mobile application, information presented in the application is directly sourced off of Firebase and does not need to have a special recovery mechanism, since the cloud database will be the single source of truth behind all system records.

#### **1.2.6 Security Procedures**

In this subsection, the security measures which are suggested to be applied to the CrayCare system are outlined. While the data handled by the system — such as water quality and water level sensor readings and feeding schedule logs — is not considered sensitive or confidential in nature, basic security measures are still deemed necessary. The main security risk of the CrayCare system is not the confidentiality of data but the prevention of





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

unauthorized access to system controls, especially to the automated feeding mechanism, and the preservation of data integrity throughout the experimental period.

### **Authentication and Access Control**

Firebase Authentication is proposed as the primary access control mechanism for the CrayCare system. Registered users will be required to authenticate using an email address and password before accessing system resources. This approach is intended to ensure that only authorized individuals can access the system during the conduct of the study.

The proposed system will implement role-based access control (RBAC) to regulate user permissions. An Administrator account, managed by the research team, will be responsible for user account management and permission assignment. The Administrator interface will be separate from the mobile application and will be used solely for assigning, modifying, or revoking user access privileges.

The CrayCare mobile application will support two user roles: Owner and Monitor User. Users assigned the Owner role will have permission to view sensor readings, water quality status, alerts, historical records, and other system information. In addition, Owners will be permitted to operate authorized devices such as the aerators, water pump, and automatic feeder, as well as configure system settings including sensor parameter ranges and threshold values used for automated monitoring and control functions.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Users assigned the Monitor User role will be limited to viewing sensor readings, water quality status classifications, alerts, historical records, and other monitoring information. Monitor Users will not be permitted to control devices or modify system settings.

Table1.2

### Role-Based Access Control (RBAC) Permissions

| Permission                    | Monitor User | Owner | Administrator |
|-------------------------------|--------------|-------|---------------|
| View Sensor Data              | ✓            | ✓     | ✗             |
| View Alerts and Notifications | ✓            | ✓     | ✗             |
| View Alerts and Notifications | ✓            | ✓     | ✗             |
| Control Aerators              | ✗            | ✓     | ✗             |
| Control Water Pump            | ✗            | ✓     | ✗             |
| Control Water Pump            | ✗            | ✓     | ✗             |
| Configure Sensor Thresholds   | ✗            | ✓     | ✗             |
| Manage User Accounts          | ✗            | ✗     | ✓             |
| Assign User Permissions       | ✗            | ✗     | ✓             |

Firebase Security Rules will be configured to enforce these permissions by controlling access based on authenticated user roles. Any request originating from an unauthenticated user or from an account lacking sufficient privileges will be



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

automatically denied, helping prevent unauthorized access, modification of settings, or control of system resources.

### **Data Transmission Security**

All the data exchanged between the ESP32 microcontroller, Firebase cloud database, and Android mobile application are carried out using HTTPS, which encrypts data during transmission. This helps prevent the interception and modification of sensor values and control commands during network communication.

### **Network Security**

It is suggested that the Wi-Fi network employed in the experimental setup should be secured with the WPA2 encryption and a strong password. This will ensure that the unauthorized device does not get access to the same local network as the ESP32 microcontroller since it might otherwise expose the system to undesired interference.

### **Device Security**

It is suggested that the Android smartphone on which the CrayCare mobile application is hosted be enabled with a screen lock to prevent the unauthorized physical access. There is no further device level security measures beyond the normal smart mobile protection practice that is deemed necessary given that the data at risk is non sensitive.

### **Overall Security Approach**



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The security solutions that will be suggested to CrayCare are deliberately kept simple and practical, given that the data that will be handled is not confidential. The emphasis is made on three fundamental areas of concern: making sure that unauthorized parties are not allowed to access and operate the system, protecting the integrity of sensor data during the experiment, and preventing unauthorized control over the automated feeding mechanism. It is believed that these measures are adequate and suitable in a small-scale research deployment of this kind.

### **1.2.7 Policies and Procedures**

The proposed CrayCare system shall follow defined policies and procedures that will guide system usage, data management, security, and operation. These guidelines will serve as a basis for managing user access, handling system information, and maintaining proper system processes.

The system shall implement user authentication to allow access only to authorized users. User permissions will be considered in managing available features and protecting stored system information.

The proposed system shall follow procedures for collecting, recording, and managing aquaculture-related data. Information gathered from the monitoring components will be processed and stored for access through the CrayCare mobile application.

Automated functions shall operate based on predefined conditions configured within the system to support selected aquaculture operations.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The Data Flow Diagram (DFD) will illustrate the movement of information within the proposed CrayCare system, including the flow of data between users, system components, and stored records. The proposed CrayCare system will use two levels of DFD, namely DFD Level 0 (Context Diagram) and DFD Level 1 (Detailed Process Diagram), which provide different levels of representation of the system flow.

The DFD Level 0 presents the overall view of the proposed CrayCare system. It illustrates the interaction between external entities and the main system process, including the flow of monitoring data from sensors, user inputs from the mobile application, and control outputs sent to connected hardware components. This diagram provides a general representation of how users, monitoring devices, and actuators interact with the CrayCare system. The DFD Level 0 diagram is presented on the next page.



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**CAPSTONE PROJECT**



# POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

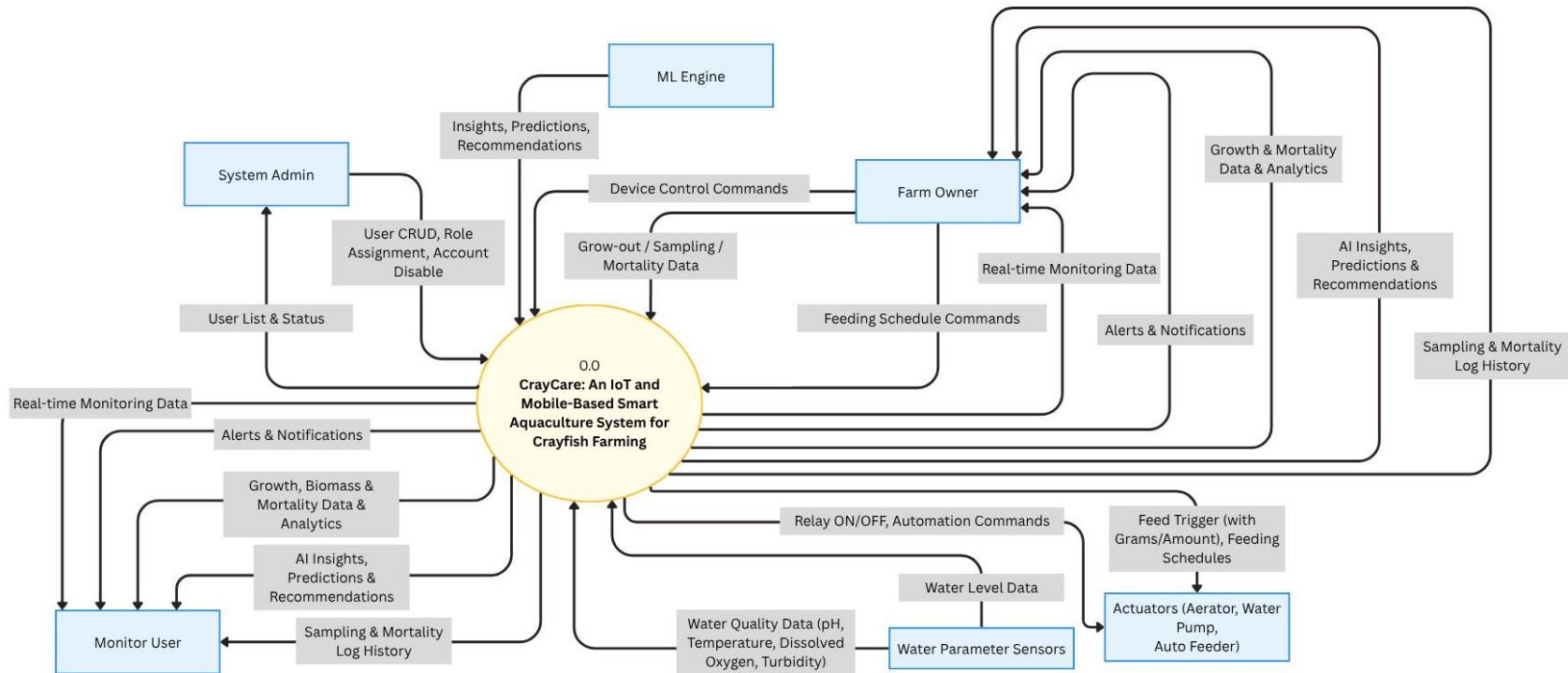


Figure 1.2. Level 0 Data Flow Diagram (DFD) of the Proposed CrayCare System



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 1.2 shows the Level 0 Data Flow Diagram (DFD) of CrayCare: An IoT and Mobile-Based Smart Aquaculture System for Crayfish Farming. The diagram presents the overall data flow between the system and its six external entities: Farm Owner, Monitor User, System Admin, Water Parameter Sensors, Actuators, and ML Engine. The Owner User serves as the primary user, providing Device Control Commands, Grow-out, Sampling, and Mortality Data, and Feeding Schedule Commands (specifying time and feed amount in grams) to the system. The Monitor User, on the other hand, only receives data from the system without sending any input commands. The System Admin manages user accounts by sending User CRUD, Role Assignment, and Account Disable commands, and receives User List and Status in return.

The Water Parameter Sensors continuously supply the system with real-time environmental readings including pH, Temperature, Dissolved Oxygen, Turbidity, and Water Level Data. The Actuators, consisting of the Aerator, Water Pump, and Auto Feeder, receive Relay ON/OFF, Automation Commands, Feed Trigger, and Feeding Schedules from the system to enable automated tank operation. The ML Engine provides the system with Insights, Predictions, and Recommendations to support intelligent decision-making. Both the Fa and Monitor User receive Real-time Monitoring Data, Alerts and Notifications, Growth, Biomass and Mortality Data and Analytics, AI Insights, Predictions and Recommendations, and Sampling and Mortality Log History from the system.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

While the Level 0 DFD presents the general flow of information within the proposed system, the Level 1 Data Flow Diagram provides a more detailed presentation of the internal processes of the proposed CrayCare system. It shows how sensor data is collected, processed, stored, and displayed through the mobile application. It also presents how system conditions and user commands are processed to activate connected actuators, such as aeration, water circulation, and feeding mechanisms. The DFD Level 1 diagram is shown on the next page.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 1.3 shows the Level 1 Data Flow Diagram (DFD) of CrayCare: An IoT and Mobile-Based Smart Aquaculture System for Crayfish Farming. This diagram breaks down the primary system into six external entities, eight internal processes, and five data stores, along with the detailed data flows between them. It provides a clearer understanding of how data moves and is processed within the system.

The system interacts with six external entities. The Farm Owner serves as the primary user who provides Grow-out Setup Data, Sampling Data, Mortality Logs, Device Control Commands, and Profile Information. The Monitor User has view-only access and receives Real-time Monitoring Data, Notifications, Tank Data, Device Status, Analytics Charts, and Growth Trends. The System Admin interacts exclusively with Process 8.0 for user account management. The Water Parameter Sensors continuously send real-time readings of Temperature, pH Level, Dissolved Oxygen, Turbidity, and Water Level. The Actuators (Aerator, Water Pump, and Auto Feeder) receive Automation Commands and Feeding Schedules from the system and return Status Feedback. The ML Engine supplies Insights, Predictions, and Recommendations to Process 5.0 to support intelligent analytics.

There are eight key internal processes in the system. Process 1.0 (Dashboard and Real-Time Monitoring) collects sensor data, stores it in D2, and displays real-time monitoring information and feeding schedules. Process 2.0 (Tank and Grow-out Management) handles grow-out setup, sampling data, and mortality logs. Process 3.0 (Controls and Automation) manages device control commands, automation rules, and feeding schedules. Process 4.0 (Notification



and Alerting) processes and sends alerts and notifications. Process 5.0 (Analytics and Reporting) generates gauges, charts, statistics, and utilizes ML insights. Process 6.0 (Settings and Profile Management) handles user profile information. Process 7.0 (Growth Trends and Analytics) produces growth, biomass, and mortality charts. Process 8.0 (User Management) allows the System Admin to view, filter, change roles, and disable user accounts.

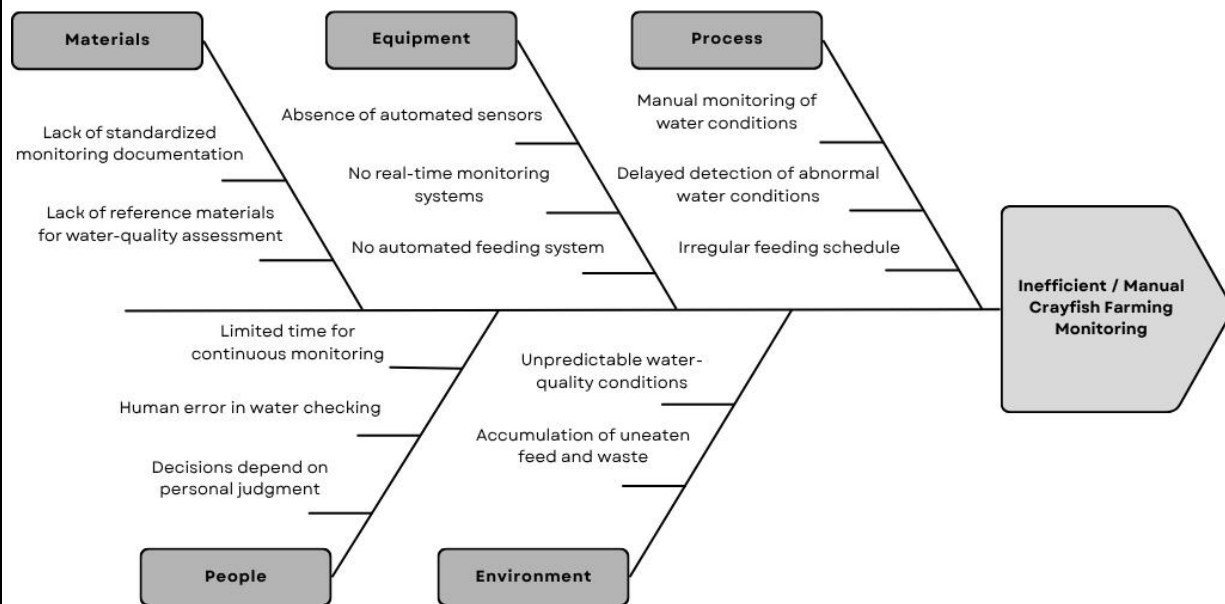
The system maintains five data stores. Growout Data (D1) stores all grow-out records including sampling history, mortality logs, and computed metrics. Sensor Data (D2) holds current and historical sensor readings. Feeder and Hardware Logs (D3) keeps feeding schedule configurations and activity logs. Notifications (D4) stores notification records with types, titles, messages, and read status. User Profile (D5) contains user details such as name, email, and avatar, and is shared between Process 6.0 and Process 8.0.

### 1.3 PROBLEM ANALYSIS

This section presents the analysis of the problems that the proposed system intends to address. It provides a structured discussion of the identified issues, possible causes, and the corresponding proposed solutions. The section includes three main components: the Fishbone Diagram, the Problem and Solution Statement, and the Problem-Requirements Matrix.

### 1.3.1 Fishbone Diagram

The Fishbone Diagram, also known as the Ishikawa Diagram, shows the different possible causes of the existing problem. It helps organize and explain the factors connected to the problem and guides the researchers in determining which issues the proposed system should address.



*Figure 1.4. Fishbone Diagram*

Figure 1.4 illustrates the different factors that contribute to problems associated with inefficient or manual crayfish farming, which are grouped into five main categories: Materials, Equipment, Process, People, and Environment. The information presented in the diagram was gathered through interviews with local crayfish farmers to determine the common challenges encountered in crayfish farming operations.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The first factor is **Materials**, where there is a lack of proper monitoring documents and reference materials for checking water quality. Without proper guides and records, farmers may have difficulty understanding the water condition and knowing what action to take.

Aside from the available materials, **Equipment** also affects the farming process. The lack of automated sensors, real-time monitoring, and an automated feeding system means that most tasks still depend on manual work. This can take more time and may cause changes in water conditions to be noticed late.

Under the **Process** category, the interviews showed that water quality is mainly checked manually, which may delay the response to sudden changes. Feeding may also become irregular when there is no system to help maintain a proper feeding schedule. These can make the daily farming activities less consistent.

The farming process is also affected by **People**. Based on the farmers' responses, limited time for monitoring, human errors, and personal judgment can affect how water conditions are checked and how problems are handled. Farmers may not always have enough time to regularly monitor the crayfish while doing other tasks.

Lastly, For the **Environment**, the interviews identified unpredictable water-quality conditions and the buildup of uneaten feed and waste as concerns. Since crayfish are sensitive to changes in water conditions, poor water quality and waste buildup may cause stress and affect growth or survival if not managed properly.



### **1.3.2 Problem and Solution Statement**

#### **1.3.2.1 Problem Statement**

Interviews with selected local crayfish farmers revealed that manual and observational methods are commonly used in crayfish farming. These methods are mainly used to monitor the farming environment, particularly the water conditions. However, manual and observational methods may make it difficult to consistently and accurately measure critical water quality parameters, including pH, temperature, turbidity, and dissolved oxygen. Manual monitoring of these parameters could result in inconsistent assessment of water conditions. This is a concern because crayfish are sensitive to changes in the environment, and even minor fluctuations in water conditions can cause stress and may lead to mass mortality.

In addition, the findings revealed that a lack of standard operational procedures contributed to inconsistent and ineffective feeding and waste management. These limitations, together with human mistakes, may lead to operational and financial losses, thereby preventing farmers from achieving a sustainable and profitable production system.

#### **1.3.2.2 Solution Statement**

To address the identified problems in crayfish cultivation, the project team developed CrayCare system. It is an IoT and mobile-based system designed to provide efficient monitoring and management of the farming environment. It uses sensors to track the temperature, turbidity, pH, and



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

dissolved oxygen of the water. These mentioned water parameters need regular checks to reduce the risk of stress and mortality among crayfish. Project team added another feature that detects water level using ultrasonic sensor, helping maintain an appropriate amount of water in the culture tank.

CrayCare provides farmers with real-time access of data to water condition through a mobile application and LCD display, allowing farmers to see the current status of the culture environment. In order to further support decision-making, CrayCare is also integrated with machine learning model named Water Quality Assessment (WQA), which classifies the overall water quality and provide recommended corrective actions. This helps prevent the possibility of improper responses to changes in water conditions.

Furthermore, CrayCare incorporates an automatic feeder to effectively manage feeding schedule and help avoid delayed feedings. Automatic feeder was also calibrated to offer accurate amount of feeds to dispense, ensuring no overfeeding and underfeeding occurs.





### 1.3.3 Problem–Requirements Matrix

The Problem-Requirements Matrix establishes a clear and logical link between the identified crayfish aquaculture challenges and the specific technical features of the system. The mapping process establishes that all CrayCare functionalities are developed to address actual needs, providing justification for the proposed solutions.

Table 1.3

**Problem-Requirements Matrix of CrayCare**

| Problem                                                                                                                                   | Requirements                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lack of proper monitoring materials and reliance on manual observation make it difficult to regularly and accurately check water quality. | Real-time monitoring of temperature, pH, turbidity, and dissolved oxygen using water-quality sensors, with automatic recording of collected data.        |
| Changes in water condition may not be noticed immediately.                                                                                | Real-time alerts when monitored water parameters reach unsafe or abnormal levels.                                                                        |
| Manual management of water conditions can be time-consuming and inconsistent.                                                             | Remote control of the water pump and aerators through the mobile application based on the current water conditions.                                      |
| Farmers may rely on personal judgment when deciding whether the water condition is safe or what action should be taken.                   | Integration of a machine learning model called Water Quality Assessment (WQA) to classify the overall water condition and recommend appropriate actions. |
| Lack of a fixed feeding routine may result in inconsistent feeding or wasted feed.                                                        | Automated feeding through a servo-based feeder with scheduled feeding times, feeding records, and feed notifications.                                    |

Table 1.3 presents the Problem–Requirements Matrix of the CrayCare system. The problems shown in the matrix are based on the Fishbone Diagram, with related problems summarized and combined to avoid redundancy. The matrix



shows how these summarized problems are connected to the requirements of the system. Each requirement is matched with a related problem to ensure that the system addresses the identified needs. It also provides a clear guide for the development and evaluation of CrayCare.

## **1.4 PURPOSE AND DESCRIPTION**

### **1.4.1 Purpose of the Study**

The purpose of this study is to develop an IoT- and mobile-based system for small-scale Australian redclaw crayfish farming and determine its potential in supporting crayfish farm management. The study focuses on how the developed system can assist in monitoring water conditions and managing farm activities while providing a more convenient approach compared with traditional farming practices.

It also aims to examine the differences between the IoT-based and traditional farming methods based on the observed growth, survival, and mortality of the crayfish. In addition, the study seeks to determine the overall performance and suitability of the developed system based on selected ISO/IEC 25010 quality characteristics. The findings of the study will serve as a basis for determining whether CrayCare can be considered a useful tool for small-scale crayfish farming.



#### **1.4.2 Project Description**

CrayCare is an IoT- and mobile-based aquaculture system designed to support small-scale Australian Redclaw Crayfish (*Cherax quadricarinatus*) cultivation. It combines sensors, actuators, an ESP32 microcontroller, cloud storage, and a mobile application to provide a more convenient way of monitoring and managing the crayfish farming environment.

The system monitors important water conditions and provides the collected information in the mobile application. It also provides notifications when certain conditions require attention. For water management, the aerator and water pump can be controlled using ON, OFF, or AUTO modes, with the AUTO mode using set thresholds for automatic control. The system also includes an automated feeding feature to support regular feeding activities.

CrayCare is intended to help reduce the difficulty of manually monitoring and managing the farming environment. The system was also designed to support the comparison between IoT-based and traditional crayfish farming methods, particularly in terms of crayfish growth, survival, and mortality.



## 1.5 SPECIFIC OBJECTIVES

In order to meet the purpose of this study, the following specific objectives have been formulated:

1. To develop an IoT- and mobile-based system that monitors water conditions and supports crayfish farm management.
2. To compare the IoT-based system and traditional crayfish farming methods in terms of growth performance and survival and mortality rate.
3. To evaluate the system's performance using ISO/IEC 25010 standards, specifically functionality, reliability, usability, efficiency, compatibility, and security.

## 1.6 SCOPE AND LIMITATIONS

### 1.6.1 Scope

This study covers the development and evaluation of CrayCare, an IoT- and mobile-based aquaculture monitoring system for small-scale Australian redclaw crayfish (*Cherax quadricarinatus*) farming. The CrayCare system includes the monitoring of temperature, pH, turbidity, dissolved oxygen, and water level, together with the use of the mobile application for viewing water conditions, receiving notifications, controlling the actuators, and managing feeding activities.

The experimental setup of this study involved two grow-out tanks. Tank A was equipped with the CrayCare system, while Tank B was managed using the traditional manual method to serve as the basis for comparison. Both setups used



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

tarpaulin containers, also known as trapal ponds, measuring 8 feet long, 4 feet wide, and 1 foot deep. Each grow-out tank was stocked with 100 one-month-old juvenile crayfish at the beginning of the experimental period. This stocking quantity was selected considering the potential Return on Investment (ROI) of the proposed setup.

All in all, the study covers the grow-out stage of crayfish cultivation within a controlled environment. A comparative experimental design was used to compare the IoT-assisted grow-out tank and a manually managed grow-out tank under similar conditions. The comparison focused on the growth, survival, and mortality of the crayfish during the experimental period.

### **1.6.2 Limitations**

The following constraints and restrictions are acknowledged as part of the study:

#### **Experimental Coverage Limitations**

Breeding and nursery stages are part of the overall crayfish culture process; however, these stages were not included in the comparative system design and evaluation of the study. The reason behind it was to minimize additional life-stage variables that may influence the comparative results and introduce operational complexity beyond the objectives of the research. Therefore, the study was limited to the grow-out stage only, represented by Tank A and Tank B, to maintain consistency in the experimental setup.



### **Absence of Client Involvement**

The study does not involve existing external clients as users of the CrayCare system. Instead, the researchers served as the system users and were responsible for operating the system, managing the experimental setup, and cultivating the crayfish throughout the study period.

### **Absence of Automated Water Quality Stabilization**

CrayCare system is intended to function primarily as a monitoring and alert-based system. Automatic stabilization of water quality parameters, particularly pH levels, is excluded from the scope of the study. Automated pH adjustment in aquaculture systems involves the controlled dosing of chemical substances such as acids or bases, which requires precise calibration, carefully controlled hardware, and consistent human supervision to avoid overdosing.

Previous studies support this limitation. **Haddaway et al. (2013)** reported that even minor pH deviations can significantly affect the survival and growth of freshwater crayfish. **Chen and Chen (2003)** further noted that abnormal pH conditions negatively influence survival, molting, feeding, and growth in freshwater crustaceans. Similarly, **Zeng et al. (2024)** found that prolonged exposure to improper pH conditions increases mortality and disrupts digestive and immune functions in juvenile red swamp crayfish. **Lindholm-Lehto (2023)** also emphasized that water quality management in recirculating aquaculture systems



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

requires careful and continuous oversight because interactions among multiple parameters create a sensitive chemical balance.

Given these findings, automated chemical intervention was not included in the small-scale experimental deployment of CrayCare. The study therefore limits the system to monitoring, alert notifications, and feeding-related automation only.

### 1.7 DEFINITION OF TERMS

This section presents the key terms used in the study together with the corresponding definitions for clearer understanding. The definitions are based on how each term is applied within the context of the proposed system. This section also aims to provide clarity and minimize possible confusion, particularly regarding the technical words and concepts discussed in the study.

- **Aeration System.** A set of air pumps used to maintain dissolved oxygen levels in Tank A. The base pump operates continuously, while the boost pump is activated automatically when oxygen or temperature levels fall below the defined safe range.
- **Aquaculture.** The controlled cultivation of aquatic organisms in a water environment. In this study, it refers to the small-scale cultivation and management of Australian Redclaw Crayfish.
- **Automated Feeding.** A system feature that uses programmed controls to dispense a predetermined amount of feed to crayfish at scheduled intervals,



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

reducing the need for manual feeding and supporting consistent feeding management within the aquaculture setup.

- **Average Body Weight (ABW).** The average weight of sampled crayfish used as a basis for feeding recommendations in the system.
- **Biomass.** The estimated total weight of the crayfish population calculated by multiplying the Average Body Weight (ABW) by the total number of crayfish in the grow-out tank. In this study, biomass serves as a basis for feeding management during the experimental period.
- **Cloud Database.** An online remote data storage system used for storing and accessing system information through internet connectivity. In the proposed system, the cloud database is implemented using Firebase Realtime Database, where monitoring records, feeding data, growth records, and sensor readings are stored.
- **Comparative Experimental Design.** A research design used to compare the IoT-assisted monitoring setup and the manually managed setup under similar conditions to observe possible differences in performance and results.
- **CrayCare.** The proposed IoT- and mobile-based smart aquaculture monitoring system in this study. It integrates sensors, ESP32 microcontroller, Firebase, and an Android app to enable real-time management of crayfish farms.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- **Crayfish.** The Australian Redclaw Crayfish (*Cherax quadricarinatus*) used as the subject of cultivation in this study.
- **Dissolved Oxygen (DO).** The amount of oxygen present and dissolved in water. In this study, it refers to the oxygen level monitored within the crayfish grow-out setup to help maintain suitable water conditions for Australian Redclaw Crayfish.
- **Environmental Sensors.** Sensors used to measure water parameters such as temperature, pH, turbidity, dissolved oxygen, and water level within the proposed system. In this study, the term is used interchangeably with water parameter sensors.
- **ESP32 Microcontroller.** The central hardware device of CrayCare that aggregates sensor data, launches automated responses, and uploads data to Firebase via Wi-Fi.
- **Experimental Setup.** The arrangement of equipment, devices, and procedures used to conduct the study under controlled conditions for testing and evaluation purposes.
- **Firebase.** Cloud platform by Google for real time data storage and access management. It captures all system-generated data such as sensor logs, feeding history, and growth measurements.
- **Firebase Authentication.** A security feature used to manage user access to the CrayCare mobile application through account authentication.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- **Firestore Security Rules.** A security configuration that manages access permissions for stored data in Firestore.
- **Grow-Out Stage.** The stage of crayfish culture where juvenile crayfish are raised until reaching a larger size suitable for production.
- **HTTPS.** A secure communication protocol used for transmitting data between the mobile application, cloud database, and connected devices through an encrypted connection.
- **Internet of Things (IoT).** A technology that connects sensors, devices, and applications through the internet. In this study, IoT is used to enable real-time monitoring and automated control of the crayfish aquaculture system.
- **Mobile Application.** A software application designed to run on a mobile device that provides users access to the CrayCare system's monitoring, notifications, feeding management, and record-keeping features.
- **pH.** The term refers to the measurement of the acidity or alkalinity of the water. In this study, it is monitored as one of the parameters used to evaluate water conditions in the crayfish grow-out tank.
- **PlatformIO.** An embedded systems development platform used for writing, compiling, uploading, and monitoring ESP32 firmware through Visual Studio Code. It supports library management, project organization, debugging, and firmware deployment for the CrayCare system.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- **Real-Time Monitoring.** The continuous collection and display of current system conditions. In this study, it is used to monitor water parameters such as temperature, pH, turbidity, dissolved oxygen, and water level through the mobile application.
- **Recirculating Aquaculture System (RAS).** An aquaculture setup that continuously circulates and treats water through filtration and other components to maintain suitable conditions for aquatic organisms.
- **Relay Module.** An electronic switching device controlled by the ESP32 microcontroller. In this study, it is used to operate connected components such as pumps, aerators, and other automated devices.
- **Tank A.** The experimental grow-out tank equipped with the CrayCare IoT-based monitoring system, automated components, and water quality sensors for data collection and management.
- **Tank B.** The comparison grow-out tank managed through traditional manual monitoring practices without IoT-based sensors and automated control features.
- **Temperature.** A water quality parameter measured in degrees Celsius ( $^{\circ}\text{C}$ ). In this study, temperature is continuously monitored by the CrayCare system to help maintain suitable conditions for crayfish culture.
- **Turbidity.** A water quality parameter that measures the cloudiness of water caused by suspended particles. In the proposed system, turbidity readings



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

are monitored to help assess water condition and circulation requirements within the grow-out setup.

- **User.** refers to an individual who accesses and uses the CrayCare system through an assigned role, such as Farm Owner, Monitor User, or System Admin. In this study, the term refers to the researchers who perform these roles to test and evaluate the functions of the proposed system since no actual clients are involved during the study.
- **Visual Studio Code.** It refers to a source-code editor and development environment used for writing, editing, and managing program codes. In this study, it is utilized for developing the mobile application and programming the ESP32-based components of the proposed CrayCare system.
- **Water Level.** A monitoring parameter used to measure the amount or height of water present in the grow-out tank. In the proposed system, water level is continuously monitored to help maintain stable culture conditions and support proper operation of the monitoring setup.
- **Wi-Fi.** A wireless communication technology used for connecting the ESP32 microcontroller, mobile application, and cloud database to allow data transmission within the CrayCare system.
- **WPA2 (Wi-Fi Protected Access 2).** A wireless security protocol used in the study to protect the Wi-Fi network connection of the CrayCare system and ensure secure data transmission between connected devices.



## CHAPTER 2

### REVIEW OF RELATED LITERATURE, STUDIES, AND SYSTEMS

This chapter presents the related literature and studies relevant to the research topic. The discussions are organized using a thematic approach to group related concepts, ideas, and previous studies according to common themes and subject areas. Through this approach, the chapter provides a clearer understanding of existing knowledge, identifies research gaps, and establishes the foundation for the study.

#### IoT-Based Water Quality Monitoring Systems

Research on IoT-based water quality monitoring in aquaculture has grown significantly over the past decade. A systematic review of 30 internationally published papers from 2011 to 2020 found that inland aquaculture dominated the research landscape at 81%, while temperature, dissolved oxygen, and pH were the most commonly tracked parameters (**Prapti et al., 2022**). Real-time monitoring solutions made up 50% of the reviewed studies, though autonomous monitoring at 3% showed potential for future growth. Similarly, a bibliometric analysis of 217 articles from 2020 to 2024 confirmed that pH appeared in 98.2% of cases, followed by temperature at 92.9% and dissolved oxygen at 62.5% (**Flores-Iwasaki et al., 2025**). The use of IoT sensors was found to improve



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

growth rates, lower mortality, and enable faster detection of abnormal ammonia levels. Both reviews agree that automating parameter control, ensuring sensor maintenance, and improving connectivity in rural areas remain key challenges for the field.

A broader look at smart aquaculture systems shows that most IoT architectures follow a four-layer structure: a physical layer for sensors, a monitoring layer for data collection, a cloud layer for storage and analysis, and a communication layer for data transfer (**Vo et al., 2021**). Protocols such as MQTT, Wi-Fi, and GSM are commonly used to transmit data to mobile devices or computers, where farmers can act on the information or let automated systems respond. **Rastegari et al. (2023)** further noted that while IoT helps meet the growing global demand for seafood — aquaculture currently supplies nearly half of global consumption — issues such as biofouling, data reliability, and user adoption still need to be addressed for wider implementation.

Keeping water quality at acceptable levels in aquaculture settings is difficult when farms still depend on manual checks that are slow and easy to get wrong. **Arivarasi et al. (2026)** tackled this by building a low-cost IoT system around a Raspberry Pi to track temperature, pH, and total dissolved solids (TDS) in real time across fish hatcheries. The researchers pointed out that manual testing not only takes time but also leaves room for human error — both of which can put fish health at risk. **Suriasni et al. (2024)** took a similar direction, embedding an IoT system into a moving bed biofilm reactor (MBBR) within a recirculating aquaculture system (RAS) to keep tabs on pH, temperature, and TDS, while also automating aerator control to manage dissolved oxygen. After running the system for 35 days, total ammoniacal nitrogen (TAN) dropped by 50%, showing that



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

automated monitoring and control can bring real stability to water conditions and support better fish production outcomes.

Beyond individual farm applications, IoT monitoring systems have also been adopted in pond-based aquaculture settings with comparable success. **San Antonio et al. (2023)** designed a system that combines automated fish feeding with real-time water quality tracking in aquaculture ponds, developing the device across four integrated components: electronics, power supply, software programming, and mechanical feeder parts. Highly positive testing results confirmed the system's reliability and practical utility for boosting aquaculture productivity. Likewise, **Adriman et al. (2022)** developed an IoT-based monitoring and notification system for prawn ponds that uses pH, salinity, and ultrasonic sensors with an ESP32 microcontroller, storing data in Cloud Firebase and displaying it through an Android application. With error rates as low as 0.63%, the system proved effective in continuous monitoring and in alerting users when water parameters deviate from safe ranges.

At a broader level, **Pratiwy and Aisyah (2026)** conducted a literature review affirming that the combination of IoT and artificial intelligence (AI) is forming a new paradigm in aquaculture management that is precise, adaptive, and efficient. Their review covered sensor infrastructures, cloud-based visualization platforms, and AI algorithms such as Random Forest, Support Vector Machine (SVM), and Long Short-Term Memory (LSTM) networks applied for time-series prediction, anomaly detection, and automated control. However, the authors cautioned that challenges including sensor durability, network dependency, and data standardization continue to hinder broad implementation — a finding that underscores the need for continued research and system refinement.



### IoT Systems with Mobile Application Integration

The combination of IoT sensors with mobile applications has made water quality monitoring more accessible and practical for everyday farmers. **Darmalim et al. (2021)** developed Nusapond, an Android-based system powered by a solar-powered NodeMCU module that tracks temperature, pH, dissolved oxygen, TDS, and salinity. The application allows farmers to monitor multiple ponds from a single device, manage production cycles, track feeding and mortality records, and receive alerts when readings fall outside safe ranges. In a similar manner, **Zuhaer et al. (2025)** implemented an Industry 4.0-based system that uses advanced dissolved oxygen and ammonia sensors paired with an Android application, achieving over 98% accuracy for dissolved oxygen and nearly 99% for ammonia — enabling farmers to manage pond conditions remotely with confidence.

Other mobile-integrated systems have been designed with specific local contexts in mind. **Natividad et al. (2023)** created A-FishTemp, an Android application for fishponds in the Philippines that displays real-time pH and temperature readings, logs historical data, issues warnings for unstable conditions, and recommends suitable fish and crustacean species based on current water conditions. The study aimed to reduce trial-and-error in fish breeding and promote more sustainable farming practices. **Olanubi et al. (2024)** took a similar approach by developing a web-based monitoring system that transmits sensor data to the cloud and sends real-time alerts via WhatsApp Messenger, making it





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

accessible from any internet-connected device worldwide and demonstrating that simple notification tools can significantly improve farm response times.

### **Fuzzy Logic-Integrated Aquaculture Monitoring**

Fuzzy logic has been widely paired with IoT technology to improve automated decision-making in aquaculture. An intelligent crayfish water quality control system combining IoT with the Sugeno fuzzy inference method was developed to monitor pH, turbidity, temperature, and water level in real time (**Suwardi Ansyah et al., 2024**). The fuzzy logic rules evaluate pH and turbidity simultaneously to determine whether drainage should be set at 0%, 30%, or 50%, and the system also activates cooling when temperatures rise too high. Tested against manual calculations, the system achieved 100% accuracy, reducing human error and manual labor while promoting more productive and sustainable crayfish farming. Similarly, **Nagothu et al. (2025)** introduced a fuzzy logic-based IoT system that monitors temperature, pH, dissolved oxygen, salinity, and total suspended solids, automatically adjusting aerators and water pumps based on defined rules. The system connects to a smartphone application and cloud platform via MQTT, making precise water management available remotely.

The integration of fuzzy logic with IoT has further enhanced the precision and responsiveness of water quality management systems in aquaculture. An IoT-enabled fuzzy logic system using an ESP32 wireless sensor network was introduced to collect real-time data on temperature, pH, dissolved oxygen, salinity, and TDS in fish farming environments (**Boumehrez et al., 2025**). A fuzzy logic controller analyzes the collected



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

data and recommends the most appropriate corrective actions, which farmers can then implement through a web-based application that provides real-time information and intelligent alerts. This approach enables the automatic management of aeration, water exchange, and temperature, resulting in improved fish health and more efficient farm operations. **Shitsukane et al. (2024)** also integrated fuzzy logic with solar power and IoT technology for real-time water quality monitoring in cage fish culture, employing a wireless sensor network with embedded microcontrollers and ZigBee wireless modules to autonomously adjust water quality conditions and optimize fish growth.

Complementing these system-level implementations, **Bautista et al. (2022)** developed a fuzzy logic-based adaptive aquaculture water monitoring system using a Mamdani-type fuzzy inference system (FIS) model tailored for Nile tilapia farming. The system transforms input limnological parameters — pH, temperature, TDS, and dissolved oxygen — into four output states: excellent, good, poor, and toxic, enabling timely predictions of water quality. These outputs were designed for integration with a feedback system that triggers appropriate treatments such as filtering, aeration, and water flushing to maintain a safe aquatic environment. Taken together, these studies demonstrate that the inclusion of fuzzy logic in IoT monitoring frameworks substantially improves the system's ability to interpret complex, real-world water quality data and respond in a nuanced, context-sensitive manner.

### **Machine Learning and AI Integration in Aquaculture Monitoring**

The incorporation of machine learning and artificial intelligence into IoT frameworks has enabled more advanced prediction and early warning capabilities in



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

aquaculture. A multi-scale feature fusion model integrating a Convolutional Autoencoder with Gated Recurrent Unit (MS-CAGRU) networks was proposed for water quality prediction in aquaponic fish ponds (**Sundararajan et al., 2025**). The model captures patterns over time and reduces overfitting, outperforming traditional prediction methods when tested on a public aquaponic dataset. **Paul et al. (2026)** further expanded this direction by combining IoT sensors with computer vision and machine learning for real-time shrimp monitoring. A YOLOv5 model detected and tracked shrimp underwater at 84% accuracy, while Decision Tree and Naïve Bayes classifiers predicted behavioral responses to water stress at 92% and 88% accuracy, respectively — providing a practical, data-driven tool for reducing mortality and improving farm management.

Disease detection has also benefited from the application of deep learning in aquaculture. **Atilkan et al. (2024)** evaluated several deep learning and machine learning models for identifying diseases in crayfish using morphological data and photographs. A hybrid model combining ResNet50 for feature extraction with Random Forest classification performed best, improving detection accuracy by 3.2% over ResNet50 alone and outperforming other models with statistical significance. The study highlighted that combining deep learning with traditional machine learning techniques can lead to earlier and more accurate disease detection, which is essential for preserving crayfish populations and maintaining ecological balance.

### **Cost-Effective and Locally Adapted IoT Solutions**

Many researchers have focused on building affordable and locally relevant IoT systems that can work within the constraints of developing regions. A low-cost monitoring



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

system targeting off-grid communities in the Philippines measures pH, turbidity, and temperature using both Wi-Fi and GSM modules, with data accessible through a web dashboard, Android app, and SMS alerts (**Abrajano et al., 2024**). This dual-connectivity approach ensures that farmers in areas with limited internet access can still receive timely water quality updates. Likewise, **Gallemit (2023)** validated a similar IoT prototype in Dapitan City aquaculture farms to support cost-efficient, data-driven farming practices, showing that even basic systems can meaningfully improve local aquaculture outcomes.

Open-source platforms and affordable hardware have also proven capable of matching the performance of expensive commercial systems. **Malandrakis (2025)** developed a Raspberry Pi-based monitoring system for Recirculating Aquaculture Systems (RAS) that tracks temperature, pH, conductivity, water level, and pump functionality, transmitting data to the ThingsBoard IoT platform and sending mobile alerts via Pushover when readings go out of range. With total hardware costs below €150, the open-source system offers a scalable and reliable alternative to commercial solutions that can cost thousands of euros.

### **Wireless Sensor Networks and Remote Monitoring in Aquaculture**

The development of wireless sensor networks has been instrumental in enabling remote, real-time monitoring of water quality across aquaculture systems. **Hongpin et al. (2015)** designed a multi-parameter monitoring system using dissolved oxygen, pH, temperature, and ammonia nitrogen sensors integrated with ZigBee and GPRS modules for long-range data transmission. The system was also connected directly to an aerator for automatic control of dissolved oxygen concentration, and it achieved a data



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

transmission packet loss rate of only 0.43% — a result that underscores the high reliability of wireless monitoring technology even in earlier implementations. This foundational work laid the groundwork for subsequent IoT-based systems that have built upon wireless connectivity to deliver increasingly sophisticated monitoring and control capabilities.

### **IoT Monitoring for Specific Aquaculture Species**

The application of IoT has further extended to species-specific and context-specific aquaculture environments, demonstrating the technology's versatility. An ESP32-based IoT system designed specifically for Mahseer fish farming was presented by **Abdikadir et al. (2024)**, utilizing the Blynk 2.0 platform for real-time data visualization and a C/C++ algorithm for effective sensor-microcontroller communication. The system proved effective in maintaining optimal water conditions and enabling early detection of potential issues. In a similar vein, an automated IoT system for monitoring and controlling water quality in a *Molobicus tilapia* RAS in the Philippines was developed to enable readings to be viewed on any internet-connected device and to allow automatic responses through a control panel (**Libao et al., 2024**). Further demonstrating these capabilities in smart aquarium management, an ESP32-based system connected to the Blynk IoT platform achieved 96% sensor accuracy and a 1.2-second response time, along with 97% reliability in automated feeding and water circulation — confirming that low-cost IoT technology is accessible and effective for both home users and commercial aquaculture businesses (**Ayon et al., 2026**).

Establishing precise water quality standards is fundamental to the success of any aquaculture operation, as different species have distinct physiological thresholds that must be maintained for optimal survival and productivity. Comprehensive guidelines on



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

redclaw crayfish aquaculture indicate that while the species is relatively tolerant of water quality variations compared to other crayfish, specific parameters must still be carefully managed (**Business Queensland, 2025**). Dissolved oxygen must be kept above 5 ppm, as levels below 1 ppm cause significant stress, prompting crayfish to migrate toward pond edges. A pH range of 7.0–8.5 is recommended, since lower values or insufficient water hardness below 50 ppm can impair moulting and shell hardening — critical biological processes for crayfish health. Furthermore, the guidelines specify that optimal growth temperatures fall between 26°C and 29°C, while temperatures below 10°C or above 34°C markedly increase mortality risk, highlighting the importance of continuous thermal monitoring in crayfish farming environments.

Several studies have developed monitoring systems tailored to specific aquaculture species, recognizing that different organisms have unique water quality requirements. For freshwater lobster farming in Indonesia, an IoT monitoring dashboard using the Antares platform was developed to display real-time temperature and dissolved oxygen readings with automatic set points and comparison graphs (**Marselina et al., 2023**). Building on this, **Muthmainnah et al. (2025)** developed a more advanced Antares-based system with pH, temperature, and turbidity sensors that achieved over 98% accuracy after calibration. A five-day field test recorded stable readings within acceptable ranges, and the integration of real-time data with predictive analytics allowed for more proactive management compared to conventional manual monitoring.

For milkfish cultivation, **Gulifardo et al. (2025)** developed iPond, a system that monitors pH and salinity in real time and connects to the Blynk IoT platform for remote access and automatic alerts. Evaluation results showed an average functionality score of



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

4.325 and error rates as low as 0.027% for pH and 0.187% for salinity, confirming the system's reliability in maintaining optimal water conditions. **Shete (2024)** further demonstrated that Arduino-based IoT systems can achieve seamless real-time monitoring in general aquafarming environments, while **Reddy (2023)** proposed a similar system tracking pH, temperature, and turbidity remotely to help farmers maintain ideal conditions and improve economic outcomes. These species-focused systems show that targeted, context-aware monitoring tools are more effective in practice than one-size-fits-all approaches.

### **Recirculating Aquaculture Systems (RAS) and the Potential for Smart Technologies**

Recirculating aquaculture systems (RAS) represent one of the most controlled and intensive forms of modern fish farming, and their complexity makes them particularly well-suited for the integration of smart monitoring technologies. **Grandez-Yoplac et al. (2025)** conducted a systematic review and bibliometric analysis of 387 research articles on rainbow trout farming in RAS, spanning publications from 1941 to 2025. The findings revealed a consistent increase in scientific output since 2011, with leading contributions from the United States, Denmark, Finland, and Germany. Research in this domain has concentrated on three main areas: fish physiology and metabolism, nutrition and growth performance, and water treatment technologies. While the field has progressively shifted from foundational biological studies toward more sustainable and technologically advanced approaches — including molecular tools such as proteomics and transcriptomics — the integration of smart technologies like IoT remains notably limited. The authors consequently emphasized the urgent need to adopt self-calibrating sensors,



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

machine learning models, and real-time autonomous control systems to advance the sustainability and intelligence of rainbow trout RAS operations. This gap between current practice and technological potential points to a significant opportunity for applied research in the design and deployment of integrated smart monitoring systems for RAS environments.

### **Growth and Molting Mechanisms of Crayfish**

**Hamasaki et al. (2023)** discussed the reproduction in *Procambarus clarkii* and pointed out the indeterminate growth by ecdysis. Intermolt (which is characterized by intensive feeding and growing tissues), premolt, ecdysis (molting process), and postmolt (hardening of the shell) were identified as stages of molting. During development, young crayfish experience numerous moltings — about 11 moltings to adulthood, gaining growth of 15% in length and up to 40% in weight during each molting period. Growth slows in adults, who molt less frequently (typically 2–3 times per year).

Further insight into the molting process was provided by **Zhu et al. (2026)**, who analyzed the alterations in gene expression of the exoskeleton in red swamp crayfish (*Procambarus clarkii*) during the molting process using RNA sequencing analysis. Five molting stages were considered in the experiment, and 7,671 genes with differential expression levels were found. The genes responsible for cuticle protein biosynthesis, breakdown of the existing exoskeleton (for example, chitinases), and sclerotization were found to be affected the most with alterations up to 10 times higher at the peak level. Moreover, the biological functions associated with cuticle biology, chitin metabolism, and hormones production were determined. This information is helpful for the better





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

comprehension of exoskeleton remodeling and the development of recommendations for the successful cultivation of crayfish.

Similarly, the research conducted by **Zou et al. (2026)** sought to investigate the molting phenomenon in crayfish (*Procambarus clarkii*). Molting is an important yet hazardous period when the organism sheds off the old exoskeleton to facilitate its growth. Due to high mortality rates recorded among farm-bred crayfish in molting periods, the researchers administered various amounts of 20-hydroxyecdysone to mimic early and mid-premolt stages. By conducting transcriptomic and microbiome analysis, the authors determined that in early premolting stages, the organism prioritizes the degradation and assimilation of old exoskeleton. When proceeding to mid-premolt stages, the body starts building a new exoskeleton, accelerating metabolic processes and antioxidant activity. It should also be mentioned that administration of hormone increases the presence of pathogenic bacteria like *Escherichia-Shigella* and *Vibrio*. The findings of the study, referring to the study of Zou and co-researchers, provides clearer molecular understanding of molting and offers practical guidance for improving survival and growth rates in crayfish farming.

### **Morphological Development and Reproductive Characteristics**

**Li et al. (2024)** studied the effects of different cultivation environments on the body shape and size of crayfish (*Procambarus clarkii*). Juvenile crayfish collected from the same population were cultured for 60 days under three conditions: ponds, paddy fields, and aquaculture barrels. Ten morphological traits were measured in both male and female crayfish, including full length, body length, claw length and weight, and cephalothorax



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

dimensions. The study found that the physical development of the crayfish was strongly influenced by both the cultivation environment and sex. Correlation analysis, pathway analysis, and gray relation analysis were used to identify which body traits were most closely associated with body weight in each group. Among the measured traits, claw weight showed the greatest variation across cultivation environments, while the traits most strongly linked to body weight differed according to both cultivation system and sex. These findings provide useful insight for improving artificial selection and crayfish farming practices.

Additionally, **Pascale et al. (2024)** examined the body shapes of Louisiana crayfish (*Procambarus clarkii*) from a Mediterranean population in Sardinia. Using body measurements and molecular analysis, the study confirmed the presence of two distinct forms: Form I (reproductive), which has larger claws and well-developed sexual features, and Form II (non-reproductive), which has smaller claws. The study also checked for the presence of the dangerous crayfish plague fungus but found none in the population. Additionally, the researchers determined that male crayfish reach maturity when carapace length is between 35.0 and 37.1 mm. These findings help improve understanding of the reproductive cycle of this invasive species and provide a useful tool for accurate identification in natural environments.

Moreover, **Hossain et al. (2019)** examined the body measurements, growth patterns, and reproductive ability of marbled crayfish (*Procambarus virginalis*). The study found that the species shows positive allometric growth, with abdomen width and claw size increasing faster than postorbital carapace length. Other body parts such as total length and carapace width grow more slowly in comparison. The condition factor indicated



that the crayfish were generally healthy. Females produced their first clutch at an average total length of 42.8 mm, with a mean fecundity of around 90 eggs. Fecundity increased significantly with larger body size and weight. The research also showed that counting eggs on the third pair of pleopods offers a reliable way to estimate total fecundity. These findings provide useful insights into the growth and reproductive potential of this parthenogenetic crayfish.

### **Crayfish Farming Practices and Production Performance**

**Wang et al. (2023)** developed an ecological index evaluation model designed to assess the growth and development of *Procambarus clarkii* based on its biological characteristics. The study emerged from the growing demand for more accurate monitoring methods as crayfish farming continues to expand on a larger scale. Using fuzzy correlation constraints and process optimization, the model was able to evaluate several key growth indicators throughout different developmental stages. The results showed that the ecological index followed the expected growth trend, reaching its highest value of 0.85 during the third measurement period before declining to 0.37 in the fourth period as growth became more stable. The findings suggest that the system can produce accurate and consistent evaluations while adapting effectively to changing conditions, making it a promising basis for developing more practical monitoring tools in crayfish aquaculture.

In response to the growing demand for efficient aquaculture systems, **Rigg et al. (2020)** reviewed the production of juvenile redclaw crayfish (*Cherax quadricarinatus*) in aquaculture. Although farming of this species has continued for more than thirty years,



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

overall production levels remain relatively low because suitable farming technology is still developing. A major challenge is the reliable supply of juveniles for stocking into ponds to grow to market size. Recent improvements in breeding and large-scale production of craylings have brought new momentum to the industry. The review summarizes current practices and highlights promising innovations that could help expand redclaw crayfish farming in the future.

Within the context of commercial crayfish farming, **Naranjo-Páramo et al. (2018)** investigated the production of female redclaw crayfish (*Cherax quadricarinatus*) in gravel-lined ponds to determine if this is a viable commercial production system. The study used a stochastic modeling approach to understand how crayfish grow into different market sizes. The crayfish were stocked at a density of 6 females per square meter and grown for 17 weeks. Results showed that the average weight reached was about 80.5 grams, with a survival rate of 76.2% and efficient feed use. The researchers found that most crayfish reached the 61-90 gram size class, while very few grew larger than 90 grams. The simulation model also indicated that predictions of production can be quite accurate if larger numbers of ponds are considered. Overall, the study confirmed that raising only female crayfish in gravel-lined ponds works well, recommending a stocking density of 6 organisms per square meter for commercial farming.

### SYNTHESIS OF THE STUDY

The reviewed literature and studies show that the use of Internet of Things (IoT) technologies in aquaculture continues to increase as more aquaculture systems adopt digital monitoring and automated management practices. Many of the reviewed studies



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

emphasized the importance of real-time monitoring in maintaining stable water conditions and reducing problems caused by sudden environmental changes. Commonly monitored water parameters include temperature, pH, turbidity, dissolved oxygen, salinity, and water level. These parameters are usually measured using sensors connected to microcontrollers, cloud-based systems, and mobile applications (Abdikadir et al., 2024; Prapti et al., 2022; Vo et al., 2021; Darmalim et al., 2021). Through these systems, monitoring data can be viewed remotely, allowing faster checking of water conditions and quicker response when parameters move outside the acceptable range.

The literature also shows a gradual transition from manual monitoring practices to more automated aquaculture management systems. Several studies explored the use of fuzzy logic, automated feeding systems, predictive models, and machine learning techniques to improve monitoring efficiency and lessen the limitations of manual observation (Suwardi Ansyah et al., 2024; Boumehrez et al., 2025; Shitsukane et al., 2024; Paul et al., 2026). Results from these studies commonly showed that automated systems can help reduce human error, maintain more stable monitoring conditions, and improve operational efficiency through continuous monitoring and automatic responses. Some studies also emphasized the need for affordable and practical systems that can still function effectively in small-scale aquaculture environments where resources and internet connectivity may be limited (Abrajano et al., 2024; Arivarasi et al., 2026).

Several studies involving Recirculating Aquaculture Systems (RAS) and controlled aquaculture environments also emphasized the importance of continuous monitoring. Manual monitoring methods were often described as repetitive, time-consuming, and less effective in responding immediately to sudden changes in water conditions (Malandrakis,



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

2025; Libao et al., 2024; Suriasni et al., 2024). Due to the limitations associated with manual monitoring, many researchers introduced IoT-based systems that support real-time monitoring, automated alerts, and remote access to water quality data. These systems were mainly developed to improve response time whenever changes in water conditions occur. Based on the reviewed studies, aquaculture management is gradually becoming more dependent on digital monitoring systems that can provide continuous environmental monitoring and faster corrective actions when problems arise.

The reviewed literature related to crayfish and freshwater lobster farming also showed that environmental conditions directly affect crayfish survival, growth, and overall productivity. Dissolved oxygen, temperature, and pH were consistently identified as important water parameters that need proper monitoring and management in crayfish culture systems (Business Queensland, 2025). Some studies also discussed the use of IoT-based monitoring systems, mobile applications, and automated management tools to make routine monitoring activities more manageable and efficient in aquaculture operations (Marselina et al., 2023; Muthmainnah et al., 2025; Suwardi Ansyah et al., 2024).

Crayfish growth, molting, and body development were also discussed in several reviewed studies. Hamasaki et al. (2023) explained that crayfish grow through a series of molting cycles, especially during the juvenile stage where body length and weight increase more rapidly after each molt. Zhu et al. (2026) and Zou et al. (2026) further examined the biological processes involved in shell formation and hardening during molting. The studies pointed out that crayfish become more vulnerable to environmental stress during this stage, making stable water conditions necessary for healthy growth and



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

survival. Other studies examined the effects of cultivation environments on crayfish body measurements, morphology, and growth performance (Li et al., 2024; Hossain et al., 2019). In addition, Rigg et al. (2020) and Naranjo-Páramo et al. (2018) discussed some of the ongoing challenges in juvenile production and grow-out management in crayfish farming. Overall, the reviewed studies suggest that consistent monitoring and proper environmental management are closely related to healthy crayfish development during the grow-out period.

Although previous studies demonstrated significant developments in IoT-based aquaculture systems and crayfish farming practices, limited studies were found that combine IoT-based water quality monitoring, automated management functions, and crayfish grow-out observation within a controlled small-scale aquaculture setup. Most of the reviewed studies focused separately on sensor technologies, automated monitoring systems, predictive models, or crayfish growth and production management. Because of this, there remains a need for a practical system that can integrate real-time monitoring, automated feeding, mobile accessibility, and crayfish grow-out management within a single small-scale experimental environment. Challenges related to long-term implementation also remain present in the literature, particularly in terms of sensor maintenance, infrastructure requirements, operational costs, and dependence on stable internet connectivity in small-scale aquaculture environments (Rastegari et al., 2023; Pratiwy & Aisyah, 2026).

The reviewed literature and studies provide a strong foundation for the proposed study. Existing research demonstrates the growing importance of IoT technologies in aquaculture while also showing the need for practical systems suitable for small-scale



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

crayfish farming. In response to the identified gaps, the proposed study aims to develop an IoT- and mobile-based monitoring system intended for small-scale crayfish grow-out operations. The proposed study also seeks to examine the implementation of IoT-assisted monitoring and traditional manual management under controlled experimental conditions using Tank A and Tank B. Through this approach, the study is expected to provide additional information regarding the possible application of IoT technologies in supporting environmental monitoring and crayfish grow-out management in small-scale aquaculture systems.





## **CHAPTER 3**

### **METHODOLOGY**

This chapter presents the methods, procedures, and techniques that will be used in the development and evaluation of the proposed system. The study will employ a comparative experimental research design to compare the performance of IoT-based monitoring and traditional manual management in crayfish grow-out operations under controlled conditions. This chapter discusses the system requirements, design specifications, development methodology, testing procedures, implementation process, and evaluation plan that will guide the study. It also includes the quality standards, ethical considerations, data analysis procedures, and statistical treatments that will be used to assess the functionality and effectiveness of the proposed IoT- and mobile-based crayfish monitoring system.

#### **3.1 REQUIREMENTS ANALYSIS**

The Requirements Analysis phase systematically translates the operational challenges and user needs identified in the problem analysis into a technical framework that the CrayCare system can implement. This section establishes the required functions through a Requirements–Features Matrix, a Use Case Diagram, and detailed Use Case Reports, linking actual farming challenges with the system’s technical capabilities. This structured approach ensures that all system functions are aligned with the project



objectives, creating a clear path to meet the productivity requirements and management needs of local crayfish farmers.

### 3.1.1 Requirements – Features Matrix

The Requirements–Features Matrix systematically maps both functional and non-functional requirements to the corresponding system features to ensure that all identified needs are addressed within the CrayCare system. The matrix also serves as a development tracking reference for verifying whether the implemented functionalities and quality attributes align with the objectives of the study.

Table 3.1

**Functional Requirements Matrix**

| <b>Hardware Requirements</b>                                     |                                                                                         |                 |               |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------|---------------|
| <b>Requirement Description</b>                                   | <b>System Feature(s)</b>                                                                | <b>Priority</b> | <b>Status</b> |
| The system must monitor water temperature in real time           | Real-time temperature sensing via DS18B20 integrated with ESP32 data acquisition module | High            | Implemented   |
| The system must monitor pH level of water in real time           | pH level monitoring using calibrated pH sensor probe interfaced with ESP32              | High            | Implemented   |
| The system must monitor turbidity of water in real time          | Turbidity level detection using turbidity sensor module                                 | High            | Implemented   |
| The system must monitor dissolved oxygen (DO) level in real time | Dissolved oxygen monitoring using DO sensor module                                      | High            | Implemented   |
| The system must monitor water level of the tank in real time     | Water level detection using ultrasonic module                                           | High            | Implemented   |
| The system must support real-time data transmission              | Wireless data transmission via ESP32 with WiFi                                          | High            | Implemented   |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                                                 |                                                                                                                      |      |             |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|------|-------------|
|                                                                 | connectivity                                                                                                         |      |             |
| The system must circulate and filter water in the grow-out tank | Water circulation and filtration using pump-based RAS (Recirculating Aquaculture System) controlled via relay module | High | Implemented |

*Continuation of Table 3.1*

| <b>Hardware Requirements</b>                                                                                                      |                                                                                                                                                    |                 |               |
|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| <b>Requirement Description</b>                                                                                                    | <b>System Feature(s)</b>                                                                                                                           | <b>Priority</b> | <b>Status</b> |
| The system must provide oxygen supply to crayfish tanks                                                                           | Aeration system using air pump (aerator) controlled via relay module                                                                               | High            | Implemented   |
| The system must automate feeding in grow-out tank                                                                                 | Automated feeding mechanism using servo motor-based feeder system                                                                                  | High            | Implemented   |
| The system must automatically trigger actuators based on sensor threshold rules to maintain water quality.                        | Automated relay triggers (ON/OFF) for the aerator and water pump when dissolved oxygen, temperature, or water levels exceed warning limits.        | High            | Implemented   |
| <b>Software Requirements</b>                                                                                                      |                                                                                                                                                    |                 |               |
| <b>Requirement Description</b>                                                                                                    | <b>System Feature(s)</b>                                                                                                                           | <b>Priority</b> | <b>Status</b> |
| The system must allow user authentication                                                                                         | Secure user authentication system for system access                                                                                                | High            | Implemented   |
| The system must support role-based user access                                                                                    | Role-based access control for Farm Owner, Monitor User, and System Admin with defined permissions per role                                         | High            | Implemented   |
| The system must restrict write actions (e.g., threshold updates, manual feeder triggers) strictly to the designated system Owner. | Role-Based Access Control (RBAC) linking Admin and Monitor users to an ownerId and enforcing permission restrictions inside the application logic. | High            | Implemented   |
| The system must manage user accounts                                                                                              | User management interface for viewing user list, filtering by role, changing roles, and disabling accounts                                         | High            | Implemented   |
| The system must display                                                                                                           | Mobile application dashboard                                                                                                                       | High            | Implemented   |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                            |                                                             |      |             |
|--------------------------------------------|-------------------------------------------------------------|------|-------------|
| real-time sensor data                      | with live IoT sensor visualization and status indicators    |      |             |
| The system must record initial stock setup | Setup modal for initial stock count and stocking date input | High | Implemented |

*Continuation of Table 3.1*

| <b>Software Requirements</b>                                                                                        |                                                                                                                                                                                                             |                 |               |
|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| <b>Requirement Description</b>                                                                                      | <b>System Feature(s)</b>                                                                                                                                                                                    | <b>Priority</b> | <b>Status</b> |
| The system must record sampling data                                                                                | Sampling input form for sample count, total weight, and total length with auto-computed Average Body Weight, Average Body Length                                                                            | High            | Implemented   |
| The system must classify growth stages                                                                              | Automatic classification into Early Juvenile, Advanced Juvenile, Pre-Adult, and Market Size based on ABW and ABL thresholds, with automatic adjustment of sensor threshold ranges per detected growth stage | Medium          | Implemented   |
| The system must display growth trends                                                                               | Line charts for ABW/ABL and estimated biomass trend, and mortality bar chart over sampling periods                                                                                                          | Medium          | Implemented   |
| The system must analyze historical and real-time parameters using machine learning to predict water quality health. | Integration with Random Forest machine learning models hosted on Hugging Face API to provide water quality status classification (Optimal, Warning, Critical) and growth insights.                          | High            | Implemented   |
| The system must display growth overview                                                                             | Growth overview panel in the Sampling tab showing initial baseline (date, ABW, ABL), latest sampling, and growth change                                                                                     | Medium          | Implemented   |
| The system must configure feed schedule                                                                             | Time-based feeding schedule interface with AM/PM                                                                                                                                                            | Medium          | Implemented   |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|  |                                                                                                                                       |  |  |
|--|---------------------------------------------------------------------------------------------------------------------------------------|--|--|
|  | grouping, native time picker, and per-schedule status tracking (Completed, Pending, Skipped, Upcoming) with countdown to next feeding |  |  |
|--|---------------------------------------------------------------------------------------------------------------------------------------|--|--|

*Continuation of Table 3.1*

| <b>Software Requirements</b>                                                                                                      |                                                                                                                                                                                                                                                        |                 |               |
|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------|
| <b>Requirement Description</b>                                                                                                    | <b>System Feature(s)</b>                                                                                                                                                                                                                               | <b>Priority</b> | <b>Status</b> |
| The system must track feeding progress                                                                                            | Dashboard feeding schedule card showing completed versus total feedings today, last fed time, next feeding countdown, and progress bar                                                                                                                 | Low             | Implemented   |
| The system must notify users of abnormal water conditions, operational status changes, feeding reminders, and sampling reminders. | FCM-based push notification system that detects sensor threshold violations and zone transitions from Firebase RTDB, sends real-time push alerts with customizable sound/vibration preferences, and includes automated feeding and sampling reminders. | Medium          | Implemented   |
| The system must store historical sensor data                                                                                      | Cloud-based database for continuous data logging and storage                                                                                                                                                                                           | Medium          | Implemented   |
| The system must cache sensor data locally during internet interruptions to ensure data reliability.                               | Local persistence via SharedPreferences and Firebase offline capabilities enabling users to view the last cached readings when offline.                                                                                                                | Medium          | Pending       |
| The system must generate machine learning-based insights, predictions, and recommendations                                        | Random Forest-based ML model for generating sensor-driven insights, predictions, and recommendations                                                                                                                                                   | High            | Implemented   |
| The system must allow remote monitoring.                                                                                          | IoT-enabled mobile application for real-time farm monitoring                                                                                                                                                                                           | High            | Implemented   |
| The system must allow                                                                                                             | Mobile control interface for                                                                                                                                                                                                                           | Medium          | Implemented   |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                            |                                                 |  |  |
|----------------------------|-------------------------------------------------|--|--|
| manual control of hardware | manual override of devices<br>(ON/OFF controls) |  |  |
|----------------------------|-------------------------------------------------|--|--|

Table 3.1 presents the functional requirements and corresponding features of the proposed crayfish monitoring and management system. It identifies the specific functions that the system should perform, along with the features developed to support these functions, the priority level, and implementation status. These requirements serve as a guide in developing the system and ensuring that the necessary capabilities are properly included.

The functional requirements focus on the actions and services that the system must provide, such as monitoring water conditions, managing devices, recording growth data, handling user access, and supporting feeding and monitoring activities. By presenting these requirements in a matrix, the development progress can be tracked and the system features can be checked based on the goals of the project.

Table 3.2

### Non-Functional Requirements Matrix

| Hardware Requirements                                               |                                                    |          |             |
|---------------------------------------------------------------------|----------------------------------------------------|----------|-------------|
| Requirement Description                                             | System Feature(s)                                  | Priority | Status      |
| The system should operate continuously in real-time farm conditions | 24/7 availability with stable real-time monitoring | High     | Implemented |
| The system should ensure reliable sensor readings                   | High accuracy and consistency of sensor data       | High     | Implemented |
| The system should withstand aquaculture environment conditions      | Durable and environment-resistant hardware design  | Medium   | Implemented |
| The system should support                                           | Reliable wireless                                  | High     | Implemented |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                                                       |                                                       |                 |               |
|-------------------------------------------------------|-------------------------------------------------------|-----------------|---------------|
| continuous data transmission                          | communication with minimal downtime                   |                 |               |
| <b>Software Requirements</b>                          |                                                       |                 |               |
| <b>Requirement Description</b>                        | <b>System Feature(s)</b>                              | <b>Priority</b> | <b>Status</b> |
| The system should provide fast real-time response     | Low latency in data processing and dashboard updates  | High            | Implemented   |
| The system should ensure system usability for farmers | Simple, intuitive, and user-friendly mobile interface | High            | Implemented   |

*Continuation of Table 3.2*

|                                                        |                                                   |                 |               |
|--------------------------------------------------------|---------------------------------------------------|-----------------|---------------|
| <b>Software Requirements</b>                           |                                                   |                 |               |
| <b>Requirement Description</b>                         | <b>System Feature(s)</b>                          | <b>Priority</b> | <b>Status</b> |
| The system should securely store farm data             | Data encryption and secure storage mechanisms     | High            | Implemented   |
| The system should support remote access and monitoring | Secure remote IoT-based monitoring                | High            | Implemented   |
| The system should maintain data accuracy and integrity | Data validation, filtering, and error correction  | High            | Implemented   |
| The system should be maintainable and updatable        | Modular and well-documented software architecture | Medium          | Implemented   |

Table 3.2 presents the non-functional requirements and corresponding system features of the proposed crayfish monitoring and management system. These requirements define the expected quality, performance, and operational conditions of the system, including reliability, usability, security, scalability and real-time system responsiveness.

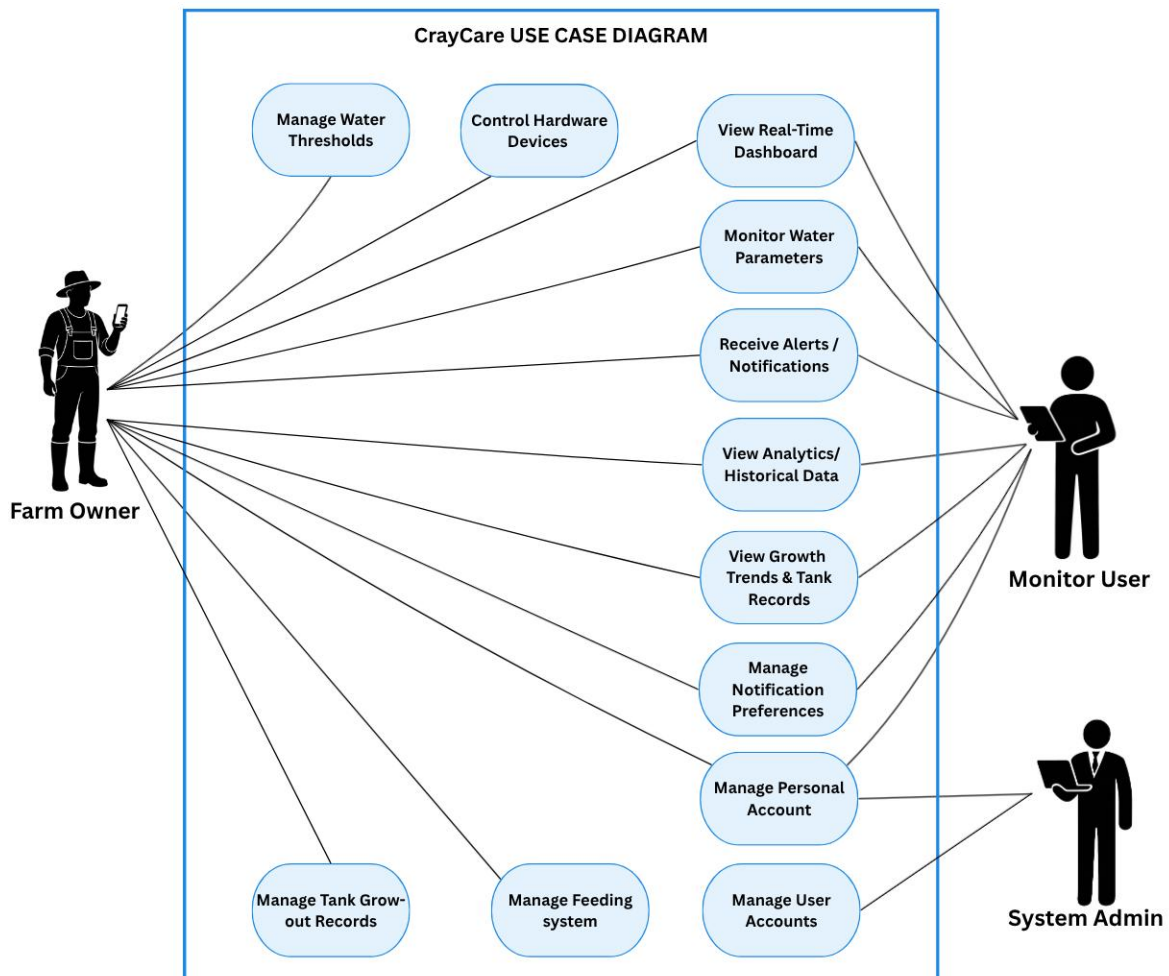
The hardware-related non-functional requirements focus on ensuring that the system components can operate consistently under aquaculture conditions. These include maintaining stable real-time monitoring, providing accurate sensor readings, supporting continuous data transmission, and ensuring that hardware

components can withstand the environmental conditions of the crayfish culture setup.

The software-related non-functional requirements focus on ensuring efficient system operation and user interaction. These include fast data processing, responsive dashboard updates, and a simple and user-friendly mobile interface to support effective monitoring and management activities.

These requirements serve as a basis for the development and assessment of the proposed system to ensure that the implemented features satisfy the expected operational and quality requirements.

### 3.1.2 Use Case Diagram







*Figure 3.1. Use Case Diagram*

Figure 3.1 illustrates the Use Case Diagram of CrayCare: An IoT and Mobile-Based Smart Aquaculture System for Crayfish Farming. The diagram presents the primary actors of the system and their respective interactions with the different functionalities of the mobile application. The diagram consists of three main actors namely the Farm Owner, the Monitor User, and the System Admin. The Farm Owner serves as the central and primary actor with full access to the system. The Farm Owner can monitor water parameters, view the real-time dashboard, receive alerts and notifications, control hardware devices, manage tank grow-out records, manage the feeding system, view analytics and historical data, view growth trends and tank records, manage water thresholds, manage their personal account, and configure notification preferences.

The Monitor User primarily provides monitoring support to the farm. This actor can monitor water parameters, view the real-time dashboard, receive alerts and notifications, view analytics and historical data, and view growth trends and tank records. In addition to these monitoring functions, the Monitor User is also granted access to manage their personal profile and notification preferences. On the other hand, the System Admin is responsible for the administrative aspects of the application through the Manage User Accounts use case, while also having access to manage their own personal profile.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

This Use Case Diagram demonstrates a well-structured role-based access control system. It clearly defines the responsibilities and permissions of each actor to ensure operational efficiency, security, and ease of use. The Farm Owner maintains full operational control of the crayfish farm while the Monitor User provides additional monitoring support. The System Admin handles the overall user management. Furthermore, the system provides personalization by allowing all actors to manage their own accounts, and enabling farm-level users to tailor their alert settings. Overall, the diagram reflects a user-centered design that addresses the real needs of crayfish farmers by simplifying daily operations through an integrated IoT mobile application.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

### 3.1.3 Use Case Report

Table 3.3

#### Use Case Report

| Use Case Name            | Actors                   | Description                                                                                                                                                                                        | Preconditions                                                                                                                                                                                                                                | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Postconditions                                                                                                                                                                                                                                | Exceptions                                                                                                                                                                                                  |
|--------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View Real-Time Dashboard | Farm Owner, Monitor User | Accesses dashboard to view live sensor readings, connection banners, tank inventory stats (biomass, stage with ABW/ABL, countdowns), and next feeding completion status with quick action buttons. | <ol style="list-style-type: none"> <li>1. User must be logged in.</li> <li>2. Dashboard section must be visible.</li> <li>3. Gauges and tank cards must exist in the system.</li> <li>4. Feeding schedule data must be available.</li> </ol> | <ol style="list-style-type: none"> <li>1. Dashboard loads with live Firebase values; connection status banner shows (yellow for waiting, red for disconnected).</li> <li>2. Displays Temp, pH, DO, Turbidity, and Water Level gauges with colored status badges (Optimal/Warning/Critical).</li> <li>3. Displays Tank Status card showing stock count, survival rate, ABW/ABL, biomass, and next sampling countdown.</li> <li>4. Displays Feeding Schedule card showing last fed time, next feed countdown, and daily completion progress.</li> <li>5. Displays horizontal Quick Actions row (Aerators, Pump, Feeder, Inventory, Sampling navigation).</li> <li>6. User can tap any gauge to open a detail panel with threshold legend and biological descriptions.</li> </ol> | <ol style="list-style-type: none"> <li>1. Gauges display live values with dynamic status colors.</li> <li>2. Tank status card and feeding schedule countdown update.</li> <li>3. Quick action buttons navigate to respective tabs.</li> </ol> | <ol style="list-style-type: none"> <li>1. No tank setup: tank fields show dashes.</li> <li>2. No sensor data: shows waiting banner.</li> <li>3. Sensor error: shows red connection error banner.</li> </ol> |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Continuation of Table 3.3

| Use Case Name            | Actors                   | Description                                                                                                                                                            | Preconditions                                                                                                                                                                                                 | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Postconditions                                                                                                                                                            | Exceptions                                                                                                                                                         |
|--------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Monitor Water Parameters | Farm Owner, Monitor User | Monitors Temp, pH, DO, Turbidity, and Water Level via live gauges with colored dots, ideal ranges, and opens detail panels for thresholds and biological descriptions. | <ol style="list-style-type: none"> <li>1. Dashboard rendered with all five parameter gauges.</li> <li>2. Sensor data streaming from Firebase.</li> <li>3. Threshold settings loaded from Firebase.</li> </ol> | <ol style="list-style-type: none"> <li>1. System displays gauges in rows: Temperature/pH/DO/Turbidity and Water Level standalone below.</li> <li>2. Each gauge shows parameter name, live value, unit, status badge, colored dot indicator, and trend arrow.</li> <li>3. User taps gauge card to open details.</li> <li>4. System opens detail panel showing parameter icon, live value, and three-tier legend (Optimal, Warning, Critical) with biological descriptions.</li> </ol> | <ol style="list-style-type: none"> <li>1. Gauges display live readings with correct status colors.</li> <li>2. Detail panel shows correct computed thresholds.</li> </ol> | <ol style="list-style-type: none"> <li>1. No sensor data: shows double dashes and grey dot.</li> <li>2. No threshold config: ideal range text is blank.</li> </ol> |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name            | Actors     | Description                                                                                                                                                  | Preconditions                                                                                                                                                                         | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                           | Postconditions                                                                                                                                                          | Exceptions                                                                                                                                                                                  |
|--------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Control Hardware Devices | Farm Owner | Controls connected Aerator 1, Aerator 2, and Water Pump by switching between OFF, AUTO, and ON modes. Changes save to Firebase and logs the last 20 actions. | <ol style="list-style-type: none"> <li>1. User must be on the Controls section.</li> <li>2. Devices tab must be visible.</li> <li>3. User must be logged in as Farm Owner.</li> </ol> | <ol style="list-style-type: none"> <li>1. User navigates to Controls section.</li> <li>2. System displays device cards showing current mode badge and runtime duration.</li> <li>3. Owner toggles mode (OFF, AUTO, ON); system saves changes to Firebase and updates UI.</li> <li>4. System logs the action in the activity log database.</li> <li>5. Tapping card header opens bottom sheet showing the last 20 log entries.</li> </ol> | <ol style="list-style-type: none"> <li>1. Selected device switches mode immediately.</li> <li>2. Mode change and log entries successfully saved to Firebase.</li> </ol> | <ol style="list-style-type: none"> <li>1. Firebase write fails: shows red error snackbar.</li> <li>2. Monitor User attempts control: all buttons disabled with read-only banner.</li> </ol> |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Continuation of Table 3.3

| Use Case Name         | Actors     | Description                                                                                                                                                    | Preconditions                                                                                                                                                                     | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                          | Postconditions                                                                                                                                                                      | Exceptions                                                                                                                                                                    |
|-----------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Manage Feeding System | Farm Owner | Configures time-based feeding schedules (add/edit/delete), triggers manual feed dispensing with status overlays, and tracks daily feeding completion progress. | <ol style="list-style-type: none"> <li>1. User must be on Controls section.</li> <li>2. Feeding tab must be visible.</li> <li>3. User must be logged in as Farm Owner.</li> </ol> | <ol style="list-style-type: none"> <li>1. User selects the Feeding tab.</li> <li>2. System displays feeder card with schedules active indicator and countdown.</li> <li>3. User adds schedule via time picker; system checks for duplicates.</li> <li>4. User taps 'Feed Now' to trigger manual dispensing.</li> <li>5. System sends command; full-screen overlay shows dispensing status and result (Feed Complete/Failed).</li> </ol> | <ol style="list-style-type: none"> <li>1. New feeding schedule saved and manual feed command executed.</li> <li>2. Feeder logs and progress display update successfully.</li> </ol> | <ol style="list-style-type: none"> <li>1. Duplicate schedule time: rejected with error snackbar.</li> <li>2. Feeder offline: Feed Now button shows offline status.</li> </ol> |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name                  | Actors                   | Description                                                                                                            | Preconditions                                                                                                                                                                           | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                             | Postconditions                                                                                                                                                                       | Exceptions                                                                                                                                                                               |
|--------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Receive Alerts & Notifications | Farm Owner, Monitor User | Receives real-time push alerts and views categorized logs (Critical, Warning, Operational, Reminders) with KPI counts. | <ol style="list-style-type: none"> <li>1. User must be logged in.</li> <li>2. User must not be System Admin.</li> <li>3. Notification permissions must be granted on device.</li> </ol> | <ol style="list-style-type: none"> <li>1. User navigates to the Notifications section.</li> <li>2. System computes KPI summary row and categorized filter tabs.</li> <li>3. System renders list grouped by date, showing type-colored icons.</li> <li>4. User taps notification to view details and mark it as read.</li> <li>5. New notifications arrive in real-time via Firebase listener or background FCM.</li> </ol> | <ol style="list-style-type: none"> <li>1. Alerts delivered and read states updated in database.</li> <li>2. Unread notification badge count updates on bottom navigation.</li> </ol> | <ol style="list-style-type: none"> <li>1. Device offline: notification delivery is delayed.</li> <li>2. Push notifications disabled: alerts only visible inside the app list.</li> </ol> |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name                    | Actors                   | Description                                                                                                                            | Preconditions                                                                                                                                                                    | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Postconditions                                                                                                                                                                                    | Exceptions                                                                                                                                                                                                                                              |
|----------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View Analytics & Historical Data | Farm Owner, Monitor User | Views historical sensor line charts and min/max/avg stats across Live, 24H, 7D, 30D, and Custom date ranges.                           | <ol style="list-style-type: none"> <li>1. User must be logged in.</li> <li>2. Analytics screen loaded with chart cards.</li> <li>3. Firebase history data must exist.</li> </ol> | <ol style="list-style-type: none"> <li>1. User selects a time range filter.</li> <li>2. System loads historical data points from Firebase.</li> <li>3. System renders line charts with minimum, maximum, and average statistics.</li> <li>4. User taps a chart card to open enlarged fullscreen view.</li> </ol>                                                                                                                                                             | <ol style="list-style-type: none"> <li>1. Historical sensor trend line charts displayed.</li> <li>2. Min/Max/Avg statistics computed and shown.</li> </ol>                                        | <ol style="list-style-type: none"> <li>1. No history data: chart shows empty/zero flat line.</li> <li>2. Query timeout: chart shows empty dataset.</li> </ol>                                                                                           |
| Manage Tank Grow-out Records     | Farm Owner               | Performs tank initialization setup, logs weekly sampling weight and length measurements, and records mortality events to track growth. | <ol style="list-style-type: none"> <li>1. Tanks section visible and Inventory tab selected.</li> <li>2. User must be logged in as Farm Owner.</li> </ol>                         | <ol style="list-style-type: none"> <li>1. Owner initializes tank with stock count and stocking date.</li> <li>2. Owner enters periodic sampling data (sample count, total weight, total length) under Sampling tab; system computes ABW, ABL, and estimated biomass.</li> <li>3. Owner logs mortality by entering death count under Inventory tab.</li> <li>4. System automatically subtracts the deaths from the live count and updates the survival percentage.</li> </ol> | <ol style="list-style-type: none"> <li>1. Tank initialization, sampling records, and mortality logs saved in Firebase.</li> <li>2. Live count and survival rates updated in real-time.</li> </ol> | <ol style="list-style-type: none"> <li>1. No initialization: shows empty state prompt to initialize.</li> <li>2. Mortality exceeds live count: system blocks entry.</li> <li>3. Monitor User: all setup, edit, and mortality forms disabled.</li> </ol> |





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name                     | Actors                   | Description                                                                                                                                                                                                           | Preconditions                                                                                                                              | Flow of Events                                                                                                                                                                                                                                                                                                                                                                                                                                     | Postconditions                                                                                                         | Exceptions                                                                                                                                                                                                                                                                  |
|-----------------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| View Growth Trends & Tank Records | Farm Owner, Monitor User | Reviews growth trends (ABW/ABL, biomass, mortality, growth stage progress) under the Trends tab, and views tank overview records (stocking details, survival rate) and sampling history logs under the Inventory tab. | 1. Trends and Inventory tabs are accessible.                                                                                               | <ol style="list-style-type: none"> <li>1. User opens the Trends tab.</li> <li>2. System displays growth trend (ABW/ABL toggle) and estimated biomass charts.</li> <li>3. System displays mortality bar chart over grow-out intervals.</li> <li>4. System displays Growth Stage progress bar with current metrics.</li> <li>5. User views tank records (survival rate, alive/dead count) and sampling history logs in the Inventory tab.</li> </ol> | 1. Growth, biomass, mortality graphs, stage progress, tank records, and sampling history logs successfully visualized. | <ol style="list-style-type: none"> <li>1. No tank setup: Trends tab displays a 'No Trend Data' placeholder, and Inventory tab shows an empty tank setup state.</li> <li>2. Single record: Weekly comparison stats and weekly sampling metrics are not displayed.</li> </ol> |
| Manage Water Thresholds           | Farm Owner               | Modifies minimum and maximum sensor thresholds per growth stage.                                                                                                                                                      | <ol style="list-style-type: none"> <li>1. User must be on the Settings screen.</li> <li>2. User must be logged in as Farm Owner</li> </ol> | <ol style="list-style-type: none"> <li>1. Owner navigates to the threshold settings section.</li> <li>2. Owner modifies sensor thresholds (min/max limits) per growth stage and saves.</li> <li>3. System saves to Firebase and updates app displays globally in real-time.</li> </ol>                                                                                                                                                             | 1. Threshold ranges successfully updated in the system.                                                                | 1. Non-owner attempts to edit thresholds: options are disabled in UI.                                                                                                                                                                                                       |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name                   | Actors                                 | Description                                                                         | Preconditions                                                                                                                                                              | Flow of Events                                                                                                                                                                                                                                                            | Postconditions                                                                                                         | Exceptions                                                                                                                                                      |
|---------------------------------|----------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Manage Personal Account         | Farm Owner, Monitor User, System Admin | Updates personal profile details such as name, photo, and changes account password. | <ol style="list-style-type: none"> <li>1. User must be logged in.</li> <li>2. User must be on the Profile Settings screen.</li> </ol>                                      | <ol style="list-style-type: none"> <li>1. User modifies profile name or uploads a new profile photo.</li> <li>2. User enters current password to request a password change.</li> <li>3. System saves the updated profile details and new password to Firebase.</li> </ol> | <ol style="list-style-type: none"> <li>1. User profile information and password updated.</li> </ol>                    | <ol style="list-style-type: none"> <li>1. Password change fails: shows reauthentication error.</li> </ol>                                                       |
| Manage Notification Preferences | Farm Owner, Monitor User               | Toggles notification preferences such as sounds and vibration for alerts.           | <ol style="list-style-type: none"> <li>1. User must be logged in (as Farm Owner or Monitor User).</li> <li>2. User must be on the Notification Settings screen.</li> </ol> | <ol style="list-style-type: none"> <li>1. User toggles notification sound/vibration preferences</li> <li>2. System saves preferences and applies them to incoming alerts.</li> </ol>                                                                                      | <ol style="list-style-type: none"> <li>1. Notification delivery behavior updated based on user preferences.</li> </ol> | <ol style="list-style-type: none"> <li>1. Device-level notification permissions are disabled: system prompts the user to enable them in OS settings.</li> </ol> |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Continuation of Table 3.3*

| Use Case Name        | Actors       | Description                                                                                                                        | Preconditions                              | Flow of Events                                                                                                                                                                                                                                | Postconditions                                                            | Exceptions                                                                                    |
|----------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Manage User Accounts | System Admin | Views all registered user profiles with stats, filters by role, searches by name/email, and modifies user roles or account status. | 1. User must be logged in as System Admin. | 1. System loads registered users list and status summaries.<br>2. Admin searches or filters list by role.<br>3. Admin changes user role or disables account status.<br>4. Disabled users are automatically logged out on their next activity. | 1. User role and active/disable d status updated in Firebase immediately. | 1. Firebase write fails: change not applied.<br>2. Non-admin attempts access: access blocked. |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Table 3.3 shows the Use Case Report of CrayCare. The report presents the detailed interactions between the three user roles — Farm Owner, Monitor User, and System Admin — and the different functional features of the system. It describes the specific use cases, including monitoring water parameters, viewing the real-time dashboard, controlling hardware devices, managing the feeding system, receiving alerts and notifications, viewing analytics and historical data, managing tank grow-out records, viewing growth trends and tank records, managing water thresholds, managing personal profiles, managing notification preferences, and managing user accounts. Each use case includes the corresponding actor, description, preconditions, flow of events, postconditions, and possible exceptions to clearly define the expected behavior and functionality of the system.

The Use Case Report serves as a detailed functional reference that explains how each user role interacts with the system during different operations and monitoring activities. It also identifies the required conditions before a process can be performed, the sequence of actions executed by the system, the expected outputs after each process, and the possible errors or limitations that may occur during operation. Through this report, the overall operational workflow and functional scope of the CrayCare system is established.



### 3.2 DESIGN SPECIFICATIONS

This section presents the design specifications of the proposed CrayCare system. It describes the overall structure, components, and technical design used in the development of the system. The section includes the activity diagrams, graphical user interface (GUI) designs, database schema, and data dictionary, which collectively illustrate the system processes, user interaction, data organization, and backend structure. In addition, the hardware design and physical setup of the IoT components are presented to show the arrangement and integration of the sensors, devices, and monitoring equipment used in the system implementation.



### 3.2.1 Activity Diagram

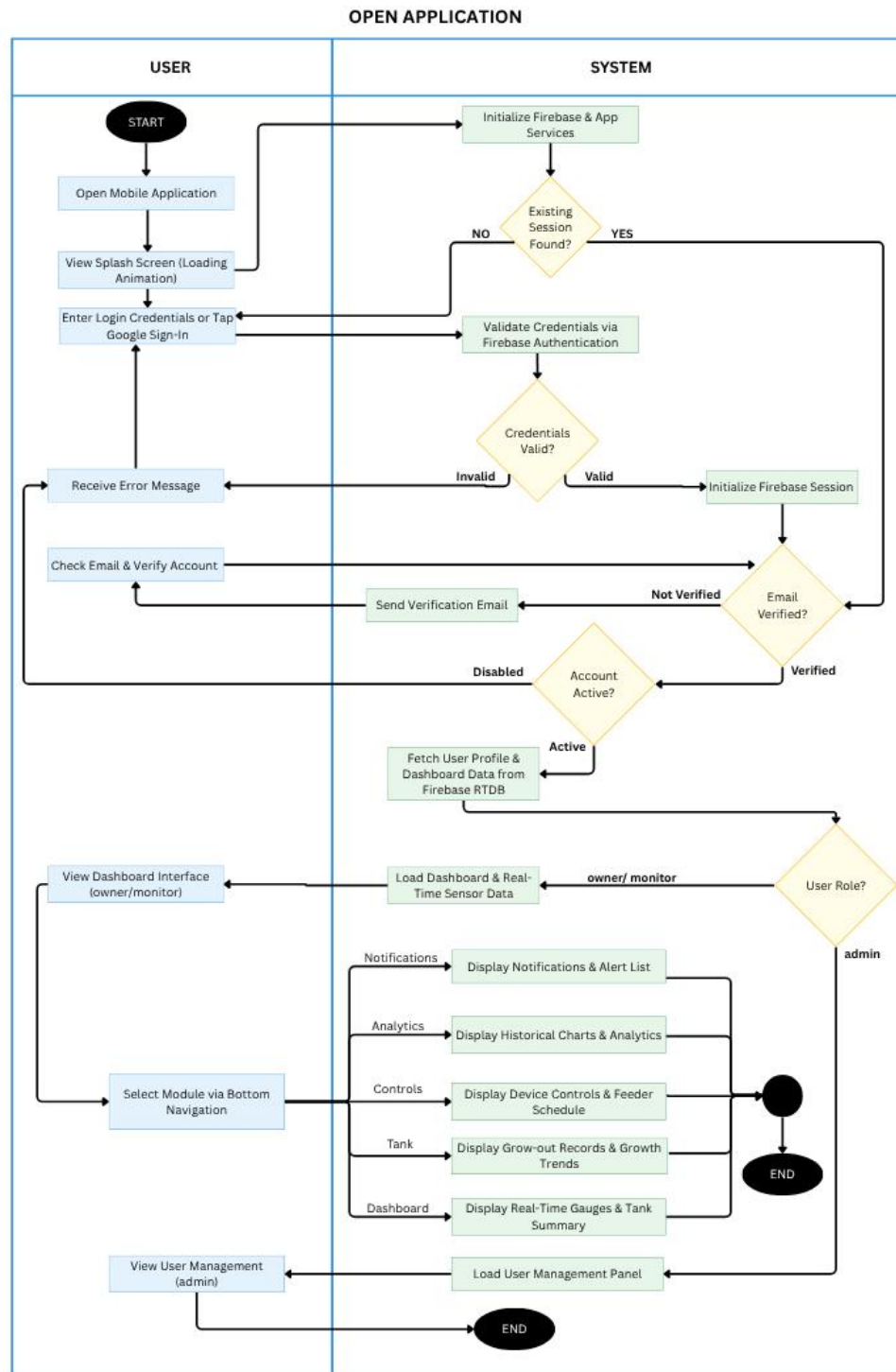


Figure 3.2. Activity Diagram for Open Mobile Application



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.2 illustrates the process of opening the CrayCare mobile application. Upon launch, the system initializes Firebase and app services while displaying a splash screen. It then checks for an existing session. If none is found, the user must enter login credentials or use Google Sign-In for Firebase Authentication. Invalid credentials display an error, while valid ones initialize a session. The system then verifies if the email is confirmed and whether the account is active. Unverified users receive a verification email, and disabled accounts are shown an error message. Active and verified accounts proceed to fetch user profile and dashboard data from Firebase.

After loading, the system checks the user's role to determine the appropriate interface. Owner and monitor users are directed to the main dashboard with real-time sensor data and bottom navigation for accessing modules such as Notifications, Analytics, Controls, Tank, and Dashboard. Admin users bypass the standard dashboard and are directed to the User Management panel for managing registered accounts.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

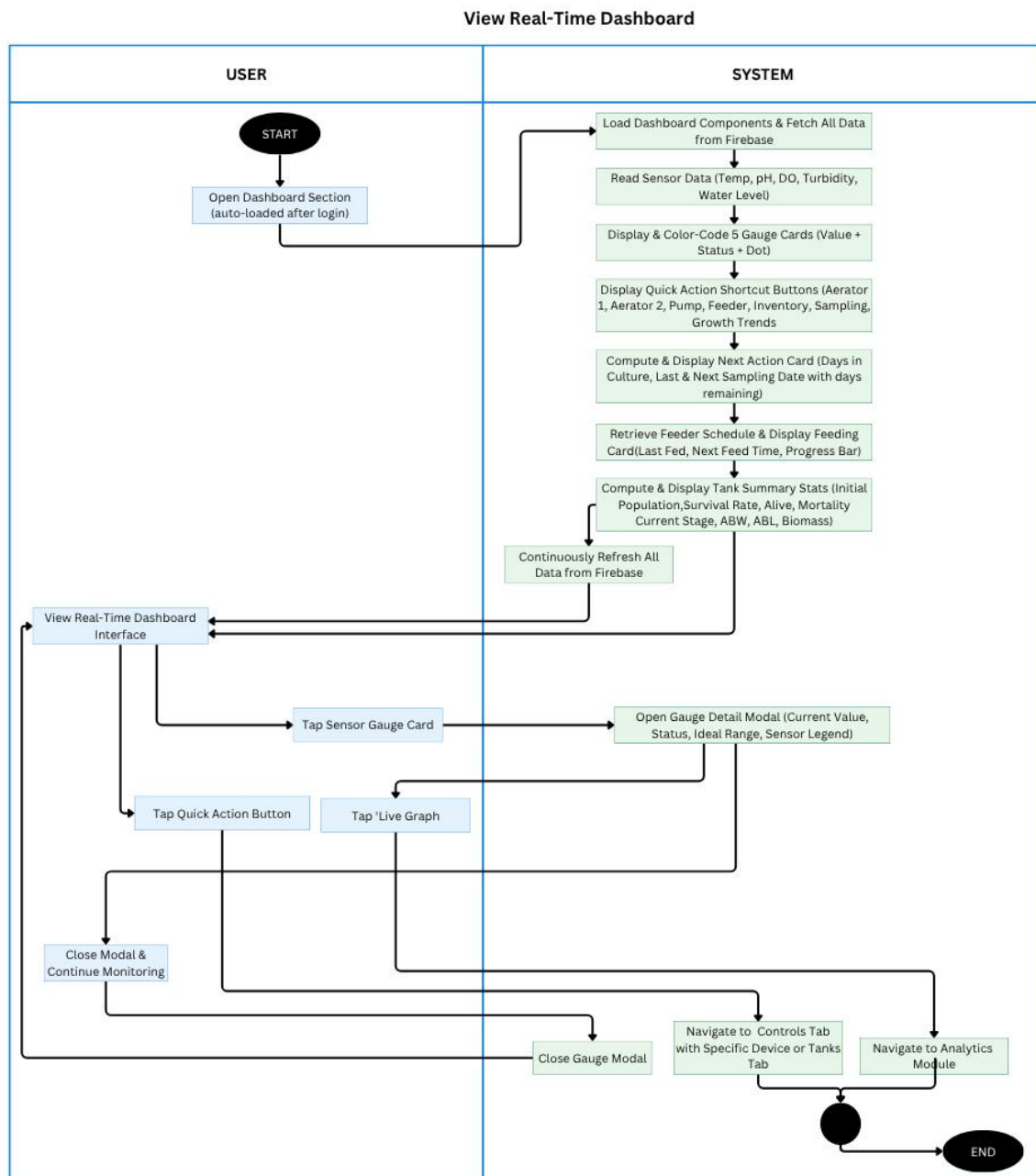


Figure 3.3. Activity Diagram for View Real-Time Dashboard

Figure 3.3 illustrates the activity diagram for the View Real-Time Dashboard use case in the CrayCare system. This diagram presents the step-by-

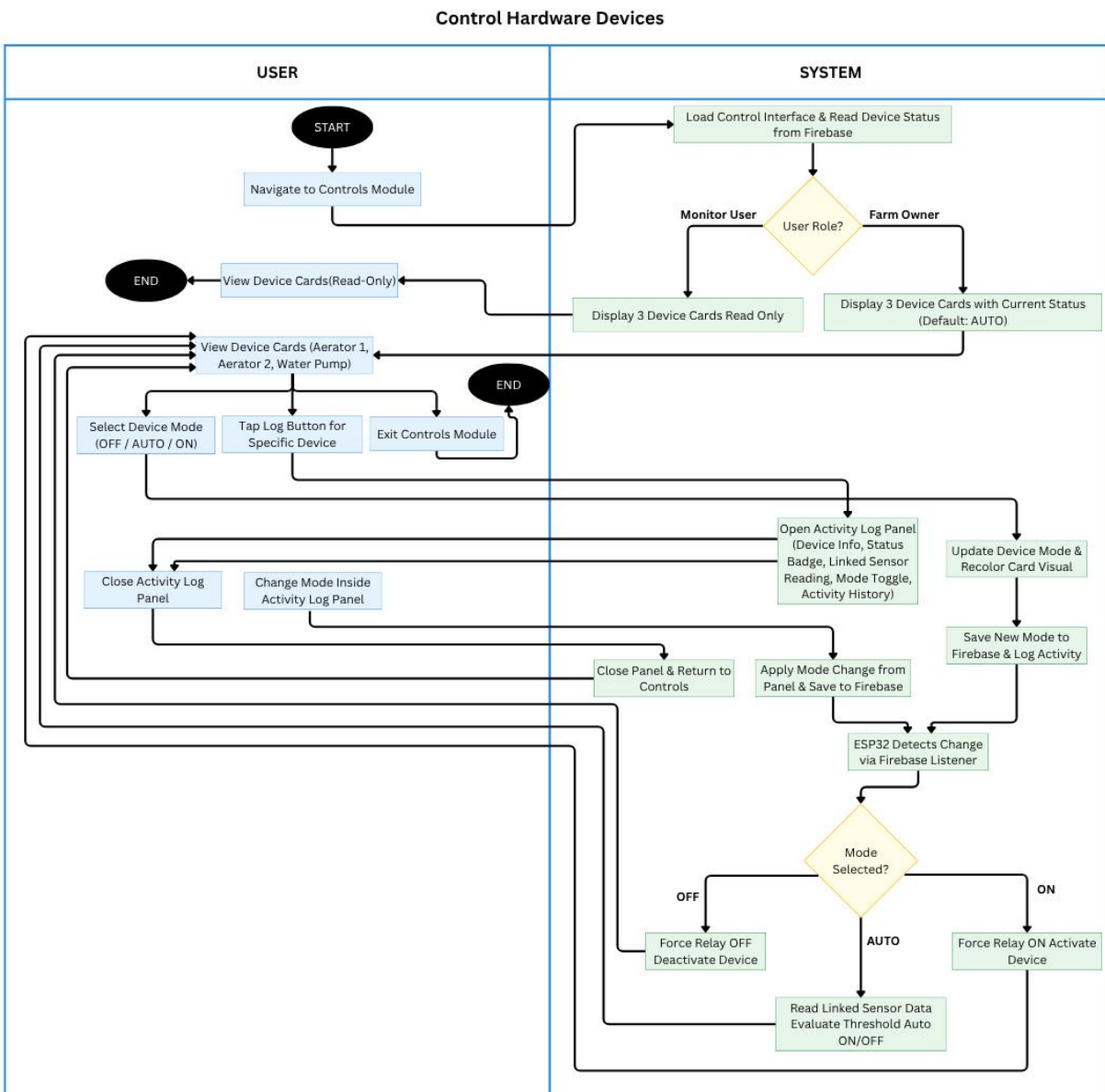




## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

step interaction between the user and the system when accessing the main dashboard after login. The process begins with the user opening the Dashboard section, which auto-loads after successful authentication. The system loads dashboard components, fetches real-time data from Firebase, reads sensor values for Temperature, pH, Dissolved Oxygen, Turbidity, and Water Level, and displays them as color-coded gauge cards with status indicators. It also shows quick action buttons, computes and displays the Next Action Card and Feeding Schedule Card with progress details, and presents Tank Summary Statistics including population, survival rate, growth stage, ABW, ABL, and biomass. The dashboard continuously refreshes data from Firebase to support ongoing real-time monitoring.

Users can interact with the dashboard by tapping sensor gauge cards to open detail modals showing current values, status, ideal ranges, and biological legends, tapping quick action buttons to navigate to Controls, Tanks, or Analytics modules, or tapping live graphs for enlarged views. After viewing details, users can close modals and continue monitoring. The session ends when the user navigates away from the dashboard.



*Figure 3.4. Activity Diagram for Control Hardware Devices*

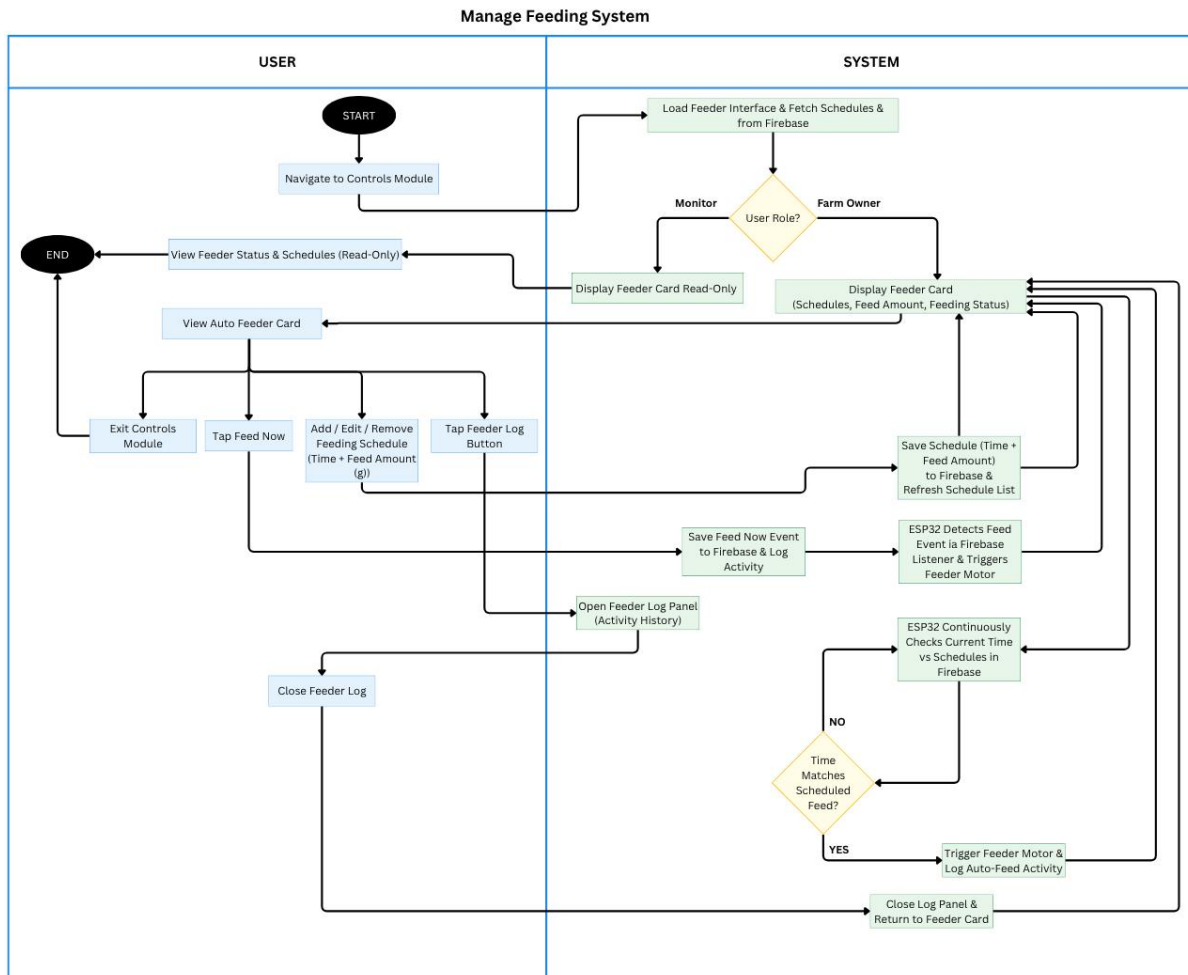
Figure 3.4 illustrates the activity diagram for the Control Hardware Devices use case in the CrayCare system. This diagram shows the interaction between the user and the system when managing hardware devices. The process starts when



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

the user navigates to the Controls Module. The system loads the control interface and reads device status from Firebase, then checks the user role. For Monitor Users, it displays three device cards (Aerator 1, Aerator 2, and Water Pump) in read-only mode. For Farm Owners, it displays the device cards with current status (default AUTO). Farm Owners can view device cards, select device mode (OFF/AUTO/ON), tap the log button for a specific device, or exit the module.

When a Farm Owner changes the mode, the system opens the activity log panel, updates the device mode and saves it to Firebase along with the log activity. The ESP32 detects the change via Firebase listener and applies the selected mode by forcing the relay OFF, setting to AUTO (with sensor threshold evaluation), or forcing the relay ON. Users can also open and close the activity log panel to view device info, status, and history. The session ends when the user exits the Controls Module or views devices in read-only mode as a Monitor User.



*Figure 3.5. Activity Diagram for Manage Feeding System*

Figure 3.5 illustrates the activity diagram for the Manage Feeding System use case in the CrayCare system. The process begins when the crayfish farmer navigates to the Controls Module. The system loads the feeder interface and fetches schedules from Firebase, then checks the user role. For Monitor Users, it displays the feeder card in read-only mode. For Farm Owners, it displays the feeder card showing schedules, feed amount, and feeding status. Farm Owners



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

can view the auto feeder card, tap Feed Now for manual dispensing, add/edit/remove feeding schedules (specifying time and feed amount in grams), tap the feeder log button, or exit the module.

When schedules are modified, the system saves the changes including time and feed amount to Firebase and refreshes the list. Tapping Feed Now saves the event and triggers the feeder motor via ESP32. The system continuously checks the current time against scheduled feeds; when a match occurs, it automatically triggers the feeder motor and logs the activity. Users can open and close the feeder log panel to view activity history. The session ends when the user exits the Controls Module or returns to the feeder card after closing the log panel.

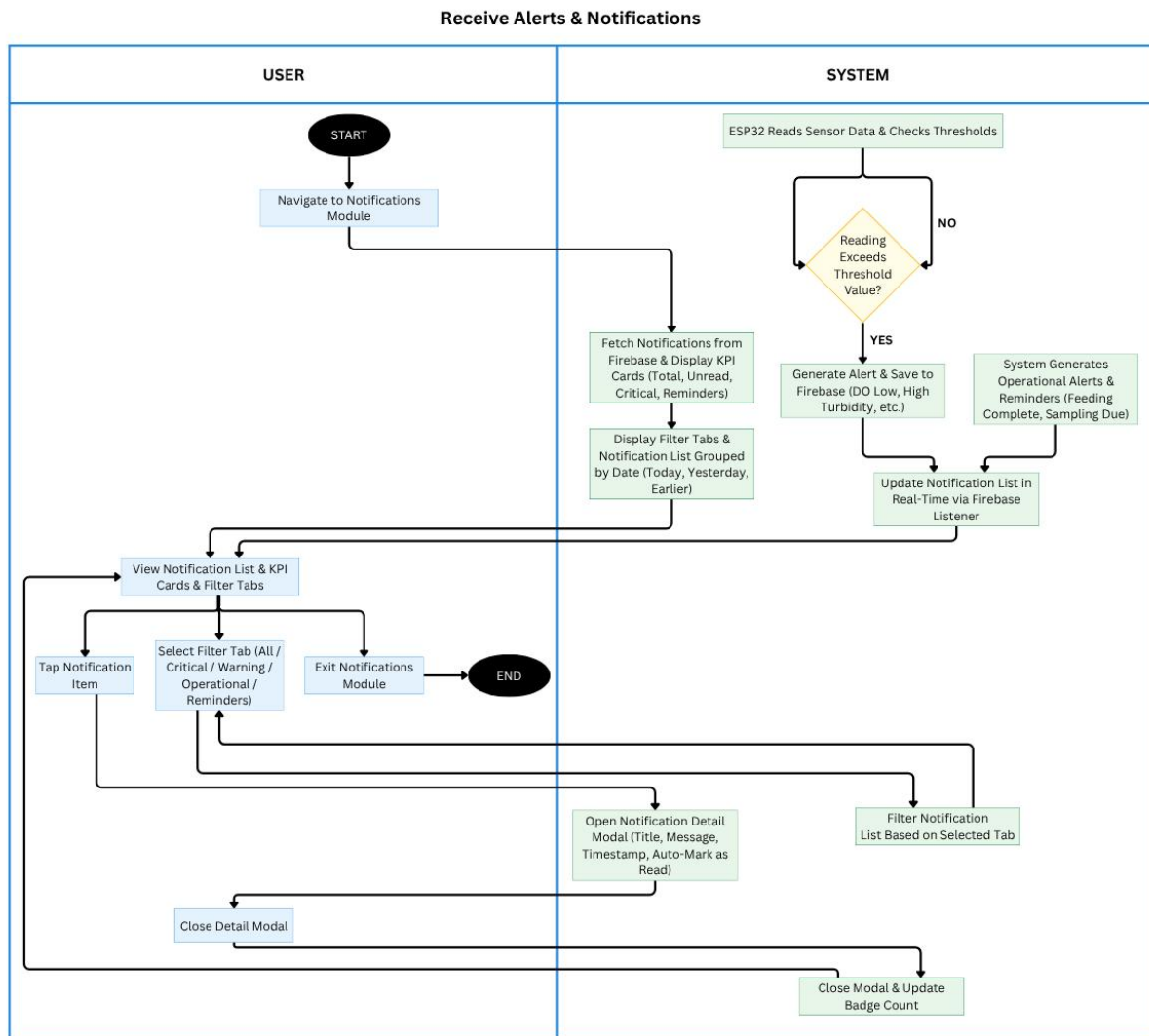


Figure 3.6. Activity Diagram for Receive Alerts and Notifications

Figure 3.6 illustrates the activity diagram for the Receive Alerts & Notifications use case in the CrayCare system. The process begins when the user navigates to the Notifications Module. On the system side, the ESP32 continuously reads sensor data and checks against defined thresholds. If a reading exceeds the threshold value, the system generates an alert, saves it to Firebase, and produces



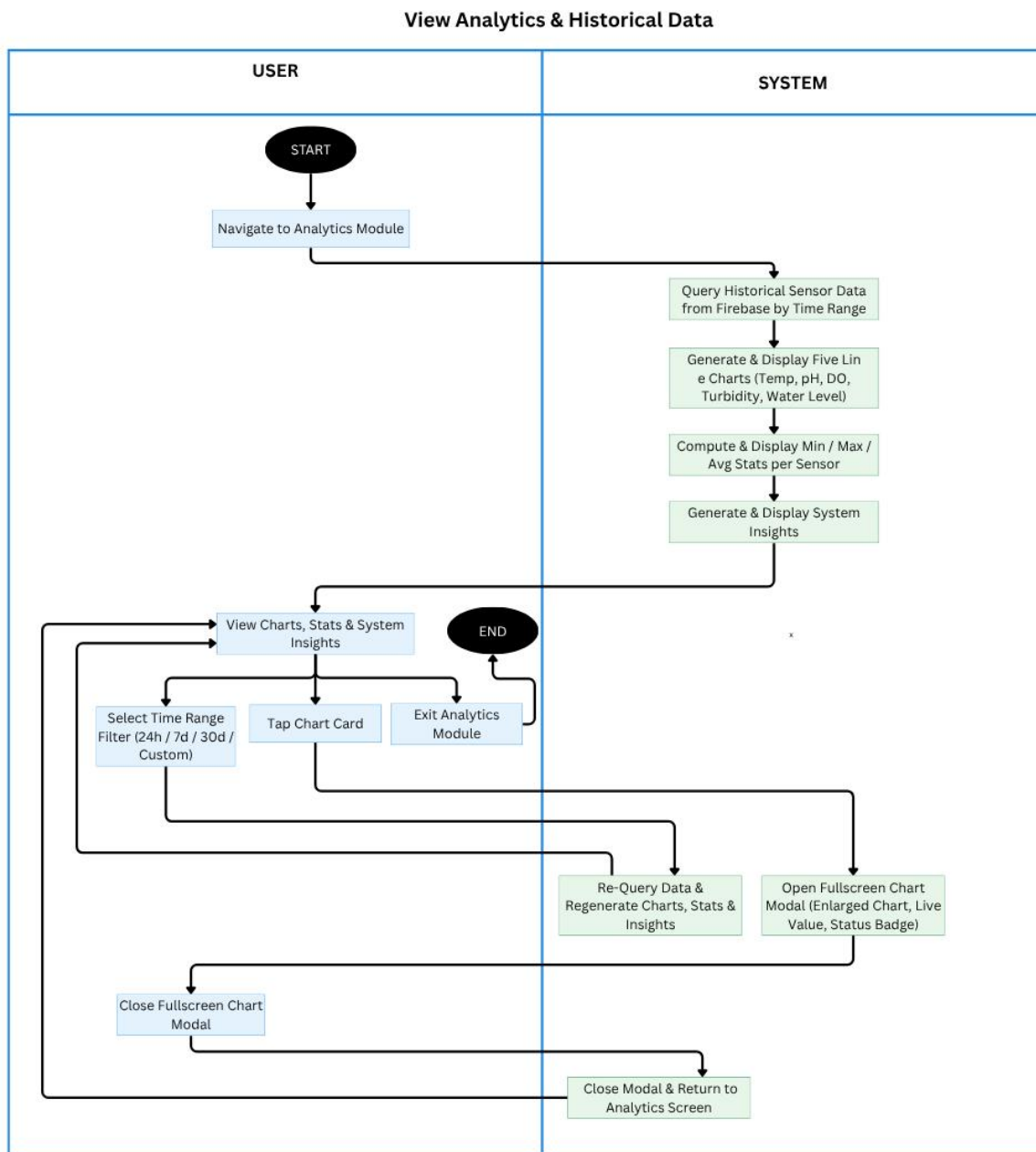
## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

operational alerts and reminders (such as feeding complete or sampling due). The system then fetches notifications from Firebase, displays KPI cards (Total, Unread, Critical, Reminders), and shows the notification list grouped by date with filter tabs (Today, Yesterday, Earlier). The notification list is updated in real-time via Firebase listener.

Users can view the notification list and KPI cards, select filter tabs (All, Critical, Warning, Operational, Reminders), tap a notification item to open the detail modal (showing title, message, timestamp, and auto-mark as read), or exit the module. After closing the detail modal, the system updates the badge count. The session ends when the user exits the Notifications Module.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES



*Figure 3.7. Activity Diagram for View Analytics and Historical Data*

Figure 3.7 illustrates the activity diagram for the View Analytics & Historical Data use case in the CrayCare system. The process begins when the user





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

navigates to the Analytics Module. The system queries historical sensor data from Firebase based on the selected time range, generates and displays five line charts for Temperature, pH, Dissolved Oxygen, Turbidity, and Water Level, computes and shows minimum, maximum, and average statistics per sensor, and generates system insights. Users can view the charts, statistics, and insights on the main analytics screen.

Users may select a time range filter (24h, 7d, 30d, or Custom), tap a chart card to open a fullscreen modal showing the enlarged chart with live value and status badge, or exit the module. Upon selecting a new time range or reopening a chart, the system re-queries data and regenerates the charts, statistics, and insights. After closing the fullscreen chart modal, the system returns to the main analytics screen. The session ends when the user exits the Analytics Module.

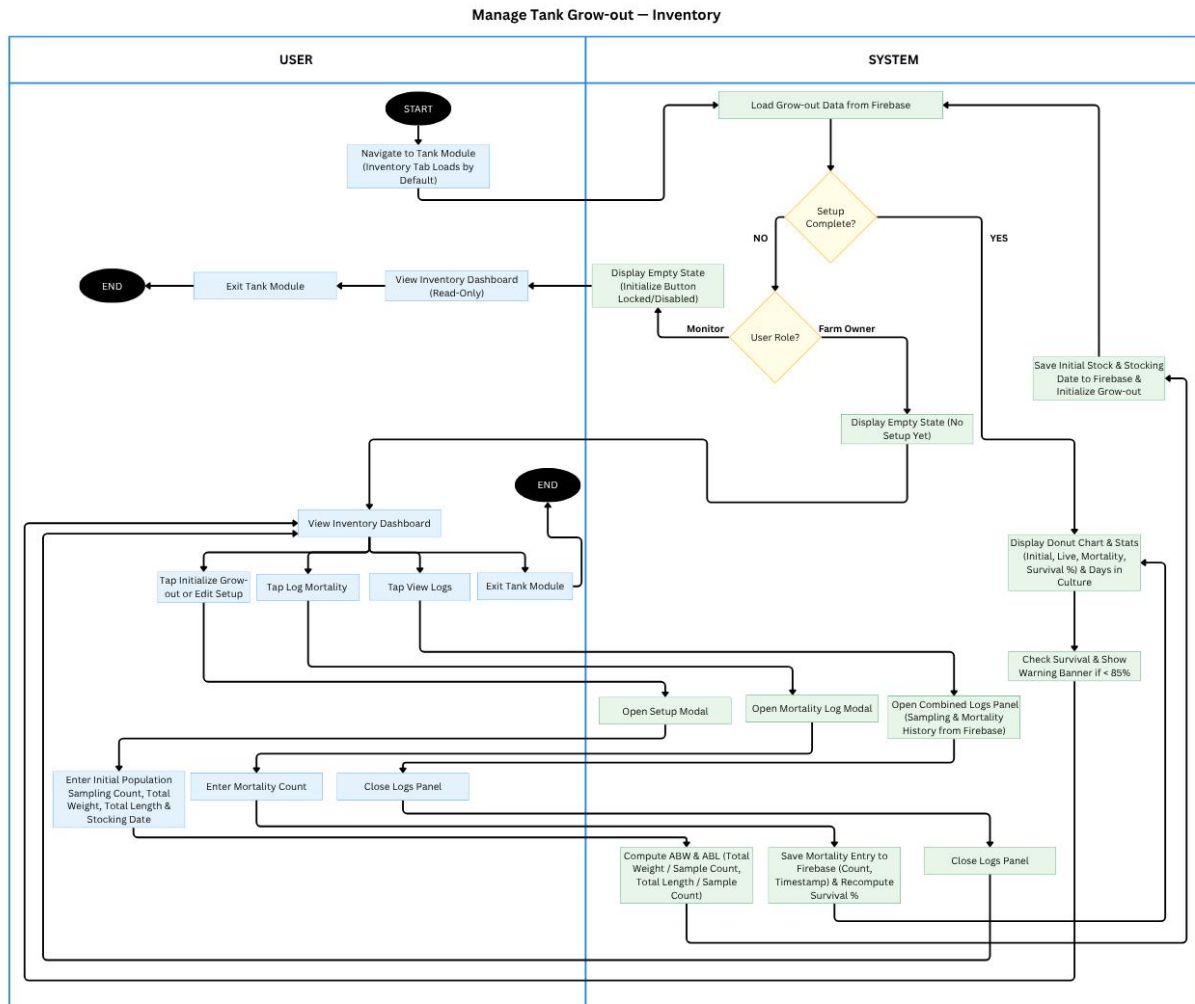


Figure 3.8. Activity Diagram for Manage Tank Grow-out — Inventory

Figure 3.8 illustrates the activity diagram for the Manage Tank Grow-out – Inventory use case in the CrayCare system. The process starts when the user navigates to the Tank Module (Inventory tab loads by default). The system loads grow-out data from Firebase and checks if setup is complete. If not, Monitor Users see a read-only empty state, while Farm Owners can initialize or edit setup by



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

entering initial population, sampling count, total weight, total length, and stocking date.

Once setup is done, the system displays the inventory dashboard with donut chart and statistics (initial stock, live count, mortality, survival rate, and days in culture). Farm Owners can tap to log mortality by entering count, view sampling and mortality logs, or exit the module. The system computes ABW and ABL, saves entries to Firebase, and updates survival percentage in real-time. The session ends when the user exits the Tank Module.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

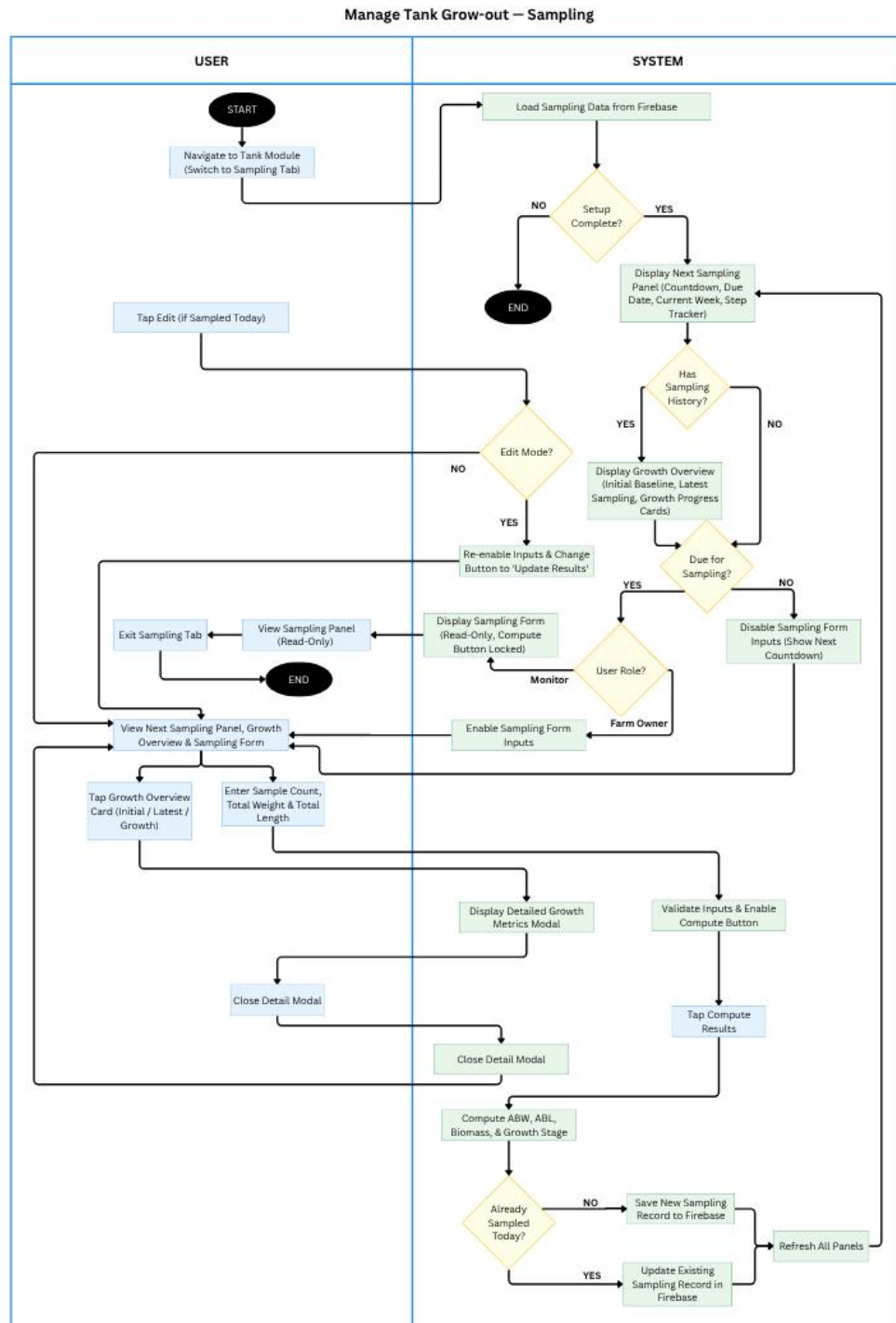


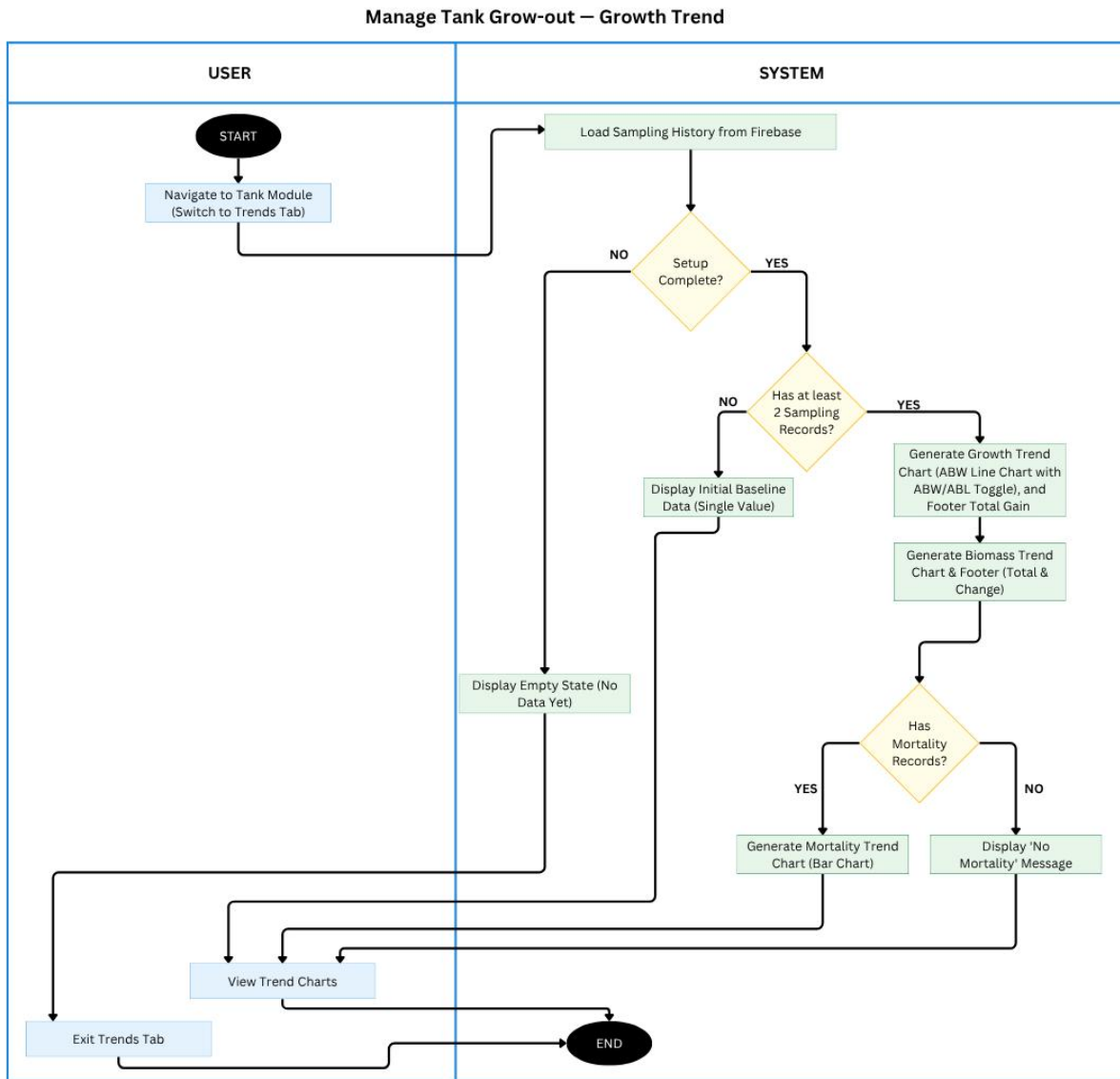
Figure 3.9. Activity Diagram for Manage Tank Grow-out — Sampling



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.9 illustrates the activity diagram for the Manage Tank Grow-out – Sampling use case in the CrayCare system. The process begins when the user navigates to the Tank Module and switches to the Sampling Tab. The system loads sampling data from Firebase and checks if the initial setup is complete. If setup is not done, the process ends. Otherwise, it displays the Next Sampling Panel with countdown, due date, and step tracker.

Farm Owners can enter sampling data (sample count, total weight, and total length). The system validates inputs, computes ABW, ABL, biomass, and growth stage, then saves the new sampling record to Firebase or updates the existing one if already sampled today. Users can view the Growth Overview panel, tap to see detailed growth metrics modal, or exit the tab. Monitor Users have read-only access. The session ends after saving the record and refreshing all related panels.



*Figure 3.10. Activity Diagram for Manage Tank Grow-out — Growth Trend*

Figure 3.10 illustrates the activity diagram for the Manage Tank Grow-out – Growth Trend use case in the CrayCare system. The process begins when the user navigates to the Tank Module and switches to the Trends Tab. The system



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

loads sampling history from Firebase and checks if the initial setup is complete. If setup is not done, it displays an empty state. If there are fewer than two sampling records, it shows only the initial baseline data.

Once there are at least two sampling records, the system generates the Growth Trend chart (ABW/ABL line chart with toggle and footer total gain), Biomass Trend chart, and Mortality Trend chart (bar chart) if mortality records exist. Users can view all trend charts or exit the Trends Tab. The session ends when the user exits the tab.

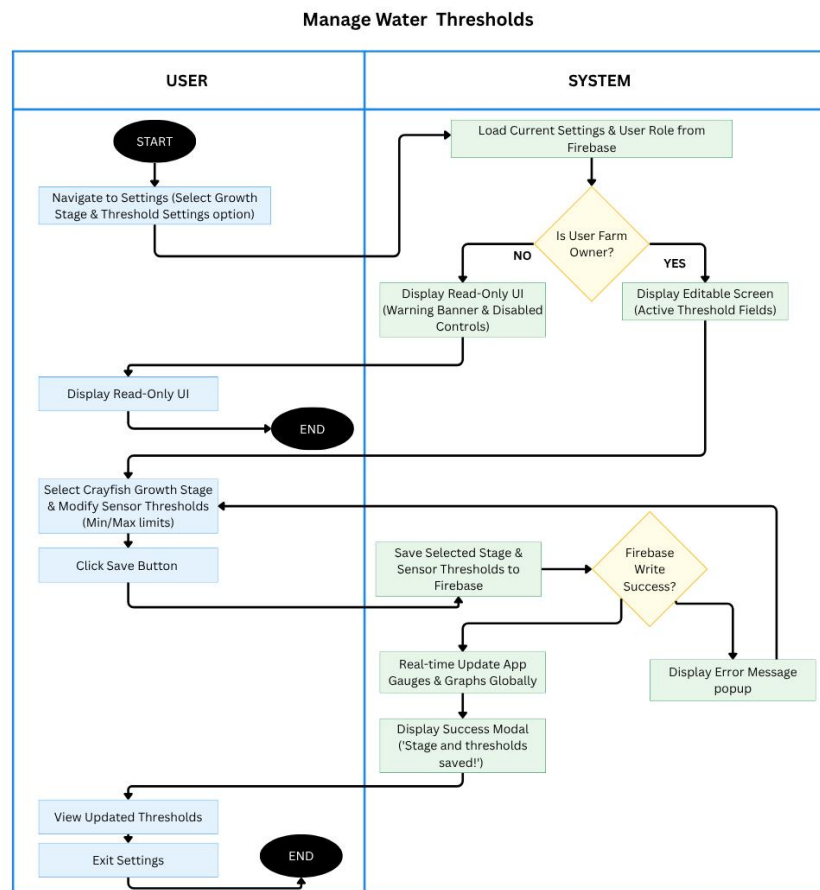


Figure 3.11. Activity Diagram for Manage Water Thresholds



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.11 illustrates the activity diagram for the Manage Water Thresholds use case in the CrayCare system. The process begins when the user navigates to Settings and selects the Threshold Settings option. The system loads current threshold settings and checks the user role from Firebase. Non-owner users (Monitor) see a read-only interface with a warning banner and disabled controls. Farm Owners are shown an editable screen with active threshold fields. The Farm Owner selects a crayfish growth stage and modifies the minimum and maximum sensor thresholds, then clicks the Save button. The system saves the updated values to Firebase. Upon successful write, it performs a real-time update to all gauges and graphs across the app and displays a success modal. If the save fails, an error message is shown. The user can view the updated thresholds before exiting the Settings screen. The session ends upon exit.



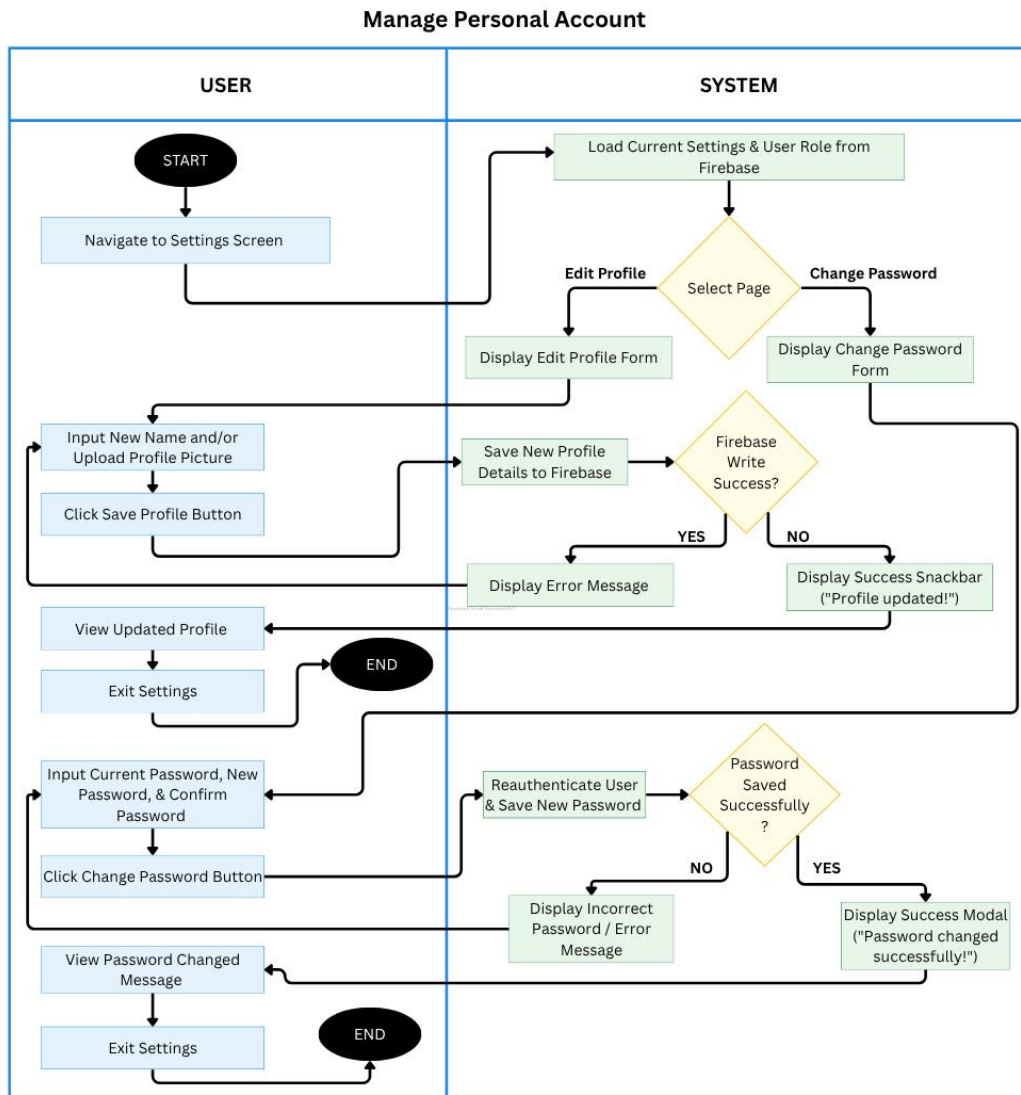


Figure 3.12. Activity Diagram for Manage Personal Account

Figure 3.12 illustrates the activity diagram for the Manage Personal Account use case in the CrayCare system. The process begins when the user navigates to the Settings Screen. The system loads current settings and user role from Firebase. Users can choose to edit their profile or change password. For



profile editing, users input a new name and/or upload a profile picture, then click the Save button. The system saves the details to Firebase and displays a success snackbar or error message accordingly. Users can view the updated profile before exiting.

For password change, users input their current password, new password, and confirmation, then click the Change Password button. The system reauthenticates the user and saves the new password. Upon success, it shows a success modal; otherwise, it displays an incorrect password error. The session ends when the user exits the Settings screen.

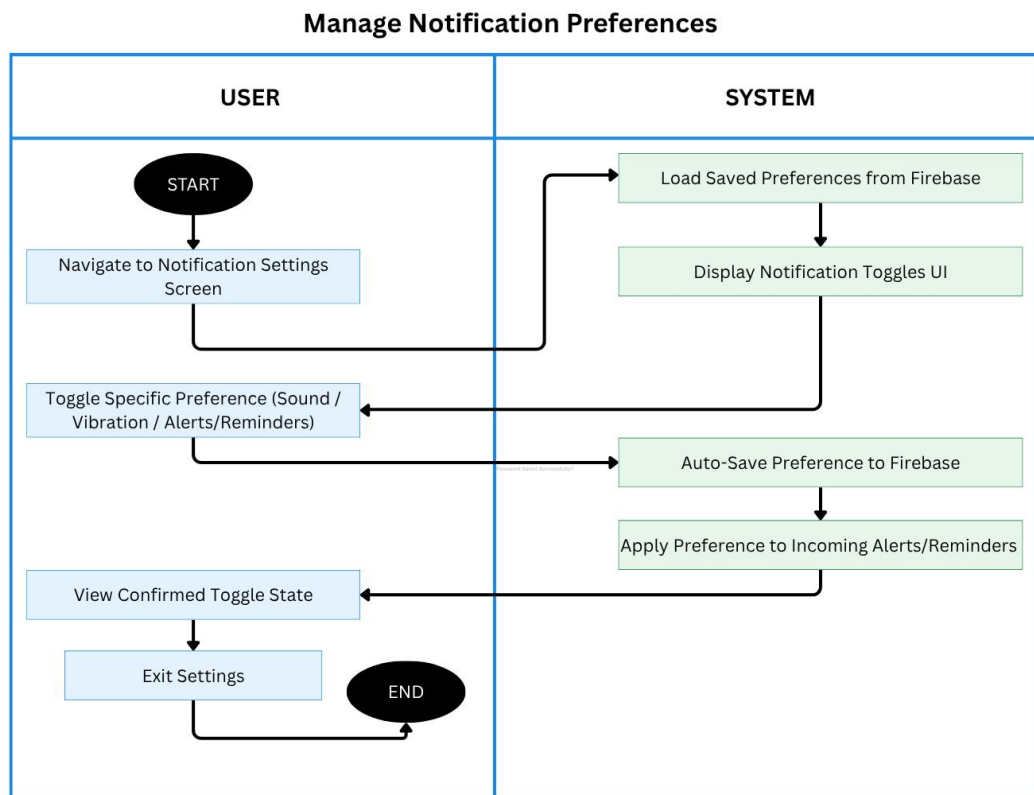


Figure 3.13. Activity Diagram for Manage Notification Preferences



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.13 illustrates the activity diagram for the Manage Notification Preferences use case in the CrayCare system. The process begins when the user navigates to the Notification Settings Screen. The system loads the saved preferences from Firebase and displays the notification toggles UI.

The user can toggle specific preferences for Sound, Vibration, or Alerts/Reminders. The system automatically saves the changes to Firebase and applies the updated preferences to incoming alerts and reminders. The user views the confirmed toggle states and can exit the Settings screen. The session ends upon exiting.

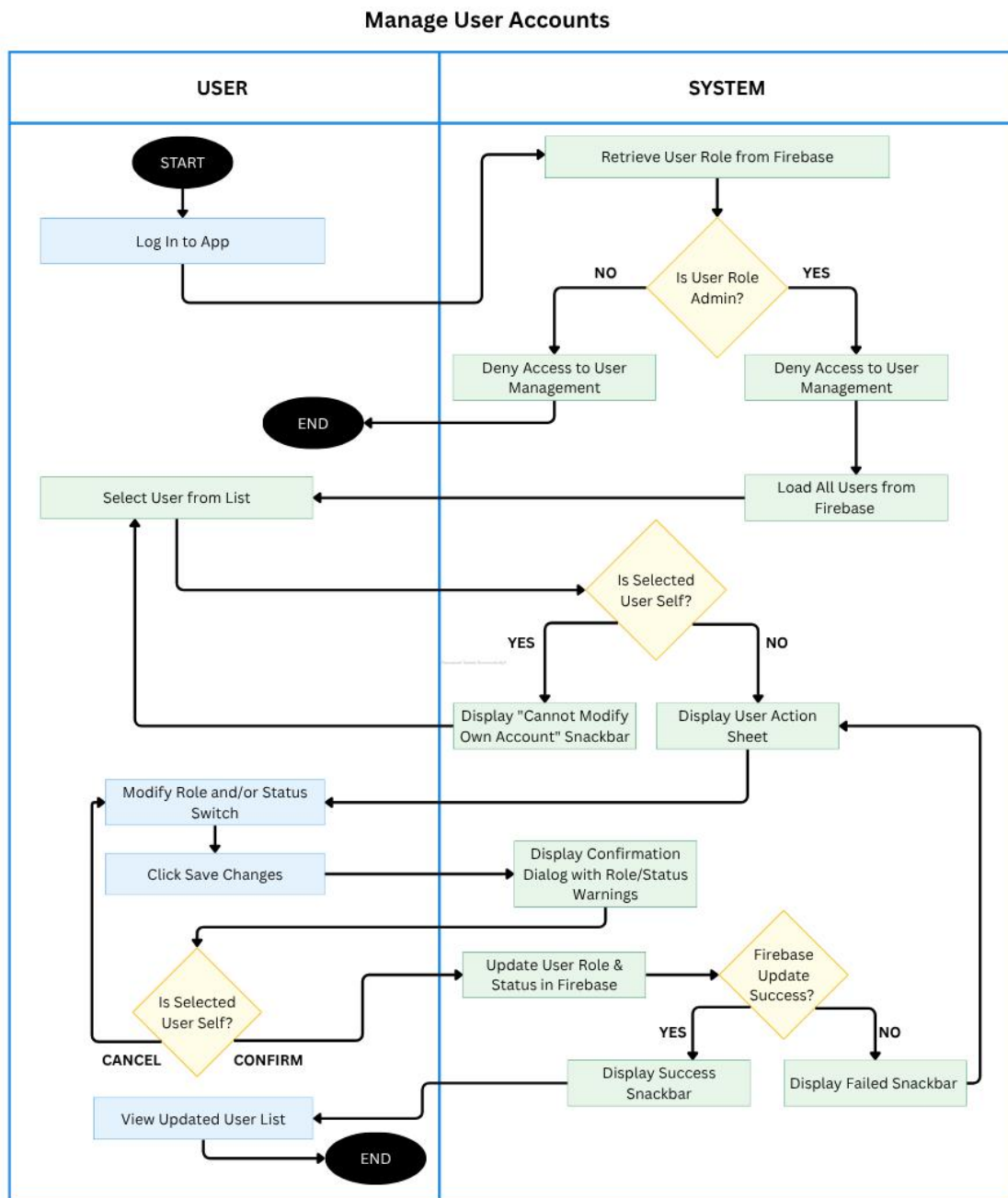


Figure 3.14. Activity Diagram for Manage User Accounts



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.14 illustrates the activity diagram for the Manage User Accounts use case in the CrayCare system. The process begins when the user logs into the app. The system retrieves the user role from Firebase and checks if the user is an Admin. Non-admin users are denied access to User Management. Admin users can load all users from Firebase, select a user from the list, and view the action sheet. The system prevents admins from modifying their own account.

The Admin can modify a user's role and/or status, then click Save Changes. The system displays a confirmation dialog and updates the user role and status in Firebase. Upon successful update, it shows a successful snackbar and refreshes the user list. If the update fails, a failed snackbar is displayed. The session ends when the Admin exits or cancels the operation.



### 3.2.2 GUI Design

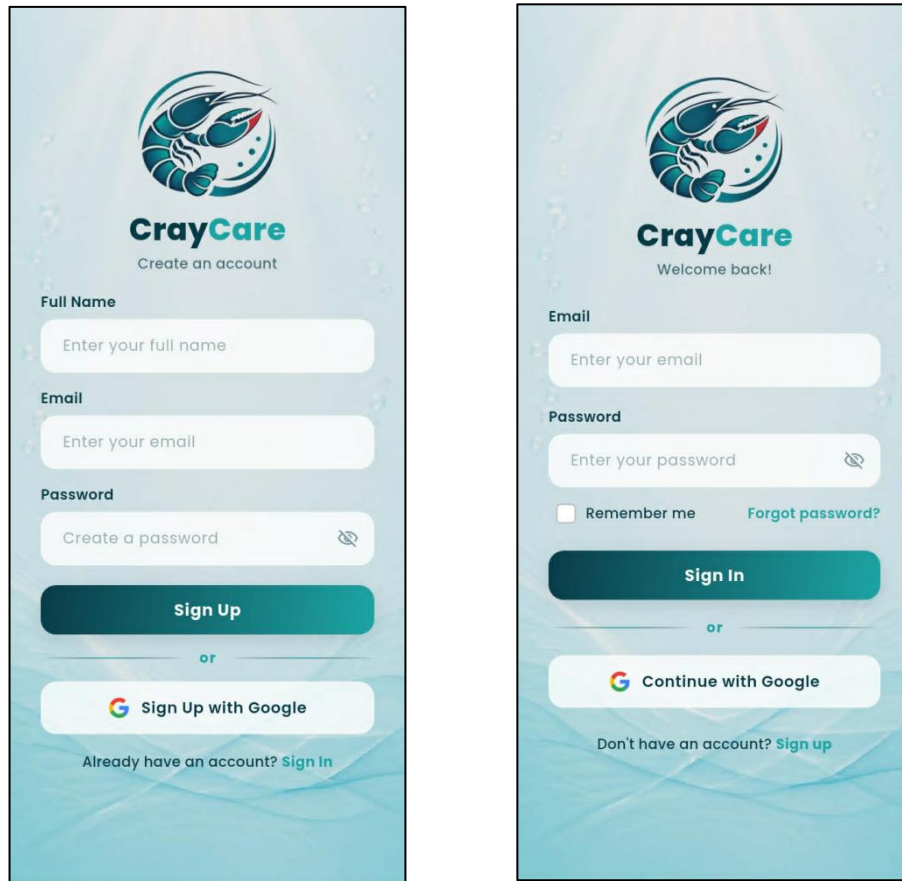


*3.15. Splash Screen*

Figure 3.15 presents the Splash Screen of the CrayCare mobile application, which serves as the initial interface displayed immediately upon launching. The screen features a centralized circular logo of a teal-colored crayfish, followed by the stylized "CrayCare" application name and the tagline "Better Care, Better Crayfish" to establish the system's focus on sustainable aquaculture. At the bottom, a "Loading..." text indicator accompanied by an animated teal horizontal progress bar provides users with visual feedback as background resources initialize. After a brief loading period of approximately three seconds, the screen automatically transitions to the login interface.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES



*Figure 3.16. User Authentication Screens*

Figures 3.16 present the Login and Sign Up screens of the CrayCare mobile application, which manage secure user access and account creation. The Login Screen (the image on the right side) features fields for email and password input, a "Remember me" checkbox, a "Forgot password?" link, and Google-integrated login, alongside a prompt directing new users to register. Upon navigating to the Sign Up Screen (the image on the left side), new users can register by inputting their full name, email, and password, or by choosing the quick



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

"Sign Up with Google" option. Both screens utilize a password visibility toggle icon within the input fields to assist users in typing and verifying their credentials.

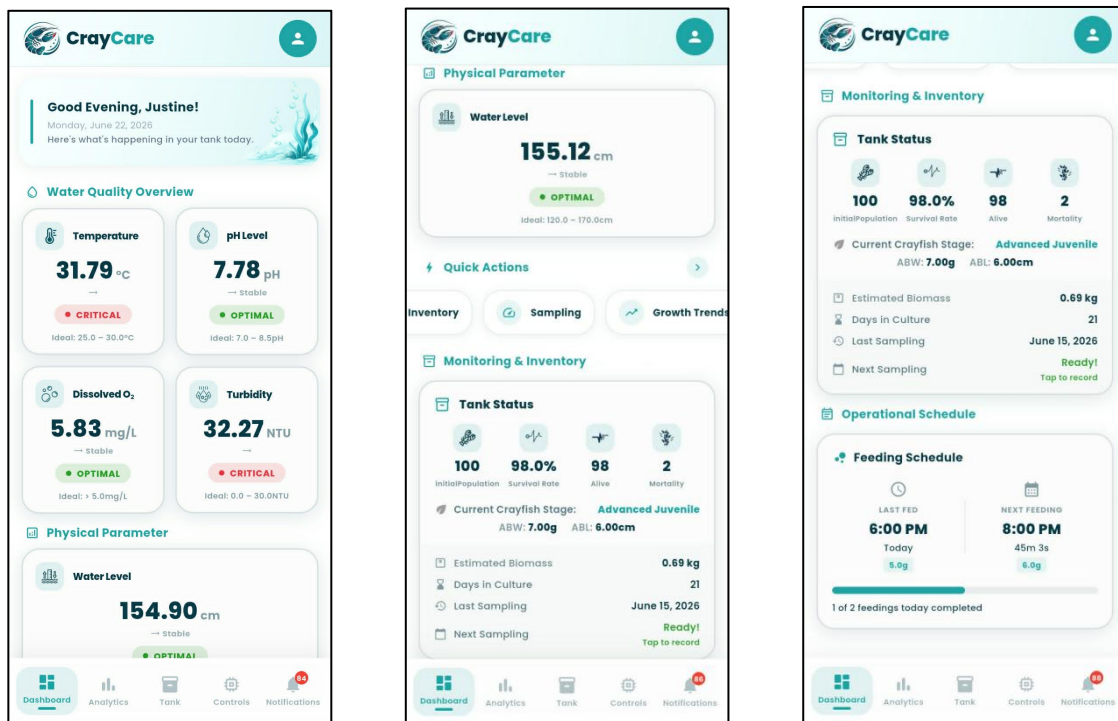


Figure 3.17. Dashboard Module Overview

Figures 3.17 display the continuous scroll of the Dashboard Module, which acts as the main hub for real-time monitoring and tank management. The upper section (the left-side image) displays a personalized welcome greeting alongside color-coded metric cards for vital water parameters—such as Temperature, pH, Dissolved Oxygen, Turbidity, and Water Level. The middle section (the middle image) introduces a Quick Actions navigation menu and the Tank Status panel, which tracks essential inventory statistics including live population, survival rates,



days in culture, and growth stages. Finally, the bottom section (the right-side image) shows the Operational Schedule panel, where users can track feeding timelines, remaining time until the next feeding, and the progress of completed feeds for the day.

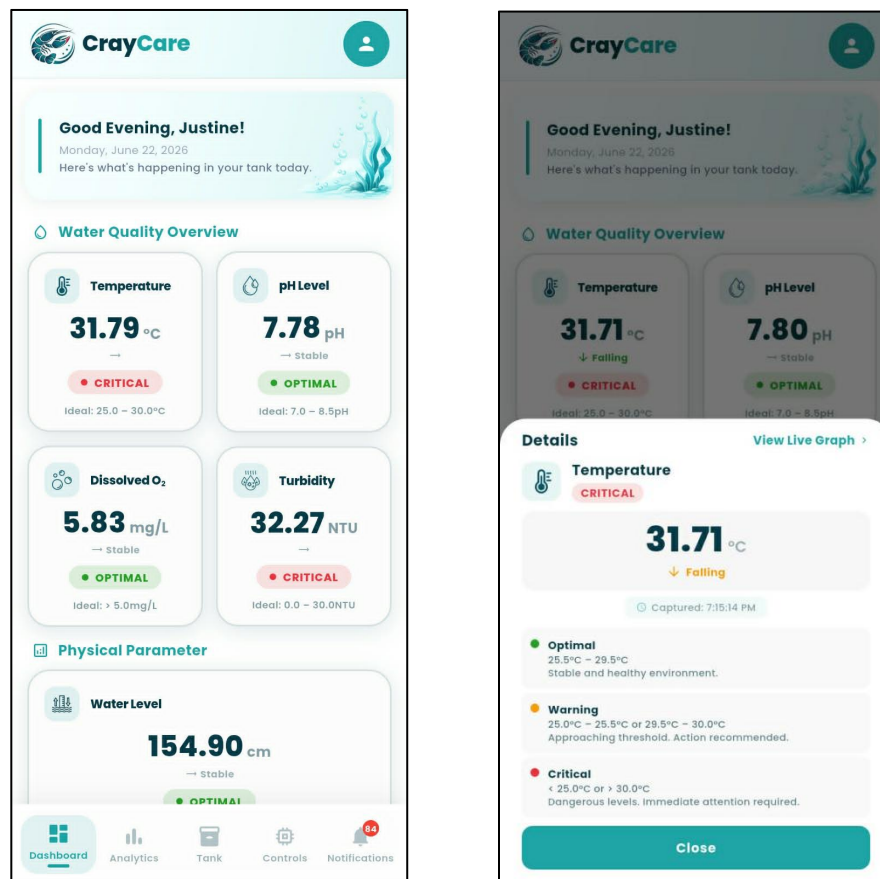


Figure 3.18. Parameter Detail Modal

Figure 3.18 presents the Parameter Detail Modal of the CrayCare mobile application, which is displayed when a user taps any of the water quality or physical parameter cards on the dashboard. Taking the Temperature gauge as an



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

example, the overlay modal presents the current sensor reading, trend direction (e.g., "Falling"), and a precise capture timestamp to give the user immediate status feedback. Below the main reading, a clear condition legend maps out the specific ranges and operational meanings for Optimal, Warning, and Critical thresholds, ensuring the user understands what action is required. Additionally, users can tap the "View Live Graph" shortcut in the top-right corner to navigate to historical analytics, or use the "Close" button at the bottom to return to the main dashboard.

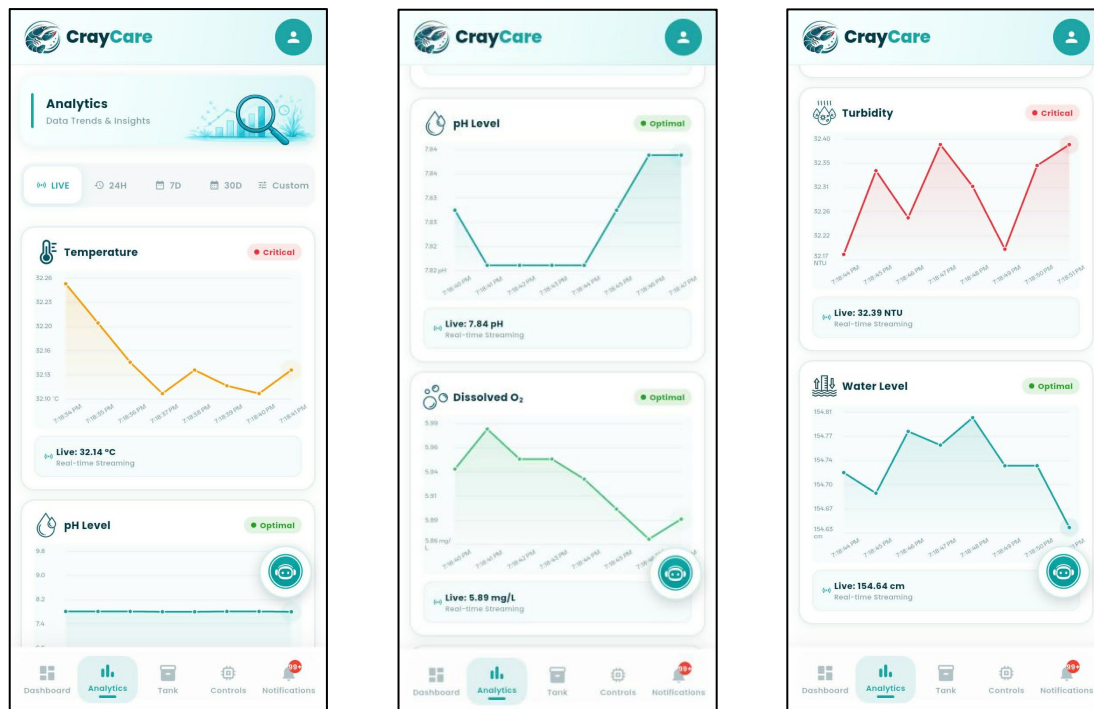
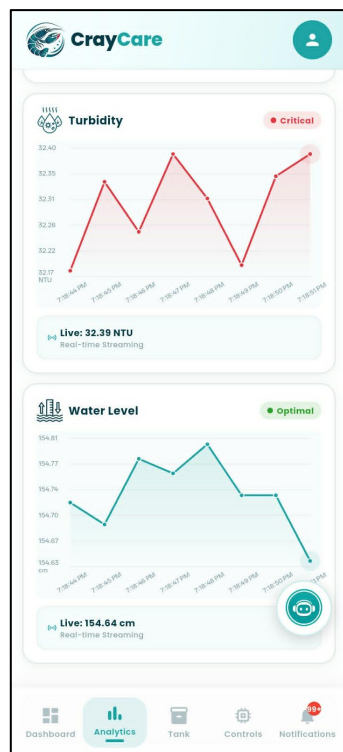


Figure 3.19. Analytics Module



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.19 presents the Analytics Module of the CrayCare mobile application, which allows users to monitor real-time data streaming and evaluate water parameter trends. Below a themed header banner, a horizontal row of filter chips—comprising Live, 24H, 7D, 30D, and Custom selections—enables users to quickly adjust the charting timeframe. The interface displays a vertical sequence of individual parameter cards for Temperature, pH Level, Turbidity, and Water Level, with each card featuring an interactive line chart, a color-coded threshold status badge, and a bottom banner showing the live streaming value. Additionally, a floating circular button with a robot icon is positioned on the screen, providing users with a draggable shortcut to access predictive machine learning insights and status recommendations.



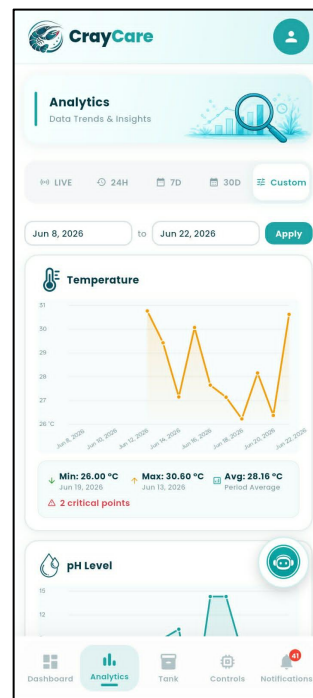
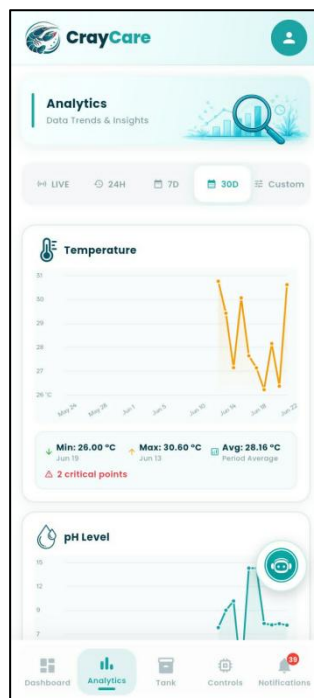
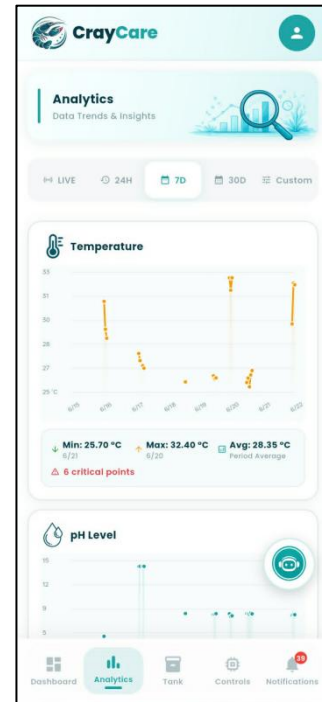


*Figure 3.20. Chart Detail Dialog*

Figure 3.20 presents the Chart Detail Dialog of the CrayCare mobile application, which is activated by tapping any of the individual sensor trend cards on the Analytics screen. Taking the Turbidity parameter as an example, the modal overlay provides an enlarged view of the sensor's timeline plot, complete with grid lines and a soft shaded gradient under the trend line. A real-time streaming status banner displays the latest value at the top, while interactive touch tooltips allow users to tap on data points to view the exact reading and corresponding timestamp. To return to the main Analytics interface, users can tap the close icon button located in the top-right corner of the dialog container.



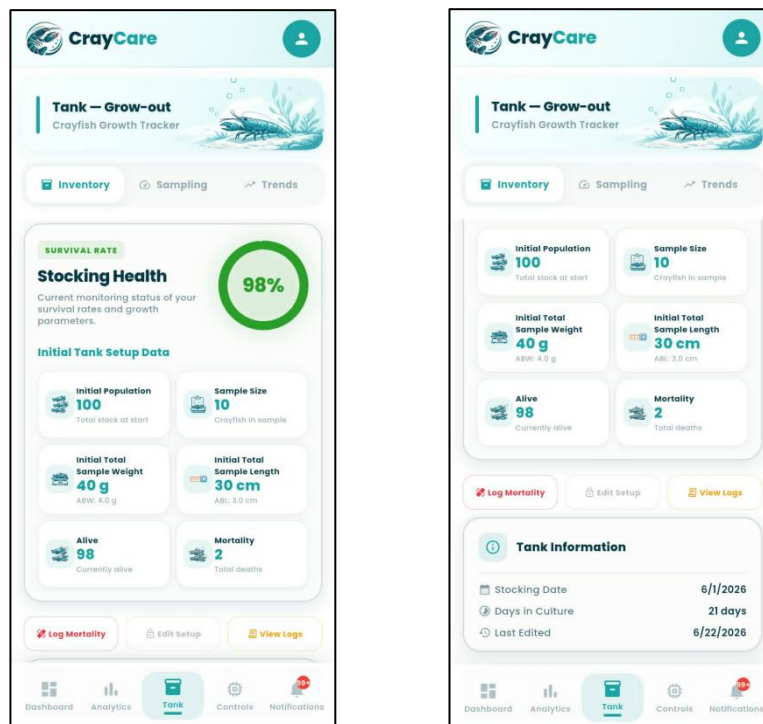
## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES





*Figure 3.21. Analytics Timeframe Filters*

Figure 3.21 presents the Analytics Timeframe Filters of the CrayCare mobile application, which allow users to dynamically adjust the time scope of the historical aquaculture data. Selecting the Live filter displays real-time streaming values and high-frequency updates, while switching to historical intervals like 24H, 7D, or 30D updates the charts to show summary metrics, including minimum, maximum, and average statistics, alongside a warning count of any critical out-of-range events. When the Custom filter chip is selected, the interface reveals two date input fields and an "Apply" button, enabling users to isolate and analyze a specific calendar range. Regardless of the active timeframe selected, the corresponding chart coordinates, data labels, and statistical averages update instantly to reflect the newly requested dataset.

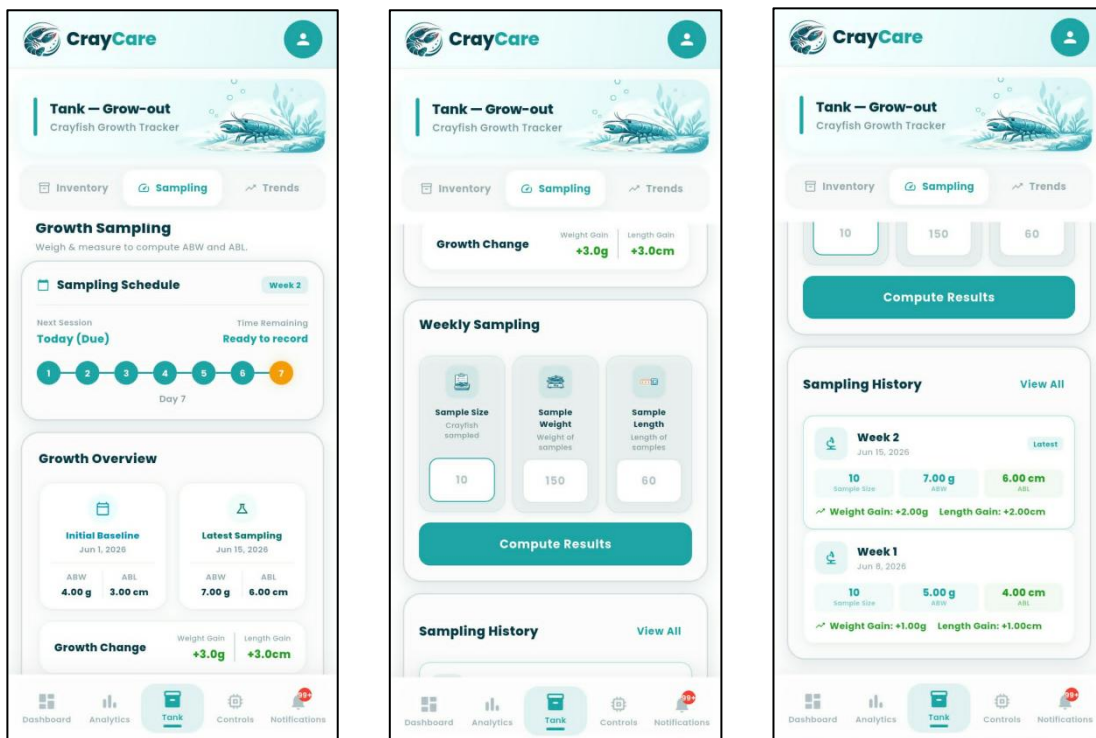




## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.22. Tank Module — Inventory Tab*

Figure 3.22 presents the Inventory Tab of the Tank Module in the CrayCare mobile application, which facilitates population tracking and grow-out monitoring. The upper section displays a circular donut chart indicating the current survival rate (98%) alongside a grid of six key metric cards: Initial Population, Sample Size, Initial Total Sample Weight (with computed ABW), Initial Total Sample Length (with computed ABL), and the calculated Alive and Mortality counts. Beneath the metric cards, three action buttons allow the farmer to record mortality events, edit the tank setup, or open the detailed activity logs. At the bottom, a Tank Information card aggregates timeline-critical data, including the initial stocking date, the total days in culture, and the timestamp of the last configuration edit.







*Figure 3.23. Tank Module — Sampling Tab*

Figure 3.23 presents the Sampling Tab of the Tank Module in the CrayCare mobile application, which allows the user to record, calculate, and log weekly crayfish growth metrics. The upper section features a Sampling Schedule card displaying a seven-day step-tracker timeline that indicates session readiness, paired with a Growth Overview card comparing the initial baseline values to the latest sampling results to show cumulative growth changes. Below this, a Weekly Sampling entry form provides input fields for Sample Size, Sample Weight, and Sample Length alongside a prominent "Compute Results" action button to process the current batch averages. At the bottom, a Sampling History panel displays a chronological list of past logs, showcasing metrics such as sample sizes, average weights and lengths, and individual weight and length gains per week.

### **Growth Monitoring through Weekly Sampling of Average Body Weight (ABW) and Average Body Length (ABL)**

Growth monitoring in crayfish is commonly done by measuring changes in body weight and body length. These measurements help determine the growth progress and development of crayfish during the culture period. In this study, weekly sampling was conducted to monitor the growth of *Cherax quadricarinatus* using Average Body Weight (ABW) and Average Body Length (ABL) as the main growth measurements.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The weekly sampling approach was based on the study of Fatihah and Chen (2022), which evaluated the growth performance of redclaw crayfish (*Cherax quadricarinatus*) in an aquaponics system. In their study, body weight and body length were measured regularly within a 28-day period. Measurements were taken on Day 0, Day 7, Day 14, and Day 28 to observe changes in crayfish growth.

Their findings showed that measuring body weight and body length regularly helped track the growth of crayfish throughout the experiment. After 28 days, the treatment group recorded a mean body weight of  $19.93 \pm 0.48$  g and a mean body length of  $11.13 \pm 0.60$  cm, which was higher compared to the control group with a mean body weight of  $17.08 \pm 0.53$  g and a mean body length of  $9.58 \pm 0.17$  cm. These results show that regular measurement of weight and length can be used to monitor growth changes and determine differences in crayfish development.

Based on this method, the present study adopted weekly sampling to observe changes in crayfish growth throughout the culture period. The Average Body Weight (ABW) is calculated by getting the average weight of the sampled crayfish, while the Average Body Length (ABL) is calculated by getting the average length of the sampled individuals. These measurements will serve as the basis for evaluating crayfish growth and determining if the culture conditions support their development.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

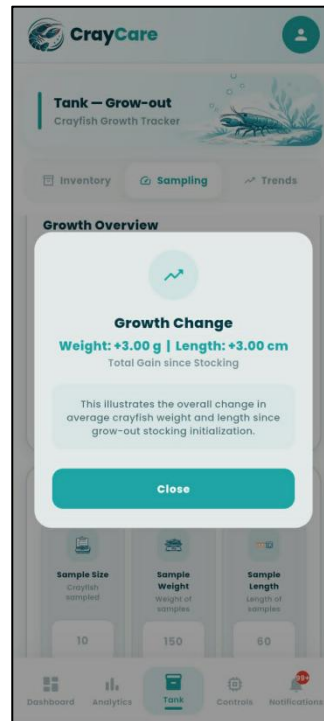
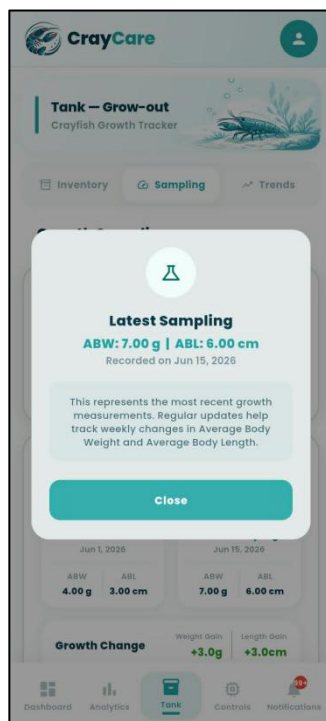
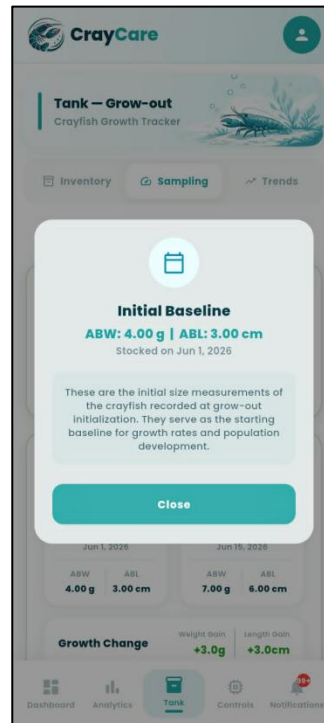
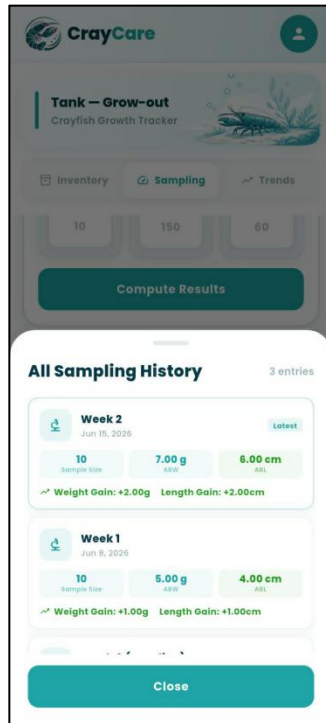
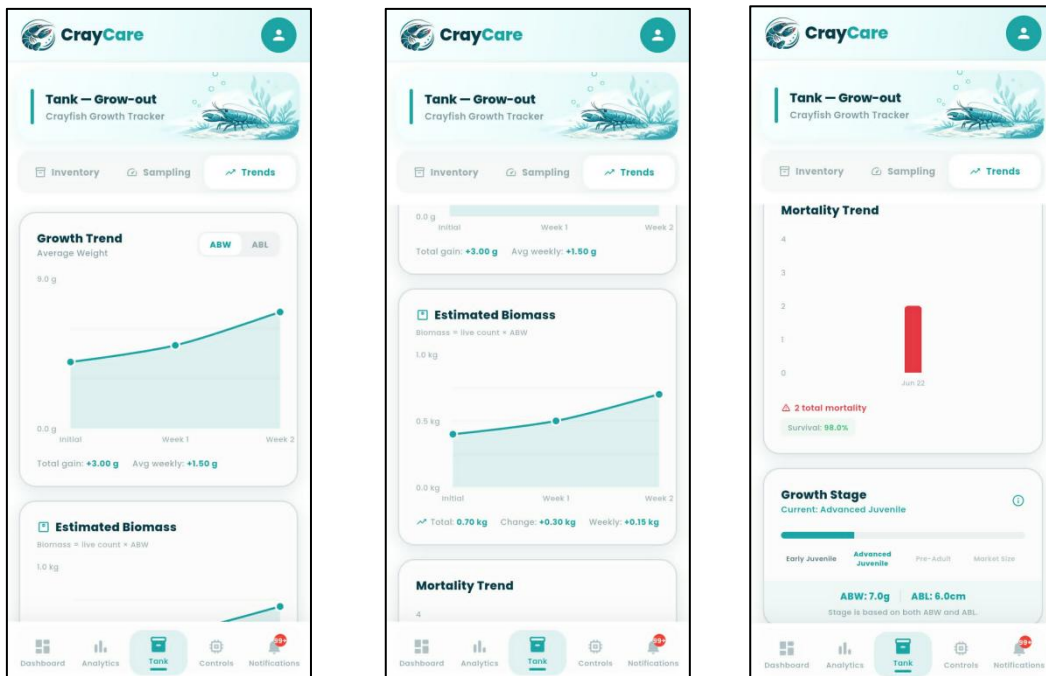




Figure 3.24. Sampling Tab Modal Overlays

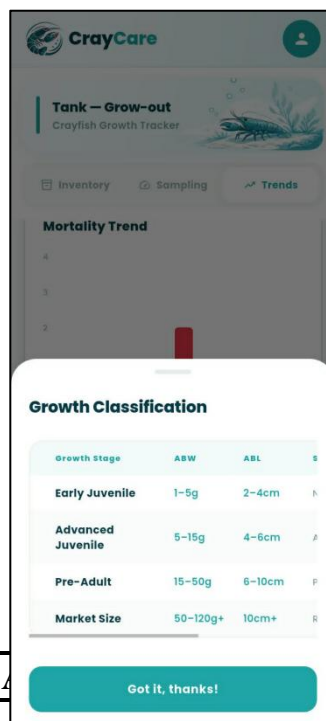
Figure 3.24 presents the detailed modal overlays and bottom sheets accessible from the Sampling Tab of the Tank Module. Tapping the "View All" link in the history section launches the "All Sampling History" bottom sheet, providing a dedicated scrollable list of all recorded weekly growth measurements and computed gains. Similarly, selecting any of the cards in the Growth Overview section opens specialized informational modals—specifically for Initial Baseline data, the Latest Sampling parameters, and the total cumulative Growth Change. Each pop-up contains calculated values, contextual explanations regarding starting sizes or cumulative development, and a prominent teal "Close" button to return the user to the main Sampling interface.





*Figure 3.25. Tank Module — Trends Tab*

Figure 3.25 presents the Trends Tab of the Tank Module in the CrayCare mobile application, which visualizes key grow-out performance indicators over time. The Growth Trend card features an interactive line graph with a toggle switch to alternate between Average Body Weight (ABW) and Average Body Length (ABL) plots, both detailed with total and average weekly growth summaries. Below this, an Estimated Biomass card plots total computed stock mass, while a Mortality Trend bar chart illustrates historical mortality occurrences alongside overall survival rates. At the bottom of the tab, a segmented Growth Stage progress bar categorizes and displays the crayfish's current developmental classification, such as "Advanced Juvenile," based on the latest physical parameters.





*Figure 3.26. Growth Classification Reference Modal*

Figure 3.26 presents the Growth Classification Reference Modal of the CrayCare mobile application, which can be accessed by tapping the information icon within the growth stage cards. The modal displays a structured reference table summarizing the classification thresholds for different stages of crayfish development based on Average Body Weight (ABW) and Average Body Length (ABL). The matrix defines four key stages—Early Juvenile, Advanced Juvenile, Pre-Adult, and Market Size—listing their corresponding weight ranges starting from 1g up to 120g+ and length brackets from 2cm to over 10cm. A prominent "Got it, thanks!" button at the bottom of the overlay allows the user to easily dismiss the reference sheet and return to the underlying tab interface.

To support the interpretation of crayfish growth during the experiment, the recorded body measurements will also be compared using the established crayfish size and weight classifications presented below. These classifications will serve as a reference for identifying the developmental stages observed throughout the grow-out period.

Table 3.4

**Crayfish Size and Weight Classification Reference**



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Growth Stage      | Average Length (cm) | Average Weight (g) | System Classification        |
|-------------------|---------------------|--------------------|------------------------------|
| Early Juvenile    | 2 – 4 cm            | 1 – 5 g            | Newly stocked young crayfish |
| Advanced Juvenile | 4 – 6 cm            | 5 – 15 g           | Active early growth          |
| Pre-Adult         | 6 – 10 cm           | 15 – 50 g          | Preparing for full maturity  |
| Market Size       | > 10 cm             | 50 g+              | Ready for harvest            |

The growth stage classification presented in Table 3.4 was developed based on existing references regarding the growth and stocking size of Red Claw crayfish (*Cherax quadricarinatus*). The ranges used in this study were not adopted as exact values from a single reference but were established by organizing available growth benchmarks into practical classifications suitable for continuous monitoring of crayfish development.

The Early Juvenile stage was defined based on the minimum stocking size reported by **Masser and Rouse (1997)**, which indicated that crayfish around 1 gram in weight or approximately 1–2 inches in length are suitable for transfer from the nursery phase to grow-out systems. Since the study requires a measurable range for identifying newly stocked crayfish, this reference point was expanded into the classification of 1–5 g and 2–4 cm, representing small juveniles that are still in the early growth phase.

The Advanced Juvenile stage was based on the stocking description provided by **Business Queensland (2025)**, which identifies advanced juveniles as crayfish commonly stocked at approximately 5–10 g. To accommodate natural variation in crayfish growth and provide a wider monitoring range, this study



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

classified Advanced Juveniles as crayfish weighing 5–15 g with a length of 4–6 cm. This stage represents crayfish that have moved beyond initial stocking size and are undergoing active growth.

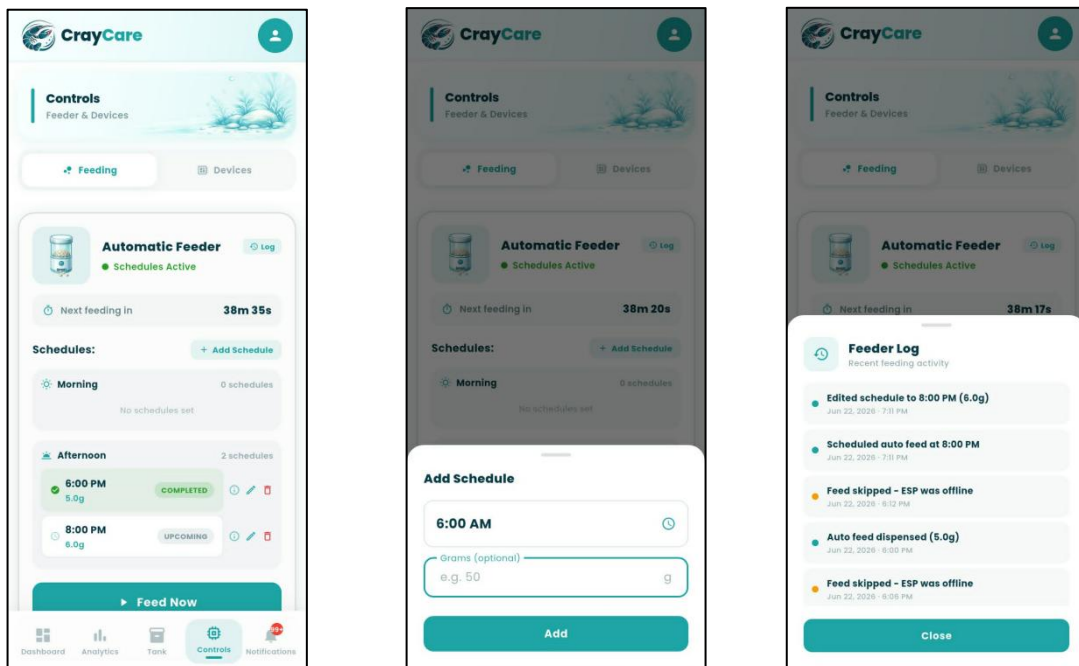
The Pre-Adult stage was established as an intermediate classification between juvenile development and harvestable size. Literature from the **Food and Agriculture Organization (2011)** and **Masser and Rouse (1997)** indicates that Red Claw crayfish continue rapid growth before reaching maturity, with maturity commonly associated with weights around 50 g and above. Based on this growth pattern, the study classified crayfish weighing 15–50 g and measuring 6–10 cm as Pre-Adults, representing individuals approaching mature size but not yet within the target harvest range.

The Market Size stage represents crayfish that have reached a size suitable for harvesting. **Business Queensland (2025)** reports that crayfish may reach approximately 35–100 g or more within the grow-out period, while **Masser and Rouse (1997)** reported that Red Claw crayfish can attain 50–100+ g under suitable growing conditions. Based on these reported growth values, this study classified crayfish weighing 50 g and above as Market Size, since this weight range indicates that crayfish have reached a more advanced stage of development and are within the expected harvestable size range. The length classification of greater than 10 cm was used as a corresponding physical indicator of this stage, representing crayfish that have developed beyond the juvenile growth phases and are suitable for harvesting evaluation.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Overall, the growth classifications used in this study were developed by interpreting available size and weight references and converting them into measurable ranges that can be applied during system monitoring. These categories allow the researchers to track changes in crayfish growth, identify developmental stages, and evaluate whether the culture conditions support expected growth progression.



*Figure 3.27. Controls Module — Feeding Tab*

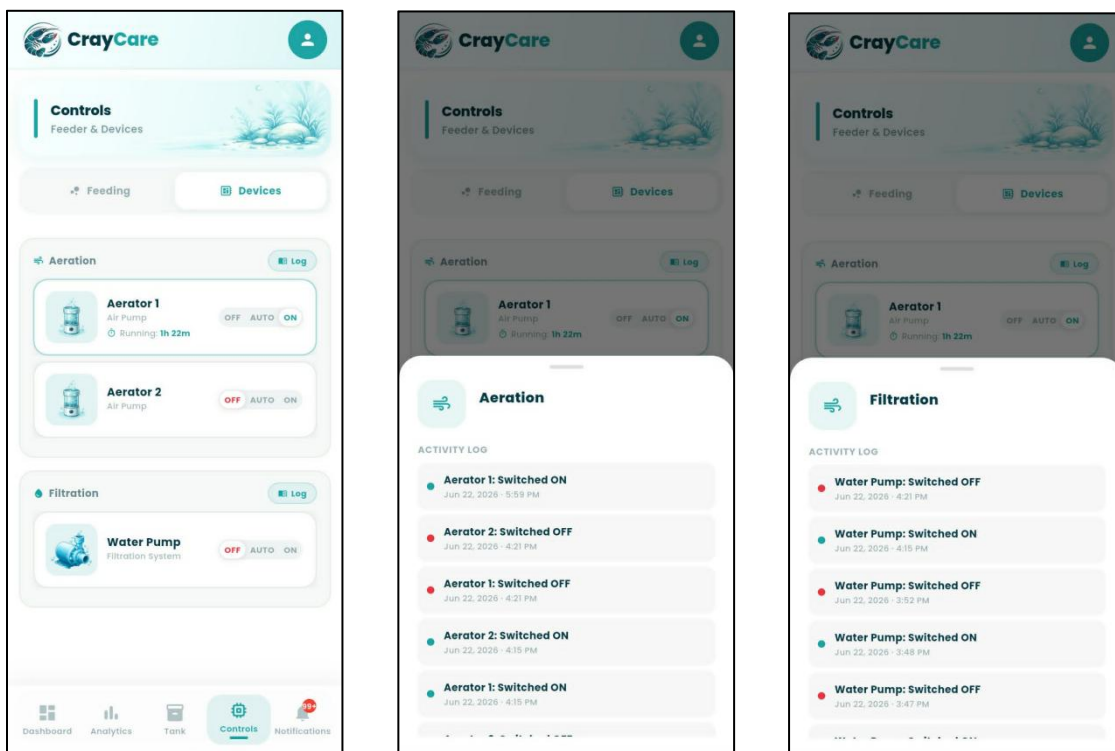
Figure 3.27 presents the Feeding Tab under the Controls Module of the CrayCare mobile application, which allows the user to manage and schedule automated feeding routines. The interface features a status card showing the remaining countdown until the next feed, a scrollable schedule list divided into





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Morning and Afternoon sessions with options to edit or delete entries, and a manual "Feed Now" action button. Tapping the "+ Add Schedule" button launches a bottom sheet overlay where users can select a precise feeding time and specify a desired portion weight in grams. Additionally, the "Feeder Log" overlay provides a chronological history of recent feeding activities, detailing successful automated dispenses alongside system skip logs caused by offline hardware warnings.



*Figure 3.28. Controls Module — Devices Tab*

Figure 3.28 presents the Devices Tab under the Controls Module of the CrayCare mobile application, which allows the user to supervise and configure auxiliary hardware systems. The interface organizes the hardware controls into



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Aeration and Filtration categories, housing dedicated cards for Aerator 1, Aerator 2, and the Water Pump. Each device card features a three-mode toggle selector—offering OFF, AUTO, and ON options—accompanied by real-time status details such as active running durations. Furthermore, tapping the respective "Log" button next to either category header launches a bottom sheet overlay displaying the chronological state-change history for those specific devices.

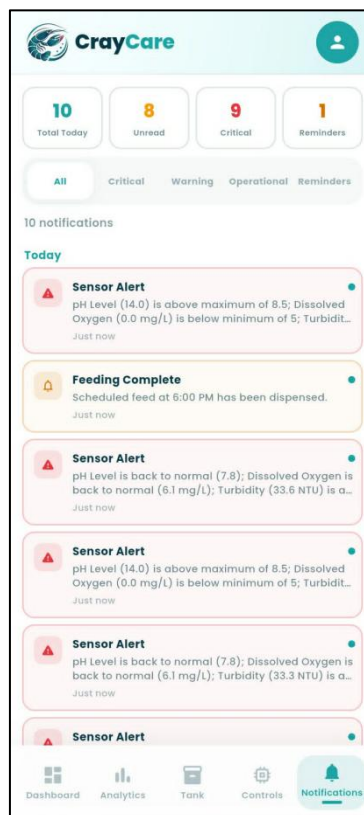


Figure 3.29. Notifications Module



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.29 presents the Notifications Module of the CrayCare mobile application, which serves as a centralized hub for real-time system alerts, reminders, and operational updates. At the top, a horizontal row of four key performance indicator (KPI) cards outlines critical metrics—such as Total Today, Unread, Critical, and Reminders—followed by interactive filter chips to sort messages by specific category types. The main area displays a scrollable, chronological list grouped by date categories, featuring color-coded notification cards that highlight specific events like critical sensor warnings and scheduled feeding completions. Each individual card includes a thematic type icon, descriptive preview text, a relative timestamp, and a small blue dot indicator to clearly distinguish unread entries from read ones.

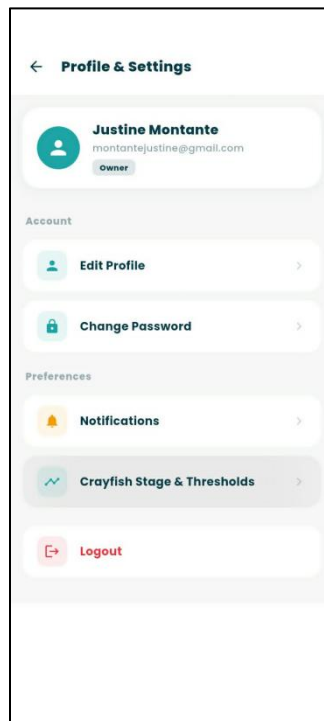
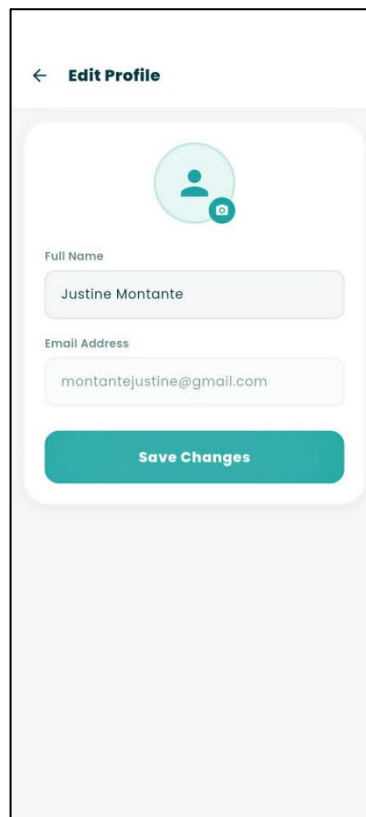


Figure 3.30. Profile & Settings Menu



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.30 presents the Profile & Settings menu of the CrayCare mobile application, which serves as the central configuration panel for user accounts. The top of the screen displays a card outlining the user's name, email, and active role badge, such as "Owner," confirming their specific system permissions. Below this card, configuration options are cleanly organized under Account and Preferences categories, offering navigational paths to update user details, change passwords, toggle notification alerts, manage crayfish growth stage thresholds, or log out of the active session.



*Figure 3.31. Edit Profile Sub-Screen*



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.31 presents the Edit Profile sub-screen, accessed by selecting the edit profile option from the main Settings menu. This interface features a circular profile photo preview containing an integrated camera icon button to trigger a device upload or photo capture. Beneath the photo uploader, the user can edit their display name via a clear text input field, while the registered email address is rendered in a locked, read-only state to protect database security before saving the changes.

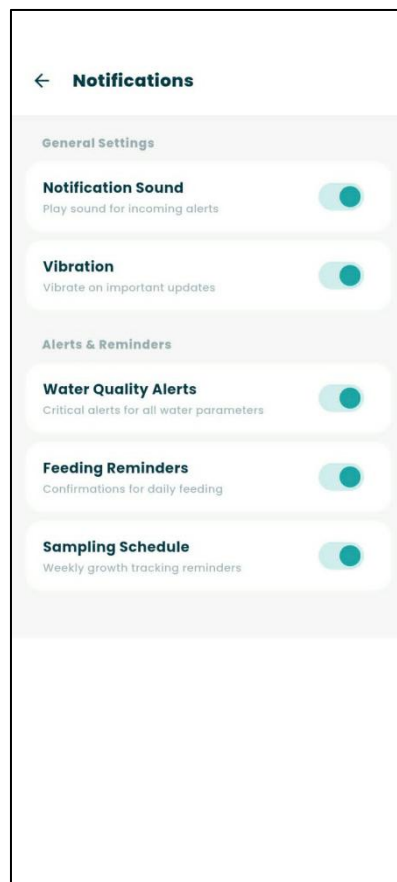
The image shows a mobile application screen titled 'Change Password'. At the top left is a back arrow icon. Below the title, a message states 'Your password must be at least 8 characters long.' There are three input fields: 'Current Password', 'New Password', and 'Confirm New Password'. Each field has a small eye icon on the right side to toggle visibility. At the bottom of the form is a teal button labeled 'Update Password'.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.32. Change Password Sub-Screen*

Figure 3.32 presents the Change Password sub-screen, which enables users to securely update their account credentials. At the top of the card container, a clear instruction notes that passwords must meet a minimum length requirement of eight characters. The interface contains three obscured password input fields—comprising Current Password, New Password, and Confirm New Password—with each input utilizing a show/hide eye icon to toggle visibility before committing updates with the "Update Password" button.

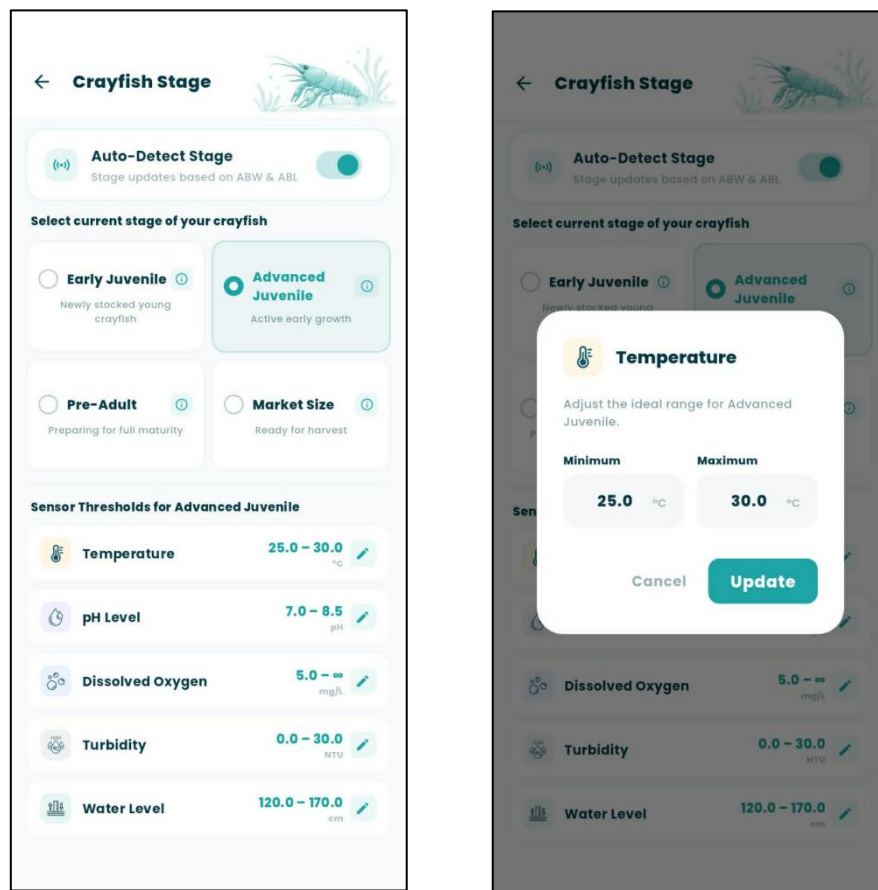




## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.33. Notifications Preferences Sub-Screen*

Figure 3.33 presents the Notifications Preferences sub-screen, allowing users to customize how they receive system alerts. The options are organized into General Settings, which control incoming alert sounds and device vibration feedback, and Alerts & Reminders, which cover specific functional notifications such as critical water quality parameters, daily feeding schedules, and weekly sampling reminders. Each option is detailed with a short descriptive label and an interactive teal toggle switch to allow independent configuration of each notification type.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.34. Crayfish Stage & Threshold Settings Interface*

Figure 3.34 presents the Crayfish Stage & Threshold Settings screen of the CrayCare mobile application, which allows the farm owner to manage targeted water parameter configurations based on the growth cycle of the crayfish. At the top, an "Auto-Detect Stage" toggle switch enables automatic stage transitions computed from average growth metrics, while manual cards below allow selection of the Early Juvenile, Advanced Juvenile, Pre-Adult, or Market Size stages. Below the selector cards, the interface lists the specific ideal safety ranges for the active stage across all five monitored parameters, with each card displaying its current thresholds alongside an edit pencil icon. Tapping the edit icon of any parameter displays an overlay modal where the user can adjust the ideal minimum and maximum tolerances for the active growth stage. This modal features numeric input fields pre-filled with the active bounds, accompanied by a "Cancel" option to discard adjustments and an "Update" button to save and apply the new thresholds.

To maintain suitable environmental conditions for crayfish growth and survival, specific water quality parameters must be monitored and maintained within acceptable ranges. The identified ranges for temperature, pH, dissolved oxygen, turbidity, and water level serve as reference values for evaluating the condition of the grow-out environment and determining possible changes that may affect crayfish development.

The following tables present the target ranges used as a basis for system monitoring and alert generation:





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Table 3.5

### Temperature Ranges

Table 3.6

### pH Ranges

| Growth Stage      | Optimal Range | Validating Sourced Basis                                       |
|-------------------|---------------|----------------------------------------------------------------|
| Early Juvenile    | 7.7 – 8.4     | Yong-chun et al. (2022)                                        |
| Advanced Juvenile | 7.0 – 8.5     | Queensland Government (2025)                                   |
| Pre-Adult         | 6.5 – 8.0     | Food and Agriculture Organization of the United Nations (2011) |
| Market Size       | 6.5 – 9.0     | Masser & Rouse (1997)                                          |

Table 3.7

### Dissolved Oxygen Ranges

| Growth Stage      | Optimal Range | Validating Sourced Basis                                                                     |
|-------------------|---------------|----------------------------------------------------------------------------------------------|
| Early Juvenile    | 6.0 – 8.5     | Huang et al. (2024); Masser & Rouse (1997)                                                   |
| Advanced Juvenile | 5.0 – 8.0     | Badryzlova et al. (2024); Jiang et al. (2025)                                                |
| Pre-Adult         | 5.0 – 7.5     | Queensland Government (2025); U.S. Fish and Wildlife Service (2020)                          |
| Market Size       | 5.0 – 7.5     | Queensland Government (2025); Food and Agriculture Organization of the United Nations (2011) |

Table 3.8

| Growth Stage      | Optimal Range | Validating Sourced Basis                                                                     |
|-------------------|---------------|----------------------------------------------------------------------------------------------|
| Early Juvenile    | 26°C – 30°C   | King (1994); Badryzlova et al. (2024)                                                        |
| Advanced Juvenile | 24°C – 30°C   | Stumpf & López Greco (2015); U.S. Fish and Wildlife Service. (2020)                          |
| Pre-Adult         | 23°C – 29°C   | Jones (1995)                                                                                 |
| Market Size       | 23°C – 30°C   | Queensland Government (2025); Food and Agriculture Organization of the United Nations (2011) |



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**Turbidity Ranges**

| <b>Growth Stage</b> | <b>Optimal Range</b> | <b>Validating Sourced Basis</b>                                                              |
|---------------------|----------------------|----------------------------------------------------------------------------------------------|
| Early Juvenile      | 10 NTU – 25 NTU      | Haubrock et al. (2021); AlpHa Measurement Solutions. (n.d.)                                  |
| Advanced Juvenile   | 30 NTU – 50 NTU      | Nur'aini & Junianto (2021); Parra et al. (2018)                                              |
| Pre-Adult           | 40 NTU – 80 NTU      | Food and Agriculture Organization of the United Nations (2011); Queensland Government (2025) |
| Market Size         | 40 NTU – 80 NTU      | Queensland Government (2025); Ruscoe (2002)                                                  |

Table 3.9

**Water Level or Water Depth Ranges**

| <b>Growth Stage</b> | <b>Optimal Range</b>              | <b>Validating Sourced Basis</b> |
|---------------------|-----------------------------------|---------------------------------|
| Early Juvenile      | 15 cm – 25 cm<br>(6 – 10 inches)  | Masser & Rouse (1997)           |
| Advanced Juvenile   | 25 cm – 30 cm<br>(10 – 12 inches) | The Fish Site (2012)            |
| Pre-Adult           | 30 cm – 40 cm<br>(12 – 16 inches) | Badryzlova et al. (2024)        |
| Market Size         | 30 cm – 40 cm<br>(12 – 16 inches) | Queensland Government (2025)    |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

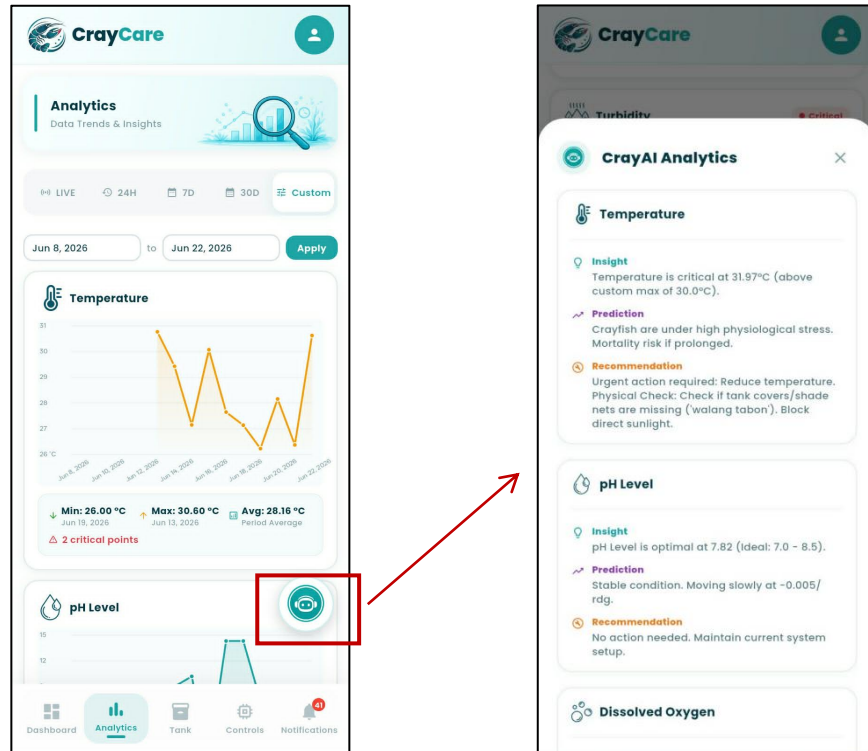


Figure 3.35. CrayAI Analytics ML Predictions

Figure 3.35 presents the CrayAI Analytics feature within the Analytics Module, which is accessed by tapping the floating robot assistant button on the

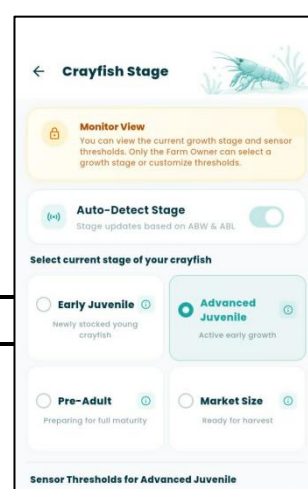
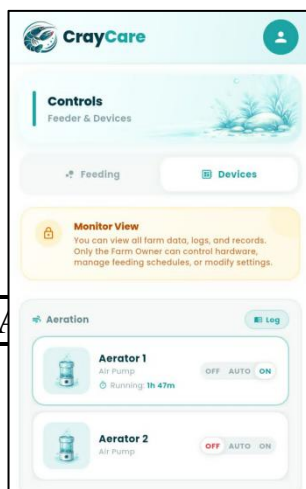
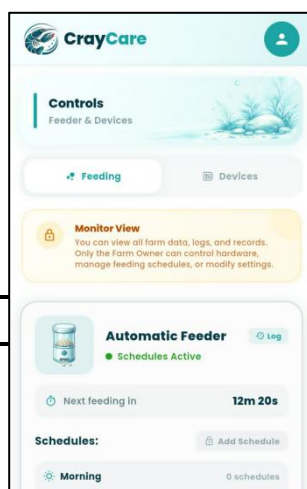
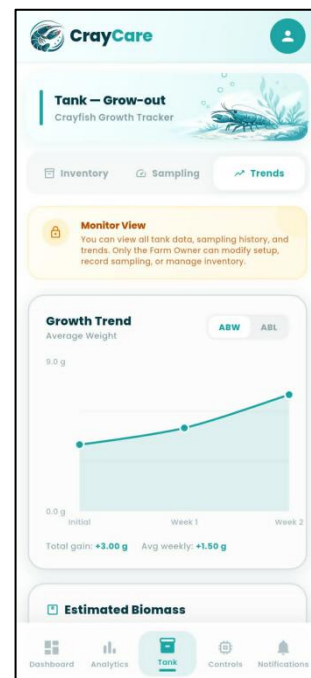
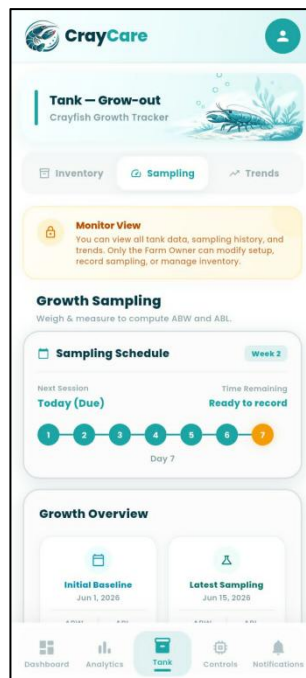
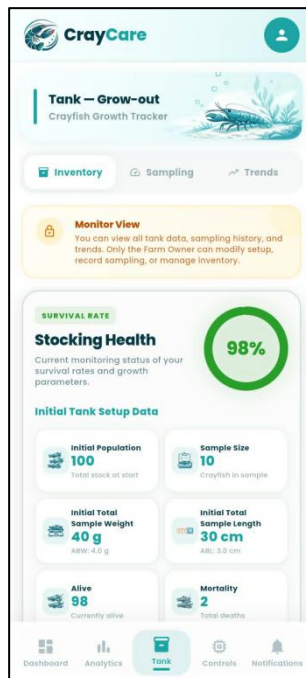


## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

bottom-right corner of the interface. Once tapped, a bottom sheet overlay labeled "CrayAI Analytics" appears, presenting detailed machine learning observations organized by all five monitored parameters: Temperature, pH Level, Dissolved Oxygen, Turbidity, and Water Level. Each parameter card is divided into three distinct segments: an Insight summarizing the current sensor reading relative to ideal limits, a physiological Prediction detailing the short-term biological impact on the crayfish, and a practical Recommendation offering actionable advice. For example, a critical temperature reading triggers a warning detailing elevated physiological stress and prospective mortality risks, paired with specific localized advice such as checking for missing tank covers, while optimal parameters indicate stable conditions and advise maintaining the current setup.



# POLYTECHNIC UNIVERSITY OF THE PHILIPPINES



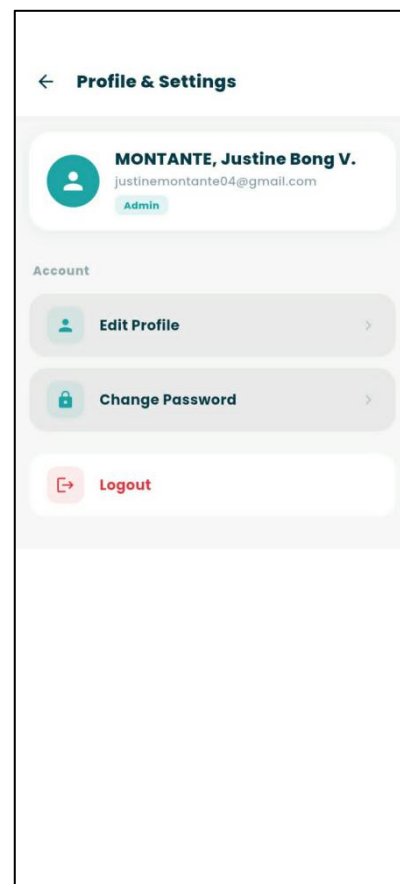
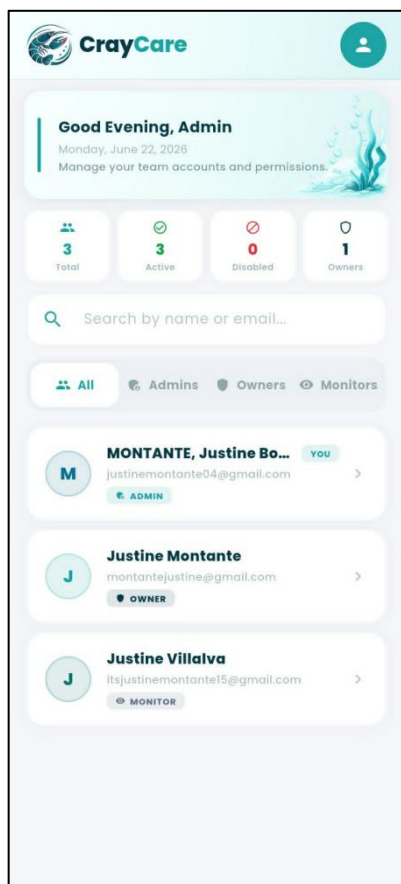


*Figure 3.36. Role-Based Monitor View Interface*

Figure 3.36 presents the customized user interface displayed when a user logs in with read-only "Monitor" role privileges. Across both the Tank and Controls Modules, a prominent yellow "Monitor View" warning banner with a lock icon is persistently displayed at the top of the viewport, informing the user that while they can view all live data, logs, and trends, only the Farm Owner can modify setups, record samplings, or manage hardware. Within the Tank Module (covering the Inventory, Sampling, and Trends tabs), active transaction components—such as the mortality logging and setup editing buttons—are hidden or restricted to protect database integrity. Similarly, in the Controls Module, the ability to add, edit, or delete feeding schedules is locked, the manual "Feed Now" button is removed, and the three-way toggle controls for aeration and water pump devices are rendered non-interactive, acting strictly as visual status indicators.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.37. Admin Module and Settings Interface*

Figure 3.37 presents the Admin Module and Settings interface of the CrayCare mobile application, which is tailored specifically for users assigned with administrative permissions. The User Management dashboard features a top header that welcomes the administrator, followed by a row of KPI summary cards tracking the total, active, and disabled accounts alongside owner counts. An integrated search bar and role-based filter chips (Admins, Owners, Monitors) allow the administrator to easily locate and manage team profiles, which are populated in a clear list displaying names, email addresses, and active role badges. When navigating to the Profile & Settings menu, an Admin is presented with a simplified, role-restricted layout focusing strictly on personal account details—including Edit Profile and Change Password options—while hardware configuration settings, stage thresholds, and general alert preferences are hidden from view.



### 3.2.3 Hardware Sketch and Setup

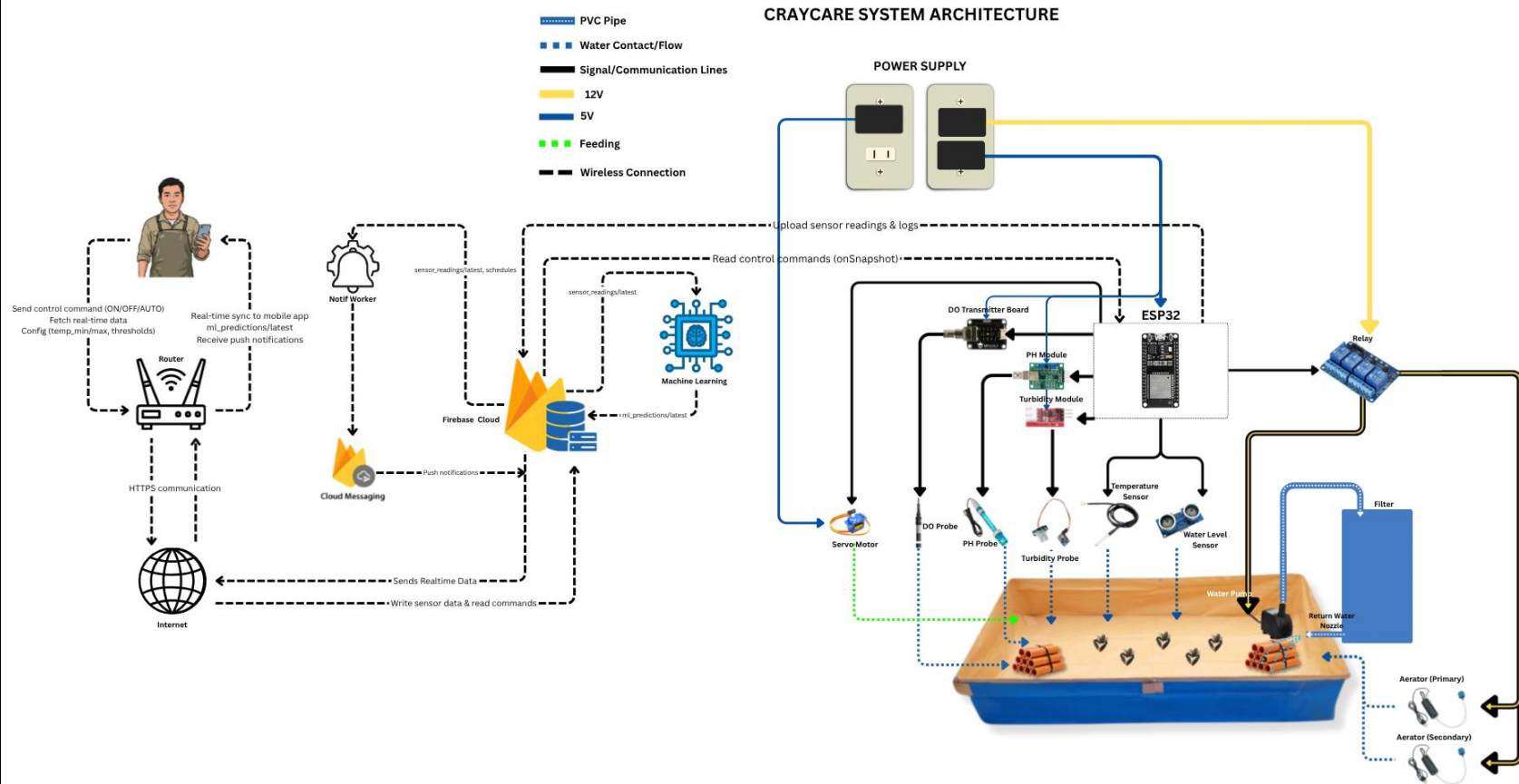


Figure 3.38. System Architecture



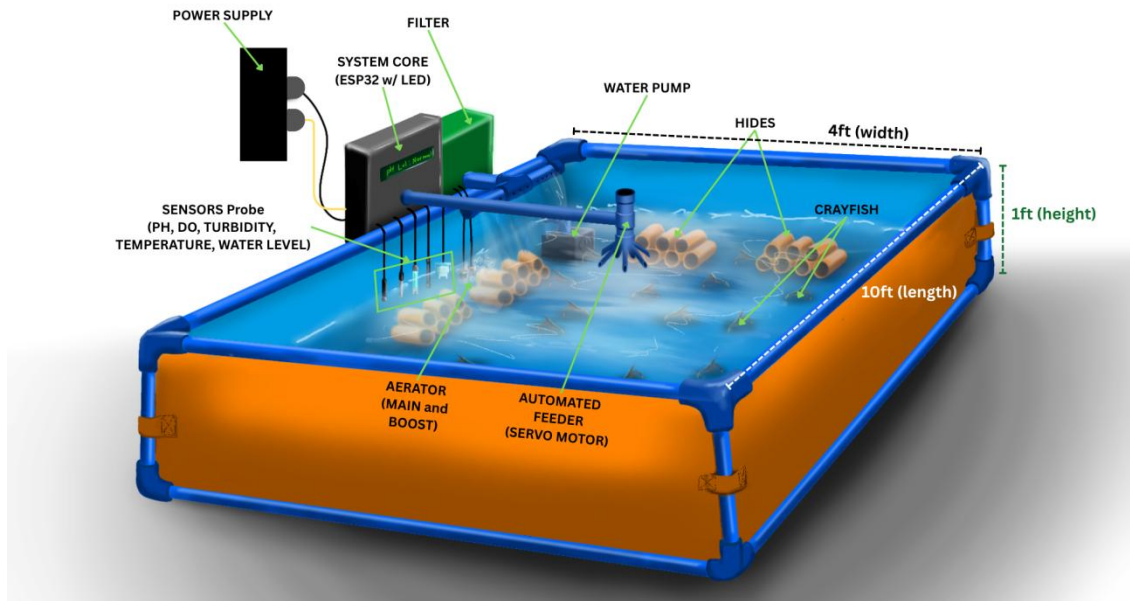
## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The CrayCare system is built around an ESP32 microcontroller connected to five sensors that measure temperature, pH, dissolved oxygen, turbidity, and water level. The ESP32 controls the aeration devices and water pump through a 4-channel relay module, while an automatic feeder uses a servo motor controlled directly by the ESP32. The system is powered through wall outlets using extension cords. A 5V charger powers the ESP32 via USB, while separate power supplies are used for the servo motor and 12V actuator components connected through the relay module.

Sensor readings are transmitted from the ESP32 to Firebase over Wi-Fi in JSON format. The mobile application reads and writes data to the same Firebase database, enabling real-time monitoring, device control, feeding schedules, notifications, sampling and mortality records, and growth tracking. Feeding recommendations are calculated using sampling data and stored in Firebase for display in the application.

A machine learning worker monitors sensor data from Firebase, generates features, performs model inference using trained Random Forest classifiers, and writes predictions, confidence scores, and insights back to Firebase. A separate notification worker sends push notifications through Firebase Cloud Messaging (FCM) when critical conditions or scheduled activities require user attention.

Tank B serves as the control group and does not utilize sensors, ESP32 automation, or IoT components. Its data is manually entered into the application for comparison with the automated setup.



*Figure 3.39. Hardware Setup Sketch*

Figure 3.39 presents the hardware setup sketch of the proposed CrayCare system. The sketch illustrates the planned arrangement and placement of the major hardware components within the crayfish grow-out tank. It serves as a guide for the physical implementation of the system, showing the integration of monitoring devices, water management components, and automated control mechanisms. The proposed layout is designed to support proper component placement and efficient operation of the system throughout the experimental setup.



### 3.2.4 Database Schema

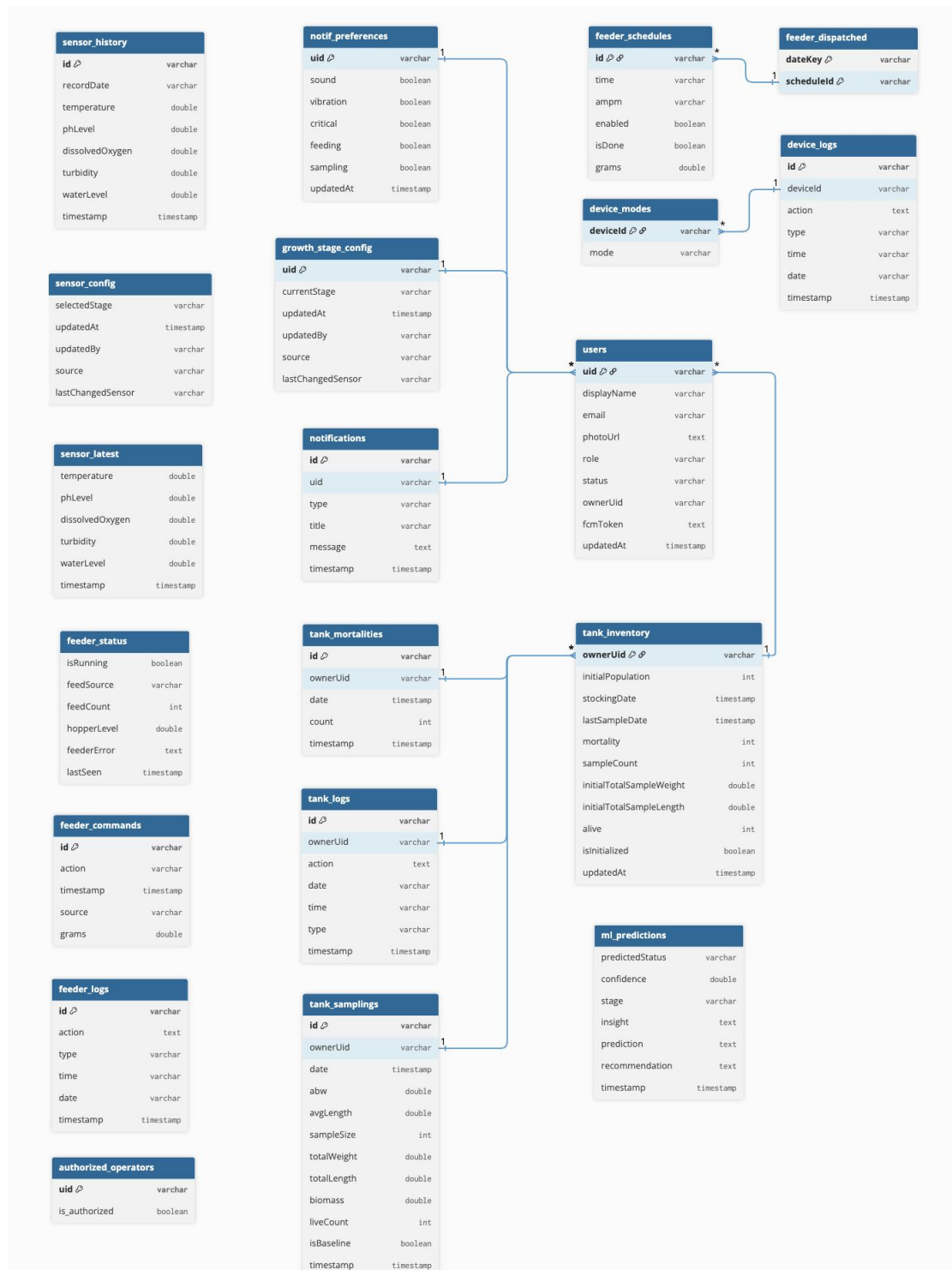


Figure 3.40. Database Schema of CrayCare



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Figure 3.40 presents the database schema of the proposed CrayCare system. The schema shows the structure and relationships of the data entities required for the operation of the system, including user information, sensor readings, tank records, feeding activities, notifications, and crayfish growth monitoring records.

The database design enables the organized storage of real-time monitoring data, historical information, system settings, and user-related records. It establishes connections among the different system components, allowing data gathered from sensors, automated processes, and the mobile application to be properly stored and accessed.

The schema serves as a guide for managing data organization and relationships within the system. It supports efficient data retrieval, record management, and analysis of information collected throughout the crayfish grow-out monitoring process.



### 3.2.5 Data Dictionary

This section provides the authoritative field-level reference for the current CrayCare Cloud Firestore database. Firestore data is organized as collections, documents, and subcollections. Each table identifies the canonical path, field name, data type, constraints or default behavior, and operational purpose used by the Flutter application, Cloud Functions, and assigned ESP32 hardware.

Data types follow Cloud Firestore conventions: string, number, boolean, timestamp, array, map, and null. Unless a field explicitly uses epoch milliseconds for ESP32 compatibility, date-time values are stored as Firestore Timestamp values or documented ISO-8601 strings. Canonical application fields use snake\_case except where firmware compatibility requires an established camelCase field.

#### 1. Users and Access Management

##### 1.1 users/{uid}

Stores the authenticated user's profile, access role, account status, tank link, and registered notification tokens. The document ID is the Firebase Authentication UID.

Table 3.10

Data Dictionary of Collection: users/{uid}

| Field Name | Data Type     | Constraints / Default        | Description                                                                                            |
|------------|---------------|------------------------------|--------------------------------------------------------------------------------------------------------|
| full_name  | string        | Required                     | User's complete display name.                                                                          |
| email      | string        | Required; unique             | Email address used for authentication and account recovery.                                            |
| role       | string        | owner   admin                | Access role. Only one active administrator account is intended; owners operate assigned tank features. |
| status     | string        | active   disabled            | Controls whether the account may use protected app features.                                           |
| tank_id    | string        | Owner: required; Admin: null | Tank owned by the user; normally equal to the owner's UID.                                             |
| photo_url  | string        | Optional                     | Canonical profile-image URL. photoUrl may remain only as a legacy alias.                               |
| fcmTokens  | array<string> | Default: empty               | Notification token for each signed-in device.                                                          |
| fcmToken   | string        | Legacy; optional             | Single-token compatibility field for older app versions.                                               |
| created_at | timestamp     | Server timestamp             | Time the profile document was created.                                                                 |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

### 1.2 users/{uid}/notification\_settings/preferences

Stores the user's notification-channel and notification-category preferences.

Table 3.11

Data Dictionary of Collection: users/{uid}/notification\_settings/preferences

| Field Name  | Data Type | Constraints / Default | Description                                                      |
|-------------|-----------|-----------------------|------------------------------------------------------------------|
| sound       | boolean   | Default: true         | Enables notification sound where permitted by the device.        |
| vibration   | boolean   | Default: true         | Enables vibration where permitted by the device.                 |
| critical    | boolean   | Default: true         | Enables critical water-quality notifications.                    |
| warning     | boolean   | Default: true         | Enables warning-level water-quality notifications.               |
| feeding     | boolean   | Default: true         | Enables feeding reminders and completed/skipped feeding results. |
| sampling    | boolean   | Default: true         | Enables tank sampling reminders.                                 |
| operational | boolean   | Default: true         | Enables actuator, device, and sensor-recovery updates.           |
| updated_at  | timestamp | Server timestamp      | Time the preferences were last updated.                          |

### 1.3 users/{uid}/notif\_markers/{key}

Stores idempotency markers used by scheduled checks so that the same reminder is not created repeatedly.

Table 3.12

Data Dictionary of Collection: users/{uid}/notif\_markers/{key}

| Field Name | Data Type                 | Constraints / Default | Description                                            |
|------------|---------------------------|-----------------------|--------------------------------------------------------|
| markerKey  | string                    | Required              | Stable identifier for the reminder or scheduled check. |
| value      | boolean   string   number | Required              | Marker state or last processed value.                  |
| updated_at | timestamp                 | Server timestamp      | Time the marker was last changed.                      |

## 2. Hardware Assignment

### 2.1 hardware\_system/currentOwner

Single system document that binds the physical CrayCare hardware to one active owner and that owner's tank.

Table 3.13

Data Dictionary of Collection: hardware\_system/currentOwner

| Field Name | Data Type | Constraints / Default | Description |
|------------|-----------|-----------------------|-------------|
|------------|-----------|-----------------------|-------------|



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name  | Data Type     | Constraints / Default     | Description                                                           |
|-------------|---------------|---------------------------|-----------------------------------------------------------------------|
| uid         | string   null | Active owner UID or null  | UID of the currently assigned owner.                                  |
| tank_id     | string   null | Must match user's tank_id | Tank receiving data and device operations from the assigned hardware. |
| assigned_by | string        | Admin UID                 | Administrator who made the assignment.                                |
| assigned_at | timestamp     | Server timestamp          | Time the assignment became effective.                                 |

Note: A valid unassigned state uses both uid and tank\_id as null. Disabling the assigned owner clears this document atomically.

### 3. Tanks and Sensor Data

#### 3.1 tanks/{tankId}

Core tank document containing ownership and current grow-out-cycle summary information.

Table 3.14

Data Dictionary of Collection: tanks/{tankId}

| Field Name                  | Data Type     | Constraints / Default | Description                                                      |
|-----------------------------|---------------|-----------------------|------------------------------------------------------------------|
| owner_uid                   | string        | Required              | UID of the owner responsible for the tank.                       |
| current_batch_id            | string   null | Optional              | Identifier of the active production batch.                       |
| stocking_date               | timestamp     | Optional              | Date and time the current grow-out cycle was stocked.            |
| last_sample_date            | timestamp     | Optional              | Date of the latest recorded sample.                              |
| sample_count                | number        | Default: 0            | Number of samples recorded for the current cycle.                |
| initial_population          | number        | Non-negative integer  | Counted population stocked at the beginning of the cycle.        |
| initial_total_sample_weight | number        | Optional; grams       | Total weight of the baseline sample.                             |
| initial_total_sample_length | number        | Optional; centimeters | Total length of the baseline sample.                             |
| is_initialized              | boolean       | Default: false        | Whether tank and baseline production data have been initialized. |
| created_at                  | timestamp     | Server timestamp      | Time the tank document was created.                              |





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

### 3.2 tanks/{tankId}/sensor\_readings/latest

Latest routed sensor snapshot used by the live dashboard. It is updated from the assigned ESP32's staging document.

Table 3.15

Data Dictionary of Collection: tanks/{tankId}/sensor\_readings/latest

| Field Name           | Data Type | Constraints / Default | Description                                                                                          |
|----------------------|-----------|-----------------------|------------------------------------------------------------------------------------------------------|
| temperature          | number    | degrees Celsius       | Current water temperature.                                                                           |
| ph_level             | number    | pH                    | Current acidity or alkalinity reading.                                                               |
| dissolved_oxygen     | number    | mg/L                  | Current dissolved oxygen concentration.                                                              |
| turbidity            | number    | NTU                   | Current water turbidity.                                                                             |
| turbidity_air        | number    | Optional              | Air/reference reading used by the turbidity integration.                                             |
| water_level          | number    | centimeters           | Current pond water level.                                                                            |
| feed_level           | number    | 0-100 percent         | Estimated percentage of feed remaining in the hopper; operational only and excluded from WQA inputs. |
| estimated_feed_grams | number    | Optional; grams       | Estimated remaining feed derived from feed level and calibrated hopper capacity.                     |
| buffered_entries     | number    | Optional              | Number of locally buffered readings reported by the ESP32.                                           |
| recorded_at          | timestamp | Required              | Original sensor capture time preserved during routing.                                               |

### 3.3 tanks/{tankId}/sensor\_readings\_history/{YYYY-MM-DD}

Daily summary document used for efficient long-range analytics. Only complete and sanitized summaries are consumed by the optimized reader.

Table 3.16

Data Dictionary of Collection: tanks/{tankId}/sensor\_readings\_history/{YYYY-MM-DD}

| Field Name          | Data Type     | Constraints / Default | Description                                                          |
|---------------------|---------------|-----------------------|----------------------------------------------------------------------|
| date_key            | string        | YYYY-MM-DD            | Calendar key represented by this daily document.                     |
| summary_version     | number        | Current: 1            | Schema/version number of the summary computation.                    |
| summary_sanitized   | boolean       | Required              | Confirms invalid sentinel and non-finite values have been removed.   |
| summary_complete    | boolean       | Required              | Indicates whether the daily summary is ready for optimized reads.    |
| sample_count        | number        | Default: 0            | Number of valid history entries included.                            |
| processed_entry_ids | array<string> | Default: empty        | Entry IDs already included, used for idempotent summary maintenance. |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name                       | Data Type     | Constraints / Default | Description                                 |
|----------------------------------|---------------|-----------------------|---------------------------------------------|
| temp_min/max/avg/sum/count       | number fields | When data exists      | Daily temperature statistics.               |
| pH_min/max/avg/sum/count         | number fields | When data exists      | Daily pH statistics.                        |
| DO_min/max/avg/sum/count         | number fields | When data exists      | Daily dissolved-oxygen statistics.          |
| turbidity_min/max/avg/sum/count  | number fields | When data exists      | Daily turbidity statistics.                 |
| waterLevel_min/max/avg/sum/count | number fields | When data exists      | Daily water-level statistics.               |
| updated_at                       | timestamp     | Server timestamp      | Time the daily summary was last maintained. |

### 3.4 tanks/{tankId}/sensor\_readings\_history/{YYYY-MM-DD}/entries/{docId}

Ten-minute MIN/MAX/AVG sensor aggregate. Invalid sensors are omitted instead of being stored as negative sentinel values.

Table 3.17

#### Data Dictionary of Collection: tanks/{tankId}/sensor\_readings\_history/{YYYY-MM-DD}/entries/{docId}

| Field Name                   | Data Type | Constraints / Default | Description                                                          |
|------------------------------|-----------|-----------------------|----------------------------------------------------------------------|
| temp_min, temp_max, temp_avg | number    | degrees Celsius       | Temperature statistics for the ten-minute window.                    |
| pH_min, pH_max, pH_avg       | number    | pH                    | pH statistics for the ten-minute window.                             |
| DO_min, DO_max, DO_avg       | number    | mg/L                  | Dissolved-oxygen statistics for the window.                          |
| turbidity_min/max/avg        | number    | NTU                   | Turbidity statistics for the window.                                 |
| waterLevel_min/max/avg       | number    | centimeters           | Water-level statistics for the window.                               |
| feed_level                   | number    | Optional; percent     | Operational hopper-level snapshot; excluded from WQA model features. |
| estimated_feed_grams         | number    | Optional; grams       | Estimated feed remaining during the aggregate window.                |
| recorded_at                  | timestamp | Required              | Original capture time of the completed ten-minute window.            |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

### 3.5 tanks/{tankId}/sensors/{sensorName}

Per-tank threshold document for temperature, ph\_level, dissolved\_oxygen, turbidity, water\_level, or feed\_level.

Table 3.18

Data Dictionary of Collection: tanks/{tankId}/sensors/{sensorName}

| Field Name            | Data Type | Constraints / Default | Description                                                                             |
|-----------------------|-----------|-----------------------|-----------------------------------------------------------------------------------------|
| min_value             | number    | Required              | Lower boundary of the configured normal/ideal range.                                    |
| max_value             | number    | Required              | Upper boundary of the configured normal/ideal range.                                    |
| critical_value        | number    | feed_level only       | Percentage below which the feed level is critical. This alone does not prevent feeding. |
| hopper_capacity_grams | number    | feed_level only       | Calibrated feed mass when the hopper is full; used to estimate remaining grams.         |
| updated_at            | timestamp | Server timestamp      | Time the threshold configuration was last updated.                                      |

Note: The feed level statuses are Normal (>20%), Low (11-20%), Critical (1-10%), and Empty (0%). Feeding is skipped only when feed is empty, unavailable, or estimated grams are below the requested amount.

## 4. Actuators

### 4.1 tanks/{tankId}/actuators/{deviceId}

Stores requested mode and reported state for pump, aerator1, or aerator2.

Table 3.19

Data Dictionary of Collection: tanks/{tankId}/actuators/{deviceId}

| Field Name    | Data Type | Constraints / Default         | Description                                 |
|---------------|-----------|-------------------------------|---------------------------------------------|
| control_mode  | string    | on   off   auto               | Operating mode requested by the owner.      |
| current_state | string    | on   off                      | Actual relay state reported by the ESP32.   |
| last_changed  | number    | Epoch milliseconds; 0 = never | Time the device state or mode last changed. |

### 4.2 tanks/{tankId}/actuator\_logs/{logId}

Audit record for actuator operations.

Table 3.20

Data Dictionary of Collection: tanks/{tankId}/actuator\_logs/{logId}

| Field Name    | Data Type | Constraints / Default      | Description                                  |
|---------------|-----------|----------------------------|----------------------------------------------|
| actuator_type | string    | pump   aerator1   aerator2 | Device associated with the event.            |
| action        | string    | Required                   | Human-readable description of the operation. |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name | Data Type          | Constraints / Default | Description                                                          |
|------------|--------------------|-----------------------|----------------------------------------------------------------------|
| type       | string             | on   off   auto       | Machine-readable operation type.                                     |
| logged_at  | number   timestamp | New writes: epoch ms  | Time the operation occurred; legacy timestamp forms remain readable. |

### 5. Automatic Feeder

#### 5.1 tanks/{tankId}/feeder/status

Single live feeder-status document written by the assigned ESP32.

Table 3.21

Data Dictionary of Collection: tanks/{tankId}/feeder/status

| Field Name           | Data Type          | Constraints / Default                                                                | Description                                                    |
|----------------------|--------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------|
| status               | string             | idle   checking_feed_level   dispensing   completed   skipped_insufficient   blocked | Current or most recent feeder state.                           |
| status_reason        | string             | Optional                                                                             | Human-readable explanation when feeding is blocked or skipped. |
| command_id           | string             | Optional                                                                             | Manual command correlated with the current outcome.            |
| dispenseCount        | number             | Default: 0                                                                           | Cumulative completed dispensing cycles.                        |
| lastSeen             | number             | Epoch milliseconds                                                                   | Most recent feeder heartbeat/status time.                      |
| last_dispensed_at    | number   timestamp | Optional                                                                             | Time of the last completed dispensing cycle.                   |
| last_dispensed_grams | number             | Optional; estimated grams                                                            | Estimated amount associated with the last completed cycle.     |
| feed_level           | number             | 0-100 percent                                                                        | Latest hopper level available to the feeder.                   |
| estimated_feed_grams | number             | Optional; grams                                                                      | Estimated mass remaining in the hopper.                        |

#### 5.2 tanks/{tankId}/feeder\_schedules/{scheduleId}

Recurring automatic-feeding schedule. The app blocks overlapping schedules that share at least one day and the same time, regardless of feed amount.

Table 3.22

Data Dictionary of Collection: tanks/{tankId}/feeder\_schedules/{scheduleId}

| Field Name | Data Type | Constraints / Default | Description                                   |
|------------|-----------|-----------------------|-----------------------------------------------|
| time       | string    | H:MM                  | Human-readable scheduled time.                |
| ampm       | string    | AM   PM               | Meridiem indicator.                           |
| timeValue  | number    | 0-1439                | Minutes since midnight, used for ordering and |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name         | Data Type                      | Constraints / Default                                     | Description                                                                          |
|--------------------|--------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------|
|                    |                                |                                                           | overlap validation.                                                                  |
| grams              | number   null                  | 20-200; multiples of 20                                   | Estimated feed amount. Null uses the default 20 g.                                   |
| days               | array<string><br>  string mask | At least one active day                                   | Recurring days of the week. Firmware may use a Sunday-first seven-character mask.    |
| enabled            | boolean                        | Default: true                                             | Whether the schedule is active.                                                      |
| isDone             | boolean                        | Legacy compatibility                                      | True only for a completed latest occurrence; not the authoritative outcome.          |
| created_at         | timestamp                      | Server timestamp                                          | Time the schedule was created.                                                       |
| effective_at_ms    | number                         | UTC epoch milliseconds                                    | Creation/edit/re-enable time; prevents earlier occurrences from being marked missed. |
| last_outcome       | string                         | completed  <br>skipped_insufficient  <br>blocked   failed | Reconciled outcome of the latest applicable occurrence.                              |
| last_occurrence_at | number                         | UTC epoch milliseconds                                    | Scheduled minute associated with last_outcome.                                       |

### 5.3 tanks/{tankId}/feeder\_logs/{logId}

Append-only audit log for manual, scheduled, missed, blocked, skipped, failed, and completed feeding events.

Table 3.23

Data Dictionary of Collection: tanks/{tankId}/feeder\_logs/{logId}

| Field Name                | Data Type | Constraints / Default                                     | Description                                                    |
|---------------------------|-----------|-----------------------------------------------------------|----------------------------------------------------------------|
| action                    | string    | Required                                                  | Human-readable feeding-event description.                      |
| type                      | string    | auto   manual   missed  <br>error                         | Source/category of the feeder event.                           |
| status                    | string    | completed  <br>skipped_insufficient  <br>blocked   failed | Terminal result reported by the ESP32.                         |
| command_id                | string    | Optional                                                  | Originating manual command ID.                                 |
| schedule_key              | string    | Scheduled feeds only                                      | Originating schedule document ID.                              |
| schedule_time             | string    | Scheduled feeds only                                      | Original schedule time shown in the audit record.              |
| occurrence_at             | number    | UTC epoch milliseconds                                    | Original occurrence time retained through offline retries.     |
| requested_grams           | number    | Required for execution                                    | Feed amount requested by the command or schedule.              |
| estimated_dispensed_grams | number    | Completed only                                            | Servo-cycle estimate; not scale-measured proof of actual mass. |
| amount_basis              | string    | servo_cycle_estimate                                      | Explains the basis of the estimated dispensed amount.          |
| estimated_available_      | number    | Optional                                                  | Estimated hopper mass available before                         |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name            | Data Type          | Constraints / Default | Description                                                                |
|-----------------------|--------------------|-----------------------|----------------------------------------------------------------------------|
| grams                 |                    |                       | execution.                                                                 |
| feed_level_before     | number             | Optional; percent     | Hopper percentage before dispensing.                                       |
| feed_level_after      | number             | Optional; percent     | Hopper percentage after dispensing.                                        |
| level_change_detected | boolean            | Optional              | Supporting indication of feed movement; not proof of exact grams.          |
| verification_note     | string             | Optional              | Additional interpretation of the before/after feed-level check.            |
| logged_at             | number   timestamp | New writes: epoch ms  | Original event time. Completed logs alone contribute to Consumption Today. |

### 5.4 tanks/{tankId}/feeder\_commands/{commandId}

Short-lived one-shot manual feeding request consumed by the assigned ESP32.

Table 3.24

#### Data Dictionary of Collection: tanks/{tankId}/feeder\_commands/{commandId}

| Field Name   | Data Type     | Constraints / Default   | Description                                                                    |
|--------------|---------------|-------------------------|--------------------------------------------------------------------------------|
| command_type | string        | feed_now                | Requested manual action.                                                       |
| grams        | number   null | 20-200; multiples of 20 | Requested amount; null uses the default 20 g.                                  |
| issued_by    | string        | Owner UID               | User who created the request.                                                  |
| issued_at    | timestamp     | Server timestamp        | Authoritative request-creation time.                                           |
| expires_at   | timestamp     | Required for new writes | Application deadline; firmware also rejects requests more than 60 seconds old. |

Note: Deletion of a command is not proof of completed feeding. The UI confirms outcomes using a matching command\_id in feeder status/log records.

### 5.5 tanks/{tankId}/feeder\_notification\_receipts/{logId}

Server-only at-most-once claim used to prevent duplicate push attempts for feeder outcome logs.

Table 3.25

#### Data Dictionary of Collection: tanks/{tankId}/feeder\_notification\_receipts/{logId}

| Field Name              | Data Type | Constraints / Default | Description                                        |
|-------------------------|-----------|-----------------------|----------------------------------------------------|
| uid                     | string    | Required              | Recipient owner UID.                               |
| push_attempt_claimed_at | timestamp | Server timestamp      | Time the backend claimed the notification attempt. |

Note: A receipt proves that the push attempt was claimed, not that the device received the FCM message. The notification inbox record remains available.



## 6. Machine Learning-Based Water Quality Assessment

### 6.1 tanks/{tankId}/machine\_learning\_assessments/{assessmentId}

Stores the current and hourly Water Quality Assessment. The current document ID is current; historical IDs use sortable UTC timestamps. The assessment analyzes water-sensor history and does not directly control actuators.

Table 3.26

#### Data Dictionary of Collection: tanks/{tankId}/machine\_learning\_assessments/{assessmentId}

| Field Name              | Data Type     | Constraints / Default                             | Description                                                                                   |
|-------------------------|---------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------|
| uid                     | string        | Required                                          | Owner UID associated with the assessment.                                                     |
| tank_id                 | string        | Required                                          | Tank evaluated by the assessment.                                                             |
| level                   | string        | Good   Moderate   Poor   Critical   Insufficient  | Final user-facing assessment after safety and data-availability handling.                     |
| model_level             | string        | Good   Moderate   Poor   Critical                 | Raw Random Forest class when model inference is applied.                                      |
| rule_level              | string        | Optional                                          | Deterministic rolling-risk safety floor used by the interpretation layer.                     |
| safety_override         | boolean       | Default: false                                    | True when the safety floor raises the displayed severity.                                     |
| source                  | string        | Required                                          | Distinguishes trained model output, deterministic fallback, insufficient data, or stale data. |
| model_algorithm         | string        | RandomForestClassifier   Rule-based   Not applied | Algorithm recorded for traceability.                                                          |
| model_version           | string        | Required when model used                          | Version embedded in the deployed model artifact.                                              |
| training_data_origin    | string        | Required                                          | Origin of model-training records, such as synthetic prototype or actual pond history.         |
| label_origin            | string        | Required                                          | Origin of training labels; documents current prototype limitations.                           |
| model_feature_count     | number        | Required when model used                          | Number of engineered model inputs.                                                            |
| analysis_window_minutes | number        | Default: 60                                       | Historical duration analyzed for the assessment.                                              |
| data_status             | string        | ready   insufficient   stale                      | Availability and freshness classification of the input history.                               |
| source_recorded_at      | string   null | ISO-8601                                          | Timestamp of the newest actual source reading.                                                |
| source_age_seconds      | number        | Non-negative                                      | Age of the newest source reading at processing time.                                          |
| confidence              | number        | Model output only                                 | Model confidence; omitted or not presented as ML confidence for fallback results.             |
| driver                  | string        | DO   pH   temp   turbidity   waterLevel   overall | Primary parameter driving the assessment.                                                     |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name                | Data Type   | Constraints / Default                  | Description                                                  |
|---------------------------|-------------|----------------------------------------|--------------------------------------------------------------|
| <b>driver_label</b>       | string      | Required                               | Human-readable driver name.                                  |
| <b>driver_value</b>       | number      | Optional                               | Representative value of the primary driver.                  |
| <b>driver_unit</b>        | string      | Optional                               | Measurement unit of driver_value.                            |
| <b>driver_min</b>         | number      | Optional                               | Minimum driver value in the analyzed window.                 |
| <b>driver_max</b>         | number      | Optional                               | Maximum driver value in the analyzed window.                 |
| <b>problem</b>            | string      | Optional                               | Short statement of the detected water-quality concern.       |
| <b>insight</b>            | string      | Optional                               | Trend-aware explanation for the user.                        |
| <b>action</b>             | string      | Optional                               | Recommended monitoring or management response.               |
| <b>concerns</b>           | array   map | Optional                               | Primary parameter concerns identified during interpretation. |
| <b>secondary_concerns</b> | array   map | Optional                               | Additional lower-priority concerns.                          |
| <b>ts_epoch</b>           | number      | UTC epoch seconds/ms by implementation | Sortable processing timestamp used by history queries.       |
| <b>timestamp</b>          | string      | ISO-8601                               | Human-readable assessment processing time.                   |

Note: Inference requires at least six valid contiguous ten-minute records. A newest source record older than 20 minutes produces Insufficient/state instead of inferring a current condition. Feed level is not a WQA feature.

## 7. Grow-Out Production and Sampling

### 7.1 tanks/{tankId}/batches/{batchId}

Stores one crayfish grow-out batch and its current production summary. Population values are counted; biomass is estimated from sampling.

Table 3.27

Data Dictionary of Collection: tanks/{tankId}/batches/{batchId}

| Field Name             | Data Type | Constraints / Default           | Description                                                     |
|------------------------|-----------|---------------------------------|-----------------------------------------------------------------|
| <b>batch_status</b>    | string    | active   harvested   superseded | Current lifecycle state of the batch.                           |
| <b>stocking_date</b>   | timestamp | Required                        | Stocking date; the setup calendar day is Day 0.                 |
| <b>harvest_date</b>    | timestamp | Optional                        | Date the batch was harvested.                                   |
| <b>initial_count</b>   | number    | Non-negative integer            | Counted population stocked at Day 0.                            |
| <b>current_count</b>   | number    | Non-negative integer            | Counted population after recorded mortality and harvest events. |
| <b>harvest_count</b>   | number    | Default: 0                      | Cumulative crayfish harvested.                                  |
| <b>total_mortality</b> | number    | Default: 0                      | Cumulative recorded mortality.                                  |





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name           | Data Type | Constraints / Default | Description                                                                               |
|----------------------|-----------|-----------------------|-------------------------------------------------------------------------------------------|
| harvest_weight_grams | number    | Optional              | Cumulative recorded harvest weight in grams.                                              |
| initial_abw          | number    | Optional; grams       | Average body weight from the baseline sample.                                             |
| initial_abl          | number    | Optional; centimeters | Average body length from the baseline sample.                                             |
| final_abw            | number    | Optional; grams       | Final average body weight at harvest.                                                     |
| final_abl            | number    | Optional; centimeters | Final average body length at harvest.                                                     |
| days_in_culture      | number    | Default: 0            | Calendar days since stocking, updated at midnight rather than as rolling 24-hour periods. |
| sample_count         | number    | Default: 0            | Number of non-baseline sampling records.                                                  |
| initial_total_weight | number    | Optional; grams       | Total weight of the baseline sample.                                                      |
| initial_total_length | number    | Optional; centimeters | Total length of the baseline sample.                                                      |
| created_at           | timestamp | Server timestamp      | Time the batch record was created.                                                        |

### 7.2 tanks/{tankId}/batches/{batchId}/sampling\_records/{recordId}

Baseline and weekly sample measurements used to monitor growth and estimate biomass without measuring every crayfish.

Table 3.28

#### Data Dictionary of Collection: tanks/{tankId}/batches/{batchId}/sampling\_records/{recordId}

| Field Name      | Data Type | Constraints / Default | Description                                                                                |
|-----------------|-----------|-----------------------|--------------------------------------------------------------------------------------------|
| sampling_date   | timestamp | Required              | Date the sample was collected.                                                             |
| avg_body_weight | number    | grams                 | Average sample weight: total_weight divided by sample_size.                                |
| avg_body_length | number    | centimeters           | Average sample length: total_length divided by sample_size.                                |
| sample_size     | number    | Positive integer      | Number of crayfish physically measured in the sample.                                      |
| total_weight    | number    | grams                 | Combined weight of sampled crayfish.                                                       |
| total_length    | number    | centimeters           | Combined measured length of sampled crayfish.                                              |
| biomass         | number    | Estimated grams       | Estimated tank biomass calculated from current counted population and average body weight. |
| live_count      | number    | Counted population    | Current counted live population used in the biomass estimate.                              |
| is_baseline     | boolean   | Default: false        | True for the Day 0 baseline; false for weekly samples.                                     |
| created_at      | timestamp | Server timestamp      | Time the sampling record was created.                                                      |

Note: Weekly sampling begins seven calendar days after stocking and continues every seven days from the previous weekly sample. The baseline record is excluded from Week N counting.

**7.3 tanks/{tankId}/batches/{batchId}/mortality\_records/{recordId}**

Records a discovered mortality event and updates the batch's counted current population.

Table 3.29

**Data Dictionary of Collection: tanks/{tankId}/batches/{batchId}/mortality\_records/{recordId}**

| Field Name      | Data Type | Constraints / Default | Description                                         |
|-----------------|-----------|-----------------------|-----------------------------------------------------|
| mortality_date  | timestamp | Required              | Date the dead crayfish were discovered or recorded. |
| mortality_count | number    | Positive integer      | Number of mortalities in the event.                 |
| created_at      | timestamp | Server timestamp      | Time the record was created.                        |

**7.4 tanks/{tankId}/batches/{batchId}/harvest\_records/{recordId}**

Records a partial or final harvest event.

Table 3.30

**Data Dictionary of Collection: tanks/{tankId}/batches/{batchId}/harvest\_records/{recordId}**

| Field Name      | Data Type | Constraints / Default | Description                                     |
|-----------------|-----------|-----------------------|-------------------------------------------------|
| batch_id        | string    | Required              | Parent batch identifier retained in the record. |
| harvest_date    | timestamp | Required              | Date the harvest was performed.                 |
| harvest_count   | number    | Positive integer      | Number of crayfish harvested.                   |
| total_weight_kg | number    | kilograms             | Combined harvest weight.                        |
| abw_grams       | number    | grams                 | Average harvested body weight.                  |
| created_at      | timestamp | Server timestamp      | Time the record was created.                    |

**8. Notifications****8.1 notifications/{notifId}**

User notification inbox record. Deterministic IDs are used where duplicate backend events must collapse into one notification.

Table 3.31

**Data Dictionary of Collection: notifications/{notifId}**

| Field Name | Data Type | Constraints / Default                                    | Description                                                    |
|------------|-----------|----------------------------------------------------------|----------------------------------------------------------------|
| uid        | string    | Required                                                 | UID of the intended recipient.                                 |
| notif_type | string    | critical   warning   reminder<br>  device_auto   general | Notification category used for grouping and preference checks. |
| title      | string    | Required                                                 | Short notification heading.                                    |
| body       | string    | Required                                                 | Detailed notification message.                                 |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Field Name | Data Type | Constraints / Default | Description                                                 |
|------------|-----------|-----------------------|-------------------------------------------------------------|
| is_read    | boolean   | Default: false        | Whether the user opened or marked the notification as read. |
| created_at | timestamp | Server timestamp      | Time the notification record was created.                   |

Note: Completed and skipped-insufficient feeder logs create inbox records and immediate FCM pushes when the user's Feeding preference is enabled.

### 9. ESP32 Sensor-Ingestion Staging

#### 9.1 sensorIngestion/current

Five-second ESP32 staging snapshot. Cloud Functions validate its capture-time assignment before routing it to the active tank.

Table 3.32

Data Dictionary of Collection: sensorIngestion/current

| Field Name              | Data Type | Constraints / Default | Description                                                       |
|-------------------------|-----------|-----------------------|-------------------------------------------------------------------|
| hardwareId              | string    | Required              | Stable identifier of the reporting hardware.                      |
| source_tank_id          | string    | Required              | Tank bound to the hardware when the reading was captured.         |
| source_owner_uid        | string    | Required              | Owner bound to the hardware at capture time.                      |
| source_assignment_at_ms | number    | Epoch milliseconds    | Assignment identity used to reject stale or cross-owner payloads. |
| captured_at_ms          | number    | Epoch milliseconds    | Original capture time when the device clock is trusted.           |
| temperature             | number    | Optional per validity | Live temperature value.                                           |
| ph_level                | number    | Optional per validity | Live pH value.                                                    |
| dissolved_oxygen        | number    | Optional per validity | Live dissolved oxygen value.                                      |
| turbidity               | number    | Optional per validity | Live water turbidity value.                                       |
| turbidity_air           | number    | Optional              | Turbidity reference/air value.                                    |
| water_level             | number    | Optional per validity | Live pond water-level value.                                      |
| feed_level              | number    | Optional per validity | Live hopper percentage.                                           |
| estimated_feed_grams    | number    | Optional              | Estimated hopper mass.                                            |
| buffered_entries        | number    | Default: 0            | Count of device readings waiting in the local buffer.             |



## 9.2 sensorIngestion/current/history/{docId}

Ten-minute ESP32 aggregate staging record. Matching assignments are routed to tank history; mismatched or unbound records remain quarantined.

Table 3.33

Data Dictionary of Collection: sensorIngestion/current/history/{docId}

| Field Name                   | Data Type | Constraints / Default       | Description                                                                                                                                    |
|------------------------------|-----------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| hardwareId                   | string    | Required                    | Stable identifier of the reporting hardware.                                                                                                   |
| source_tank_id               | string    | Required                    | Tank bound at capture time.                                                                                                                    |
| source_owner_uid             | string    | Required                    | Owner bound at capture time.                                                                                                                   |
| source_assignment_at_ms      | number    | Epoch milliseconds          | Capture-time assignment identity.                                                                                                              |
| captured_at_ms               | number    | Epoch milliseconds          | Original aggregate completion time.                                                                                                            |
| sensor<br>MIN/MAX/AVG fields | number    | Only valid sensors included | Ten-minute aggregates for temperature, pH, dissolved oxygen, turbidity, and water level; optional operational feed values may also be present. |
| routing_status               | string    | Optional: quarantined       | Routing result when the payload cannot be safely assigned.                                                                                     |
| routing_reason               | string    | Optional                    | Reason the payload remains in staging.                                                                                                         |

## 3.3 DEVELOPMENT METHODOLOGY

This section describes the phases of the selected development methodology and how the proposed workflow is intended to guide the development of the CrayCare system throughout the study. The chosen methodology is expected to serve as the framework for organizing and managing the different activities involved in the project, from the initial planning stage up to system testing and implementation. Through this approach, project tasks are intended to be identified, arranged, and completed in a more systematic and organized manner during the development process.

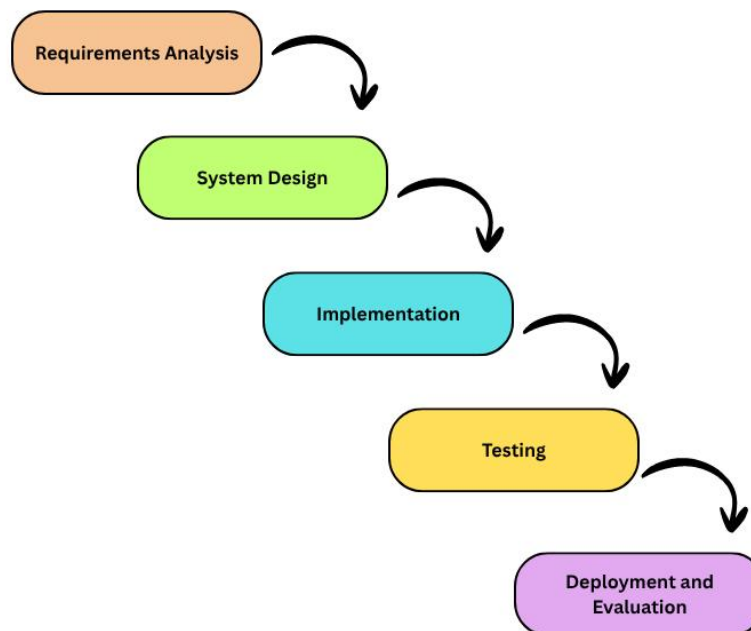
The selected methodology is also expected to provide a clear structure for the planning, design, development, testing, and implementation phases of the proposed system. Following a defined development process may help maintain consistency



throughout the study and support the development of a functional and reliable IoT- and mobile-based crayfish monitoring system for the experimental setup.

### 3.3.1 Process Model

The CrayCare system development will be based on the Waterfall Model, which is a software development model that consists of a sequence of software development phases. This model will be adopted because the project is well-defined and has a clearly defined system scope prior to the start of development. A research-based project like CrayCare is one in which proper documentation, traceability, and systematic progression throughout development are critical, making the Waterfall Model suitable.





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

*Figure 3.41. Waterfall Model*

The process will start from the Requirements Analysis phase, where the research team will perform thorough requirements analysis through literature and studies review, to obtain the functional and non-functional requirements for the system. The researchers will also determine the scope and limitations and set the boundaries of the study. Further, interviews and consultations will be held with aquaculture experts and crayfish farmers to gain a better understanding of the practical needs and common challenges associated with crayfish farming operations. This phase will be used to develop the remaining phases of the system.

Once the analysis phase has been completed, the project will move to the System Design phase in which the high-level design of the CrayCare system will be detailed. The hardware configuration with the ESP32 microcontroller and environmental sensors, network infrastructure using Wi-Fi and Firebase, mobile application interface layout, and database structure will be designed during this phase. Diagrams such as the hardware architecture, network topology, and data flow diagrams will also be prepared to serve as guides during system implementation and integration.

The Implementation phase will focus on building and integrating all components of the proposed system. PlatformIO will be used to program the ESP32 microcontroller to control sensor data collection, automated responses in response to predefined thresholds, and real-time data transmission to the Firebase. Simultaneously, the mobile application will be developed using Flutter framework



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

and Visual Studio Code to provide real-time monitoring, alert notifications, and automatic feeding management. Additionally, integration between the hardware and software components will be implemented to facilitate the seamless communication and synchronization of the system.

In the Testing phase, the researchers will perform a set of tests to ensure the system meets all required features. Unit testing, sensor calibration testing, software testing, and integration testing will be performed to identify and resolve possible errors, inconsistencies, or communication issues within the system. The testing methods will involve measuring sensor values, checking the reliability of automated responses, the responsiveness of the mobile app, and the stability of the communication between the ESP32 microcontroller and Firebase.

Finally, the Deployment and Evaluation phase will involve the installation and operation of the CrayCare system within the experimental setup consisting of two grow-out tanks. The data will be gathered during a defined timeframe of experimentation and will be compared to the performance of the IoT-based monitoring method applied in Tank A and the traditional manual monitoring method applied in Tank B. The collected data will then be analyzed to assess the effectiveness, accuracy, reliability, and overall performance of the proposed CrayCare system.

### 3.3.2 Development Tools

Table 3.32

#### Development Tools



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

| Category                     | Tool / Technology  | Description                                                                                                                                 |
|------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Programming Language         | C++                | Used for programming the ESP32 microcontroller through the Arduino framework and handling sensor integration and hardware control functions |
|                              | Dart               | Used for developing the mobile application together with the Flutter framework                                                              |
| Mobile Application Framework | Flutter            | Used for creating the cross-platform mobile application interface and functionalities                                                       |
| Development Environment      | PlatformIO         | Used for writing, compiling, uploading, and monitoring firmware for the ESP32 microcontroller through Visual Studio Code                    |
| Code Editor                  | Visual Studio Code | Used for editing, organizing, and debugging the mobile application source code                                                              |

*Continuation of Table 3.32*

| Category                | Tool / Technology          | Description                                                                                                   |
|-------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------|
| Design Tools            | Figma                      | Used to create UI/UX prototypes and interactive wireframes.                                                   |
|                         | Canva                      | Used for visual assets, infographics, and system documentation.                                               |
| Microcontroller         | ESP32                      | Serves as the main processing unit for sensor monitoring, Wi-Fi communication, and automated hardware control |
| Cloud Database Platform | Firebase Realtime Database | Used for storing, synchronizing, and managing real-time sensor and monitoring data                            |
| Authentication Service  | Firebase Authentication    | Used for managing user access and account security within the mobile application                              |





|                        |                              |                                                                                          |
|------------------------|------------------------------|------------------------------------------------------------------------------------------|
| Version Control System | Git                          | Used for tracking code revisions and managing source code changes throughout development |
| Collaboration Platform | GitHub                       | Used for project backup, repository management, and collaborative development            |
| Prototyping Tools      | Breadboards and Jumper Wires | Used for temporary circuit assembly and hardware testing during prototyping              |

Table 3.32 presents the development tools and technologies intended to be used in the design, development, and implementation of the proposed CrayCare system. The listed tools support the different stages of system development, including hardware configuration, mobile application creation, database integration, interface design, testing, and project management.

The selection of these tools was based on suitability for the requirements of the proposed system and the ability to support the integration of hardware and software components. These tools will serve as the working environment for the researchers in developing and preparing the proposed system for implementation.

### 3.4 TEST METHODOLOGY/PROCEDURES

This section presents the testing methods and procedures that will be used to evaluate the functionality, performance, and reliability of the proposed IoT- and mobile-based crayfish monitoring system. Testing activities will be conducted to identify possible errors, verify whether the system performs according to the intended requirements, and



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

ensure that both the hardware and software components operate properly within the experimental setup.

Several testing procedures will be carried out during the development and implementation phases of the study. Hardware testing will be performed to check the functionality and responsiveness of the environmental sensors, ESP32 microcontroller, automated feeder, and other connected components. Software testing will also be conducted to assess the performance of the mobile application, database connectivity, real-time monitoring features, notifications, and data transmission processes.

The study will also conduct integration testing to verify proper communication among the sensors, microcontroller, cloud database, and mobile application. Monitoring accuracy, sensor responsiveness, alert generation, and automated feeding operation will be observed during the experimental period to determine whether the different components of the system function properly as a whole.

Actual testing within the grow-out setup will also be performed to evaluate the overall stability and performance of the proposed system under controlled conditions. Any issues or inconsistencies identified during the testing procedures will be documented and addressed to further improve the reliability and functionality of the proposed system before the final evaluation phase.

### **3.5 SYSTEM REQUIREMENTS**

This section presents the hardware, software, and other technical resources required for the development, implementation, and operation of the proposed CrayCare system. The specified requirements are categorized into minimum and recommended



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

specifications to help ensure proper system functionality, stable monitoring performance, and efficient access to the IoT- and mobile-based monitoring features. These requirements are intended to support both the development process and the actual implementation of the proposed system within the experimental setup.

Table 3.33

**System Requirements for the CrayCare System**

| Category         | Minimum Requirement                    | Recommended Requirement                          |
|------------------|----------------------------------------|--------------------------------------------------|
| Processor        | Intel Core i3 or equivalent            | Intel Core i5 or higher                          |
| RAM              | 4 GB                                   | 8 GB or higher                                   |
| Storage          | 500 GB HDD or 128 GB SSD               | 256 GB SSD or higher                             |
| Operating System | Windows 10 / Android 8.0 or higher     | Windows 11 / Android 10 or higher                |
| Database         | Firebase Realtime Database (free tier) | Firebase Realtime Database with expanded storage |

*Continuation of Table 3.33*

| Category            | Minimum Requirement                           | Recommended Requirement                                                                  |
|---------------------|-----------------------------------------------|------------------------------------------------------------------------------------------|
| Development Tools   | PlatformIO, Visual Studio Code, Flutter SDK   | PlatformIO with Visual Studio Code, Dart/Flutter extensions, Flutter SDK (latest stable) |
| Internet Connection | Standard broadband for Firebase data sync     | Stable broadband or LAN access with consistent upload/download speed                     |
| Mobile Device       | Android smartphone running Android 8.0 (Oreo) | Android smartphone running Android 10 or higher with Wi-Fi capability                    |
| Microcontroller     | ESP32 with standard firmware                  | ESP32 with latest firmware and stable power supply unit                                  |



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

|                  |                                                                                              |                                                                  |
|------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Sensors          | pH Sensor, Temperature Sensor, Dissolved Oxygen Sensor, Turbidity Sensor, Water Level Sensor | Calibrated sensors with improved accuracy and durability         |
| Relay Module     | 4-channel relay module                                                                       | High-quality 4-channel relay module for stable automated control |
| Automated Feeder | Basic servo- or motor-based feeder mechanism                                                 | Stable automated feeder with adjustable feeding setup            |

As shown in Table 3.33, the minimum requirements represent the basic resources necessary for the system to operate properly, while the recommended requirements provide more stable performance, improved processing capability, and better overall system reliability. Hardware components such as the ESP32 microcontroller, sensors, 4-channel relay module, and automated feeder are expected to support the monitoring and automation functions of the proposed system. Meanwhile, the specified software tools, database platform, and internet connectivity are intended to support data processing, remote monitoring, and communication between the hardware devices and the mobile application.

### 3.6 QUALITY PLAN

The Quality Plan describes the approaches and techniques that should be employed throughout the development process to ensure that CrayCare reaches the appropriate level of performance, functionality, and user satisfaction.

In order to achieve quality assurance, structured processes will be used for development and testing of CrayCare. Code review at specified intervals will be done in



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

order to ensure clean coding. System testing will be conducted at each phase to confirm that all system components function properly before moving to the next development phase. Feedback gathering will be done during testing and development processes in order to fix problems and improve user experience. Ensuring high-quality coding standards and proper documentation will facilitate system maintenance and scalability, allowing developers to efficiently implement updates and address potential issues.

The quality of the proposed system will be evaluated using selected characteristics from the ISO/IEC 25010, namely: Functional Suitability, Usability, Reliability, Performance Efficiency, Compatibility, and Security. These selected characteristics are considered the most relevant and applicable criteria for assessing the proposed IoT- and mobile-based crayfish monitoring system.

Functional Suitability will be determined based on the system's ability to provide accurate water parameter monitoring and properly perform its intended automated functions. Performance Efficiency will be assessed through the real-time processing and transmission of monitoring data with minimal delays between the sensors, database, and mobile application. Usability will be evaluated through testing sessions and feedback gathered from the evaluators regarding the ease of use and understandability of the system. Reliability will be determined based on the consistency of system operation, stability of automated functions, and continuous recording of monitoring data throughout the experimental period. Compatibility will be evaluated based on the ability of the system components, including the ESP32, sensors, Firebase database, and mobile application, to work together properly within the proposed setup. Security will be assessed based on the



system's ability to protect stored data and restrict access through implemented security measures such as authentication and controlled user permissions.

User Acceptance Testing (UAT) will be carried out to help verify whether CrayCare performs necessary tasks efficiently and effectively and is user-friendly. During User Acceptance Testing, members of the research team will act as end-users to perform representative actions such as viewing sensor data, receiving notification messages about water conditions, and setting automatic feeding schedules. The results obtained from the test sessions will be used to optimize the mobile application interface. Through this iterative process, the study aims to develop a fully functional and user-friendly version of the system suitable for a small-scale crayfish farming environment.

### **3.7 IMPLEMENTATION PLAN**

The Implementation Plan outlines the proposed process for deploying the developed system for actual use within the experimental setup of the study. The proposed IoT- and mobile-based crayfish monitoring system will be installed and configured according to the operational requirements of the research, including the setup of user access, mobile application connectivity, sensor integration, and database configuration.

Prior to the actual implementation, the hardware and software components of the system will undergo final integration and functionality testing to verify proper operation of the sensors, ESP32 microcontroller, mobile application, automated feeder, and cloud database connectivity. Necessary system configurations and monitoring parameters will also be prepared to ensure accurate collection and transmission of water parameter data during the experimental period.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The implementation process will further include trial operation and observation within the grow-out setup to assess the stability and usability of the proposed system under controlled conditions. The researchers will observe and evaluate the system during the implementation process to identify possible technical issues and verify whether the system operates according to the intended design and functional requirements.

Once the system has passed the necessary testing and validation procedures, it will be used during the comparative experimental phase involving Tank A and Tank B. Basic procedures for system operation, monitoring, and maintenance will also be prepared to support proper management of the system throughout the duration of the study.

### **3.8 EVALUATION PLAN**

The evaluation plan describes the procedures that will be used to assess the functionality, usability, reliability, and overall performance of the proposed IoT- and mobile-based crayfish monitoring system. The evaluation will focus on determining whether the proposed system is capable of meeting the intended objectives and operational requirements of the study.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

System evaluation will be conducted through actual testing and observation within the experimental setup involving Tank A and Tank B. The performance of the proposed system will be assessed in terms of real-time monitoring, sensor functionality, mobile accessibility, alert notifications, automated feeding support, and overall system responsiveness during the experimental period.

The proposed system will also be evaluated by selected IT experts and individuals knowledgeable in aquaculture practices. A survey questionnaire using a 5-point Likert Scale will be used to gather feedback regarding the functionality, usability, reliability, and overall effectiveness of the proposed system. The scale will consist of the following responses: 5 – Strongly Agree, 4 – Agree, 3 – Neutral, 2 – Disagree, and 1 – Strongly Disagree. Evaluation will be based on observations and actual experience during the system demonstration and testing procedures.

The evaluation will focus on ease of use, accuracy of sensor readings, responsiveness, reliability, and effectiveness of the monitoring features. System performance during continuous operation, including the delivery of monitoring data and notifications, will also be assessed to determine whether the proposed system can properly support the management of the grow-out setup.

The results gathered from the evaluation process will be analyzed to identify possible issues, system limitations, and areas that may still require improvement. The findings will serve as the basis for determining the overall practicality, usability, and effectiveness of the proposed system within the scope of the study.





### **3.9 ETHICAL CONSIDERATIONS**

This section presents the ethical principles and standards that will be observed throughout the conduct of the study. Ethical practices will be maintained during the development of the proposed system, data gathering procedures, experimental activities, and evaluation process to ensure responsible and respectful research conduct.

The study will follow the provisions of the Data Privacy Act of 2012 in handling all collected information and system-generated data. While the study does not involve highly sensitive personal information, all gathered data will still be handled responsibly and used solely for academic and research purposes. Information obtained during testing and evaluation activities will remain confidential, and the presentation of results will avoid revealing the identity of any participant involved in the study.

Participation in evaluation activities will be voluntary. Individuals involved in the evaluation process, including IT experts and persons knowledgeable in aquaculture practices, will first be informed about the purpose of the study, the procedures involved, and the nature of participation before any evaluation activities are conducted. Consent will be secured prior to testing and evaluation, and participation may be withdrawn at any point if desired.

Proper acknowledgment of related literature, studies, and other reference materials used in the development of the proposed system will also be maintained throughout the study. Academic publications, technical documents, and online resources will be properly cited using appropriate academic standards. The study will strictly avoid plagiarism and any improper use of information or intellectual property.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The study will also promote responsible and ethical research practices throughout the experimental process. The proposed monitoring and automated feeding features are intended only to support proper crayfish management within the experimental setup. Proper care and handling of the crayfish will be observed during the study to help maintain suitable living conditions and avoid unnecessary stress or harm to the organisms involved.

### **3.10 DATA ANALYSIS**

This section describes how the data gathered during the study will be processed and analyzed to evaluate the performance and effectiveness of the proposed IoT- and mobile-based crayfish monitoring system.

Collected data from sensor readings, monitoring records, biomass computations, crayfish growth measurements, survival observations, and system evaluation questionnaires will be organized, summarized, and interpreted throughout the experimental period. The gathered information will be used to assess water parameter conditions, crayfish growth performance, feeding management, and overall system operation within the grow-out setup.

Periodic measurements of crayfish body length and body weight will also be collected to monitor growth and development during the experiment. The recorded measurements will be used in computing Average Body Weight (ABW), biomass, and other growth-related observations. In addition, the gathered measurements will be interpreted using the established crayfish size and weight classifications to help identify the developmental stages observed throughout the grow-out period.



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The collected experimental data from Tank A and Tank B will also be examined to determine possible differences between the IoT-based monitoring setup and the traditional manual monitoring setup under similar grow-out conditions. Observed results throughout the experimental period will serve as the basis for evaluating the effectiveness and practicality of the proposed monitoring approach. In addition, Return on Investment (ROI) computations will be included in the data analysis to estimate the potential economic viability, cost-efficiency, and business potential of the proposed CrayCare system within a small-scale aquaculture environment.

Responses gathered from the system evaluation questionnaire will also be analyzed to assess the functionality, usability, reliability, responsiveness, and overall effectiveness of the proposed system based on the evaluation results and feedback obtained during testing and demonstration activities.

Spreadsheet software such as Microsoft Excel or WPS Spreadsheet will be used to organize, record, process, and present the gathered data throughout the study. The software will assist in encoding monitoring results, organizing experimental records, and preparing summarized data for analysis and interpretation.

The analyzed data will serve as the basis for identifying the strengths, limitations, and overall practicality of the proposed system within the scope of the study.

### 3.11 STATISTICAL TREATMENTS



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

This section presents the statistical methods that will be used to analyze and interpret the data gathered during the study. The statistical treatments will help evaluate the performance of the proposed IoT- and mobile-based crayfish monitoring system and support the comparison between the IoT-monitored grow-out tank and the manually managed grow-out tank.

The statistical tools to be used are as follows:

### 1. Descriptive Statistics

Descriptive statistics will be used to summarize and present the collected experimental data throughout the study. It will allow the researchers to describe the gathered information through numerical summaries, tables, and graphical representations.

The statistical measures that will be utilized in this study are the following:

**Mean** — The arithmetic mean will be used to determine the average values of the collected quantitative data, including crayfish growth measurements and other numerical observations obtained during the experimental period. This will be applied to summarize the growth performance of crayfish in both Tank A and Tank B.

$$\bar{x} = \frac{\sum x}{n}$$

Where:

$\bar{x}$  = mean



$\sum x$  = sum of all values

$n$  = total number of observations

**Percentage** — Percentage will be employed to determine the proportion and distribution of observed results, particularly for mortality rate, survival rate, and other recorded experimental outcomes during the study.

$$P = \frac{f}{N} \times 100$$

Where:

$P$  = percentage

$f$  = frequency

$N$  = total number

## 2. Independent Samples t-Test

The Independent Samples t-Test will be used to determine whether there is a significant difference between the IoT-assisted setup (Tank A) and the manually managed setup (Tank B). This test will be applied to selected quantitative variables to determine whether the two management approaches produce different outcomes.

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$



Where:

$t$  = computed t-value

$\bar{x}_1$  = mean of Tank A

$\bar{x}_2$  = mean of Tank B

$s_1^2$  = variance of Tank A

$s_2^2$  = variance of Tank B

$n_1$  = number of observations in Tank A

$n_2$  = number of observations in Tank B

### 3. Weighted Mean

For the system evaluation questionnaire, weighted mean will be used to analyze the responses gathered from the evaluators. This statistical treatment will help determine the level of agreement regarding the functionality, usability, reliability, responsiveness, and overall effectiveness of the proposed system based on the evaluation results.

$$WM = \frac{\sum fw}{N}$$

Where:

WM = weighted mean

$f$  = frequency of responses

$w$  = assigned weight

$N$  = total number of responses



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

The survey will utilize a 5-point Likert Scale to gather and interpret the responses of the evaluators involved in the study. The scale to be used for the evaluation is presented below:

| Scale | Range       | Verbal Interpretation |
|-------|-------------|-----------------------|
| 5     | 4.21 – 5.00 | Strongly Agree        |
| 4     | 3.41 – 4.20 | Agree                 |
| 3     | 2.61 – 3.40 | Neutral               |
| 2     | 1.81 – 2.60 | Disagree              |
| 1     | 1.00 – 1.80 | Strongly Disagree     |

## REFERENCES

### A. Journal Articles

- Ahmed, F., Bijoy, M. H. I., Hemal, H. R., & Noori, S. R. H. (2024). Smart aquaculture analytics: Enhancing shrimp farming in Bangladesh through real-time IoT monitoring and predictive machine learning analysis. *Heliyon*, 10(20), Article e37330. <https://doi.org/10.1016/j.heliyon.2024.e37330>
- Arivarasi, A., Raj, A., Kumar, A., Nesam, J. J., Harini, & Muralidhar. (2026). IoT-based automated water quality monitoring system for fish hatcheries. *EPJ Web of Conferences*, 367, 03009. <https://doi.org/10.1051/epjconf/202636703009>
- Atilkan, Y., Kirik, B., Acici, K., Benzer, R., Ekinici, F., Guzel, M. S., Benzer, S., & Asuroglu, T. (2024). Advancing crayfish disease detection: A comparative study of deep



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- learning and canonical machine learning techniques. *Applied Sciences*, 14(14), 6211. <https://doi.org/10.3390/app14146211>
- Ayon, M. F. I., Nahar, S., Rahman, A., Arif, M. T., Hasib, A., & Akib, A. S. M. A. S. (2026). An IoT-enabled smart aquarium system for real-time water quality monitoring and automated feeding. *arXiv preprint arXiv:2601.08484*.  
<https://doi.org/10.48550/arXiv.2601.08484>
- Badryzlova, N., Assylbekova, S., Koishybayeva, S., Bulavin, Y., & Makhmetov, I. (2024). Prospects for growing Australian red claw crayfish (*Cherax quadricarinatus*) in the industrial conditions of Kazakhstani fish farms. *AACL Bioflux*, 17(6), 2535–2548.  
<https://bioflux.com.ro/docs/2024.2535-2548.pdf>
- Bautista, M. G. A. C., Palconit, M. G. B., Rosales, M. A., Concepcion, R. S., II, Bandala, A. A., Dadios, E. P., & Duarte, B. (2022). Fuzzy logic-based adaptive aquaculture water monitoring system based on instantaneous limnological parameters. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 26(6), 937–943. <https://doi.org/10.20965/jaciii.2022.p0937>
- Chen, S. M., & Chen, J. C. (2003). Effects of pH on survival, growth, molting and feeding of giant freshwater prawn *Macrobrachium rosenbergii*. *Aquaculture*, 218(1–4), 613–623. [https://doi.org/10.1016/S0044-8486\(02\)00265-X](https://doi.org/10.1016/S0044-8486(02)00265-X)
- Cheng, W. K., Tan, T. B., Tan, J. S., Chan, J. Y.-L., Chen, Y.-L., & others. (2025). Smart aquaculture: Integrating computer vision and IoT for automated crayfish health assessment via federated learning. *IEEE Access*, 13.  
<https://ieeexplore.ieee.org/document/11214143>
- Flores-Iwasaki, M., Guadalupe, G. A., Pachas-Caycho, M., Chapa-Gonza, S., Mori-Zabarburú, R. C., & Guerrero-Abad, J. C. (2025). Internet of Things (IoT) sensors for water quality monitoring in aquaculture systems: A systematic review and bibliometric analysis. *AgriEngineering*, 7(3), Article 78.  
<https://doi.org/10.3390/agriengineering7030078>
- Gallemit, A. B. (2023). Water monitoring and analysis system: Validating an IoT-enabled prototype towards sustainable aquaculture. *International Journal of Advanced Multidisciplinary Studies*, 3(6), 496–517. <https://www.ijams-bbp.net/archive/vol-3-issue-6june/water-monitoring-and-analysis-system-validating-an-iot-enabled-prototype-towards-sustainable-aquaculture/>
- Grandez-Yoplac, D. E., Pachas-Caycho, M., Cristobal, J., Chapa-Gonza, S., Mori-Zabarburú, R. C., & Guadalupe, G. A. (2025). Recirculating aquaculture systems (RAS) for cultivating *Oncorhynchus mykiss* and the potential for IoT integration: A systematic review and bibliometric analysis. *Sustainability*, 17(15), 6729.  
<https://doi.org/10.3390/su17156729>





POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- Gulifardo, A. B. E., Balane, B. J. D., Ebor, M. T., Serrano, M. R., Ellazar, E. P., Nicolas, A. V., & Maaño, R. C. (2025). iPond: An Internet of Things–based water quality monitoring system for fishpond. IOP Conference Series: Earth and Environmental Science, 1446(1), 012005. <https://doi.org/10.1088/1755-1315/1446/1/012005>
- Haddaway, Neal & Mortimer, Robert & Christmas, Martin & Dunn, Alison. (2013). Effect of pH on growth and survival in the freshwater crayfish *Austropotamobius pallipes*. Freshwater Crayfish. 19. 53-62. [https://www.researchgate.net/publication/235660782\\_Effect\\_of\\_pH\\_on\\_growth\\_and\\_survival\\_in\\_the\\_freshwater\\_crayfish\\_Austropotamobius\\_pallipes](https://www.researchgate.net/publication/235660782_Effect_of_pH_on_growth_and_survival_in_the_freshwater_crayfish_Austropotamobius_pallipes)
- Hamasaki, K., Dan, S., & Kawai, T. (2023). Reproductive biology of the red swamp crayfish *Procambarus clarkii* (Girard, 1852) (Decapoda: Astacidea: Cambaridae): A review. Journal of Crustacean Biology, 43(4), ruad057. <https://doi.org/10.1093/jcabi/ruad057>
- Haubrock, P.J., Oficialdegui, F.J., Zeng, Y., Patoka, J., Yeo, D.C.J. and Kouba, A. (2021), The redclaw crayfish: A prominent aquaculture species with invasive potential in tropical and subtropical biodiversity hotspots. Rev. Aquacult., 13: 1488-1530. <https://doi.org/10.1111/raq.12531>
- Hongpin, L., Guanglin, L., Weifeng, P., Jie, S., & Qiuwei, B. (2015). Real-time remote monitoring system for aquaculture water quality. International Journal of Agricultural and Biological Engineering, 8(6), 136–143. <https://ijabe.org/index.php/ijabe/article/view/1486>
- Hossain, M. S., Kouba, A., & Buřič, M. (2019). Morphometry, size at maturity, and fecundity of marbled crayfish (*Procambarus virginalis*). Zoologischer Anzeiger, 281, 68–75. <https://doi.org/10.1016/j.jcz.2019.06.005>
- Huang, Cuixue, Nie, Xiangxing, Wei, Jie, Wang, Yakun, Hong, Kunhao, Mu, Xidong, Liu, Chao, Chu, Zhangjie, Zhu, Xinping, Yu, Lingyun, Effects of Light Spectrum on Survival, Growth, Physiological, and Biochemical Indices of Redclaw Crayfish (*Cherax quadricarinatus*) Juveniles, Aquaculture Research, 2024, 8897473, 10 pages, 2024. <https://doi.org/10.1155/2024/8897473>
- Jiang, Z., Xu, C., Yang, X., Han, F., Chi, M., & Li, E. (2025). Analysis of the optimum krill oil supplementation strategy for ovarian high-quality development in the female redclaw crayfish, *Cherax quadricarinatus*. Aquaculture Reports, 42, Article 102837. <https://doi.org/10.1016/j.aqrep.2025.102837>
- Jones, C. (1995). Effect of temperature on growth and survival of the tropical freshwater crayfish, *Cherax quadricarinatus* (von Martens) (Decapoda, Parastacidae). Freshwater Crayfish, 8, 391–398. <https://www.researchgate.net/publication/257939004>



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- King, C. R. (1994). Growth and survival of redclaw crayfish hatchlings (*Cherax quadricarinatus* von Martens) in relation to temperature, with comments on the relative suitability of *Cherax quadricarinatus* and *Cherax destructor* for culture in Queensland. *Aquaculture*, 122(1), 75–80. [https://doi.org/10.1016/0044-8486\(94\)90335-2](https://doi.org/10.1016/0044-8486(94)90335-2)
- Li, J., Qin, Q., Tian, X., Guo, J., Tang, B., He, Z., Xie, Z., Wang, Y., & Wang, D. (2024). Effects of different cultivation modes on morphological traits and correlations between traits and body mass of crayfish (*Procambarus clarkii*). *Biology*, 13(6), 395. <https://doi.org/10.3390/biology13060395>
- Lindholm-Lehto, P. (2023). Water quality monitoring in recirculating aquaculture systems. *Aquaculture, Fish and Fisheries*. <https://doi.org/10.1002/aff2.102>
- Muthmainnah, M., Khasanah, I. K., Hananto, F. S., Romadani, A., Tazi, I., Mulyono, A., & Tirono, M. (2025). Internet of things-based water quality monitoring design to improve freshwater lobster farming management. *International Journal of Electrical and Computer Engineering*, 15(4), 3717–3726. <https://doi.org/10.11591/ijece.v15i4.pp3717-3726>
- Nagothu, S. K., Bindu Sri, P., Anitha, G., et al. (2025). Advancing aquaculture: Fuzzy logic-based water quality monitoring and maintenance system for precision aquaculture. *Aquaculture International*, 33, 32. <https://doi.org/10.1007/s10499-024-01701-2>
- Naranjo-Páramo, J., Hernández-Llamas, A., Vargas-Mendieta, M., Mercier, L., & Villarreal, H. (2018). Dynamics of commercial size interval populations of female redclaw crayfish (*Cherax quadricarinatus*) reared in gravel-lined ponds: A stochastic approach. *Aquaculture*, 484, 82–89. <https://doi.org/10.1016/j.aquaculture.2017.10.044>
- Natividad, A. N., Miranda, C., Valdoria, J. C., & Balubal, D. (2023). An IoT based pH level monitoring mobile application on fishponds using pH sensor and waterproof temperature sensor. *Journal for Educators, Teachers and Trainers*, 14(3), 798–805. [https://digibug.ugr.es/bitstream/handle/10481/83506/798-805\\_JETT1403091.pdf?sequence=1](https://digibug.ugr.es/bitstream/handle/10481/83506/798-805_JETT1403091.pdf?sequence=1)
- Nur'aini, D. A. K. D., & Junianto. (2021). Influence of reference feed on the growth of freshwater crayfish. *Global Scientific Journal*, 9(4). [https://www.globalscientificjournal.com/researchpaper/Influence\\_of\\_Reference\\_Feed\\_on\\_The\\_Growth\\_of\\_Freshwater\\_Crayfish.pdf](https://www.globalscientificjournal.com/researchpaper/Influence_of_Reference_Feed_on_The_Growth_of_Freshwater_Crayfish.pdf)
- Olanubi, O. O., Akano, T. T., & Asaolu, O. S. (2024). Design and development of an IoT-based intelligent water quality management system for aquaculture. *Journal of Electrical Systems and Information Technology*, 11, Article 15. <https://doi.org/10.1186/s43067-024-00139-z>



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- Parra, L., Rocher, J., Escrivá, J., & Lloret, J. (2018). Design and development of low cost smart turbidity sensor for water quality monitoring in fish farms. *Aquacultural Engineering*, 81. <https://doi.org/10.1016/j.aquaeng.2018.01.004>
- Pascale, N., Azzena, I., Locci, C., Deplano, I., Orrù, F., Puzzi, C., Are, F., Scarpa, F., Sanna, D., & Casu, M. (2024). New insight into the crayfish *Procambarus clarkii* (Girard, 1852) (Crustacea, Cambaridae): A morphometric combined approach to describe the case of a Mediterranean population. *Animals*, 14(24), 3558. <https://doi.org/10.3390/ani14243558>
- Paul, L., Yadav, V. K., Bhowmik, T., et al. (2026). IoT and ML for identification and behavioural analysis in shrimp aquaculture. *Discover Sustainability*. <https://doi.org/10.1007/s43621-026-03086-z>
- Prapti, D. R., Shariff, A. R. M., Man, H. C., Ramli, N. M., Perumal, T., & Shariff, M. (2022). Internet of Things (IoT)-based aquaculture: An overview of IoT application on water quality monitoring. *Reviews in Aquaculture*. <https://doi.org/10.1111/raq.12637>
- Pratiwy, & Aisyah, A. (2026). Literature review: The role of IoT and AI in water quality monitoring in aquaculture. *GPH-International Journal of Applied Science*, 9(1), 98–111. <https://doi.org/10.5281/zenodo.18632014>
- Rastegari, H., Nadi, F., Lam, S. S., Ikhwanuddin, M., Kasan, N. A., Rahmat, R. F., & Wan Mahari, W. A. (2023). Internet of Things in aquaculture: A review of the challenges and potential solutions based on current and future trends. *Smart Agricultural Technology*, 4, 100187. <https://doi.org/10.1016/j.atech.2023.100187>
- Rigg, D. P., Seymour, J. E., Courtney, R. L., & Jones, C. M. (2020). A review of juvenile redclaw crayfish *Cherax quadricarinatus* (von Martens, 1898) production in aquaculture. *Freshwater Crayfish*, 25(1), 13–30. <https://doi.org/10.5869/fc.2020.v25-1.013>
- Shete, R. P., Bongale, A. M., & Dharrao, D. (2024). IoT-enabled effective real-time water quality monitoring method for aquaculture. *MethodsX*, 13, Article 102906. <https://doi.org/10.1016/j.mex.2024.102906>
- Stumpf, L., & López Greco, L. S. (2015). Compensatory Growth in Juveniles of Freshwater Redclaw Crayfish *Cherax quadricarinatus* Reared at Three Different Temperatures: Hyperphagia and Food Efficiency as Primary Mechanisms. *PloS one*, 10(9), e0139372. <https://doi.org/10.1371/journal.pone.0139372>
- Sundararajan, S. C. M., Shankar, Y. B., Selvam, S. P., et al. (2025). IoT-based prediction model for aquaponic fish pond water quality using multiscale feature fusion with



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- convolutional autoencoder and GRU networks. *Scientific Reports*, 15, 1925.  
<https://doi.org/10.1038/s41598-024-84943-7>
- Suriasni, P. A., Faizal, F., Hermawan, W., Subhan, U., Panatarani, C., & Joni, I. M. (2024). IoT water quality monitoring and control system in moving bed biofilm reactor to reduce total ammonia nitrogen. *Sensors*, 24(2), 494.  
<https://doi.org/10.3390/s24020494>
- Suwardi Ansyah, A. S., Arifin, M., & Laili, U. (2024). Applying fuzzy logic and IoT for intelligent automation in crayfish water quality control. *Jurnal Ilmiah KURSOR*, 12(3), 101–110. <https://doi.org/10.21107/kursor.v12i3.334>
- Malandrakis, E. E. (2025). Design and implementation of a cost-effective IoT-based monitoring and alerting system for recirculating aquaculture systems (RAS). *Sensors*, 25(21), 6692. <https://doi.org/10.3390/s25216692>
- Vo, T. T. E., Ko, H., Huh, J.-H., & Kim, Y. (2021). Overview of smart aquaculture system: Focusing on applications of machine learning and computer vision. *Electronics*, 10(22), 2882. <https://doi.org/10.3390/electronics10222882>
- Wang, J., Wang, Z., Wang, Q., & Zhou, Z. (2023). Ecological index analysis of growth and development of *Procambarus clarkii* based on biological characteristics. *Journal of Sea Research*, 192, 102363. <https://doi.org/10.1016/j.seares.2023.102363>
- Yong-chun, W., Shun, C., Dan-li, W., Mei-li, C., Jian-bo, Z., Yong-yi, J., Fei, L., Shi-li, L., Yi-nuo, L., Zhi-min, G. (2022). The effect of ammonia nitrogen, nitrite and pH on artificial incubation of red claw crayfish *Cherax quadricarinatus* eggs and growth of juveniles. *Aquaculture Research*, 53, 3788–3796.  
<https://doi.org/10.1111/are.15885>
- Zeng, Q., Luo, M., Qin, L., Guo, C., et al. (2024). Growth performance, enzyme activities and metabolite level response to low pH stress in juvenile red swamp crayfish *Procambarus clarkii*. *Aquaculture Reports*, 39, 102569.  
<https://doi.org/10.1016/j.aqrep.2024.102481>
- Zhu, Y., Xu, Y., Lu, S., Hu, Y., Xu, H., Li, J., Lin, H., Li, X., & Xu, Z. (2026). Molecular insights into exoskeletal remodeling: Transcriptomic profiling of the molting cycle in the red swamp crayfish *Procambarus clarkii*. *Fishes*, 11(3), 166.  
<https://doi.org/10.3390/fishes11030166>
- Zou, Y., Zhang, C. Y., Cao, X. T., Niu, R. G., & Lan, J. F. (2026). Transcriptomic and microbiome analyses of *Procambarus clarkii* exposed to different doses of 20E. *Biology*, 15(5), 434. <https://doi.org/10.3390/biology15050434>
- Zuhaer, A., Khandoker, A., Enayet, N., Partha, P. K. P., & Awal, M. A. (2025). Sustainable aquaculture: An IoT-integrated system for real-time water quality monitoring



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

featuring advanced DO and ammonia sensors. Aquacultural Engineering, 112, 102620. <https://doi.org/10.1016/j.aquaeng.2025.102620>

### B. Conference Papers

Abdikadir, N. M., Abdullah, A. S., Abdullahi, H. O., & Hassan, A. A. (2024). Smart aquaculture: IoT-enabled monitoring and management of water quality for Mahseer fish farming. SSRG International Journal of Electrical and Electronics Engineering, 11(11), 84–92. <https://doi.org/10.14445/23488379/IJEEE-V11I11P109>

Adriman, R., Fitria, M., Afdhal, A., & Fernanda, A. Y. (2022). An IoT-based system for water quality monitoring and notification system of aquaculture prawn pond. 2022 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT), 356–360. <https://doi.org/10.1109/COMNETSAT56033.2022.9994388>

Abrajano, J. V., Botangen, K. A., Nabua, J., Apanay, J., & Peña, C. F. (2024). IoT-based water quality monitoring system in Philippine off-grid communities. arXiv preprint arXiv:2410.14619. <https://doi.org/10.48550/arXiv.2410.14619>

Boumehezz, F., Sahour, A., Maamri, F., Djellab, H., & Bekhouche, A. (2025). IoT-enabled fuzzy logic system for aquaculture water quality management. 2025 IEEE International Conference on Advanced Systems and Emergent Technologies (IC\_ASET), 1–6. [https://doi.org/10.1109/IC\\_ASET65966.2025.11232273](https://doi.org/10.1109/IC_ASET65966.2025.11232273)

Darmalim, F., Hidayat, A. A., Cenggoro, T. W., Purwandari, K., Darmalim, S., & Pardamean, B. (2021). An integrated system of mobile application and IoT solution for pond monitoring. IOP Conference Series: Earth and Environmental Science, 794(1), 012106. <https://doi.org/10.1088/1755-1315/794/1/012106>

Krishna Reddy, D., Siva Priyanka, S., Sai Kumar, A., Kunduru, J., & Batta, N. (2023). IoT based water quality monitoring for smart aquaculture. In 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT) (pp. 1–6). IEEE. <https://doi.org/10.1109/ICCCNT56998.2023.10307651>

Libao, F. J. D., et al. (2024). Automated control and IoT-based water quality monitoring system for a Molobicus tilapia recirculating aquaculture system (RAS). 2024 IEEE Conference on Technologies for Sustainability (SusTech), 410–415. <https://doi.org/10.1109/SusTech60925.2024.10553538>



## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

Marselina, S., Fakhrurroja, H., & Sari, B. (2023). Designing an IoT-based freshwater lobster cultivation monitoring dashboard. <https://doi.org/10.4108/eai.11-10-2022.2326420>

San Antonio, D. T., Rivera, J. R. A., Balid, A. C. N., Belaos, R. R. P., Brizuela, A. I., & Caballero, J. A. (2023). IoT-based water quality monitoring and automated fish feeder: Enhancing aquaculture productivity. 2023 IEEE Region 10 Symposium (TENSYP), 1–6. <https://doi.org/10.1109/TENSYP55890.2023.10223636>

Shitsukane, A., Egessa, M., Orina, P., Omieno, K., & Amuti, M. (2024). Integrating fuzzy logic, solar power and IoT for real-time water quality monitoring in cage fish culture. In 2024 IST-Africa Conference (IST-Africa) (pp. 1–10). IEEE. <https://doi.org/10.23919/IST-Africa63983.2024.10569527>

### C. Government & Institutional Documents

BFAR Administrative Circular No. 001, s. 2025. (2025). Guidelines on the culture of Australian Redclaw Crayfish (*Cherax quadricarinatus*) in the Philippines. Bureau of Fisheries and Aquatic Resources. <https://www.bfar.da.gov.ph/wp-content/uploads/2025/11/BFAR-Administrative-Circular-on-Crayfish-Culture.pdf>

Business Queensland. (2025). Redclaw crayfish aquaculture. Queensland Government. <https://www.business.qld.gov.au/industries/farms-fishing-forestry/fisheries/aquaculture/species/redclaw-crayfish>

Queensland Government. (2025). Redclaw crayfish aquaculture. Business Queensland. <https://www.business.qld.gov.au/industries/farms-fishing-forestry/fisheries/aquaculture/species/redclaw-crayfish>

Masser, M. P., & Rouse, D. B. (1997). Australian red claw crayfish (SRAC Publication No. 244). Southern Regional Aquaculture Center. <https://srac.msstate.edu/pdfs/Fact%20Sheets/244%20Australian%20Red%20Claw%20Crayfish.pdf>

Republic Act No. 8550. (1998). An act providing for the development, management and conservation of the fisheries and aquatic resources, integrating all laws pertinent thereto, and for other purposes (as amended by Republic Act No. 10654). Official Gazette of the Republic of the Philippines. <https://www.officialgazette.gov.ph/1998/02/25/republic-act-no-8550/>

Republic Act No. 10654. (2015). An act to prevent, deter and eliminate illegal, unreported and unregulated fishing, amending Republic Act No. 8550, otherwise known as "The Philippine Fisheries Code of 1998," as amended. Official Gazette of the Republic of the Philippines. <https://www.officialgazette.gov.ph/2015/02/27/republic-act-no-10654/>





## POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

- Ruscoe, I. (2002, November). Redclaw crayfish aquaculture (*Cherax quadricarinatus*) (Fishnote No. 32). Northern Territory Government, Department of Primary Industry, Fisheries and Mines. <https://daf.nt.gov.au/publications/publications-search/publications-database/fisheries/aquaculture-and-fishing/fishnote/fn32.pdf>
- U. S. Fish and Wildlife Service. (2020). Ecological risk screening summary: Redclaw (*Cherax quadricarinatus*). <https://www.fws.gov/sites/default/files/documents/Ecological-Risk-Screening-Summary-Redclaw.pdf>

### D. Standards

- International Organization for Standardization. (2011). Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models (ISO/IEC Standard No. 25010:2011). <https://www.iso.org/standard/35733.html>

### E. Web Articles / Online Sources

- AlpHa Measurement Solutions. (n.d.). Turbidity in aquaculture, agriculture & hydroponics. <https://alpha-measure.com/turbidity-in-aquaculture-agriculture-hydroponics/alpha-measure.com>
- FAO. (2011). Cultured aquatic species information programme — *Cherax quadricarinatus* von Martens 1868 (Text by Jones, C.). Fisheries and Aquaculture. [http://www.fao.org/fishery/culturedspecies/cherax\\_quadricarinatus/en](http://www.fao.org/fishery/culturedspecies/cherax_quadricarinatus/en)
- The Fish Site. (2012, November 5). How to farm red claw crayfish. <https://thefishsite.com/articles/cultured-aquatic-species-red-claw-crayfish/thefishsite.com>



POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

**CAPSTONE PROJECT**

192